Data Parallelism in LLM Training

Spring 2026

Lecturer: Yuedong (Steven) Xu

Fudan University

ydxu@fudan.edu.cn

Disclaimer

Machine learning systems is a broad and rapidly evolving field. The course material has been developed using a broad spectrum of resources, including research papers, lecture slides, blogposts, research talks, tutorial videos, and other materials shared by the research community. We sincerely appreciate their efforts and assistance, and try our best to cite the sources of the materials used in this course.

Distributed LLM Training: Outline

- **Transformer: A Quick Overview**
- Data Parallelism
  - Parameter-Server
  - All-Reduce
  - Memory Optimization

Transformer Overview

- Transformer everywhere
  - Basic object in Transformer library: ***pipeline()*** which connects a model with its necessary preprocessing and postprocessing steps

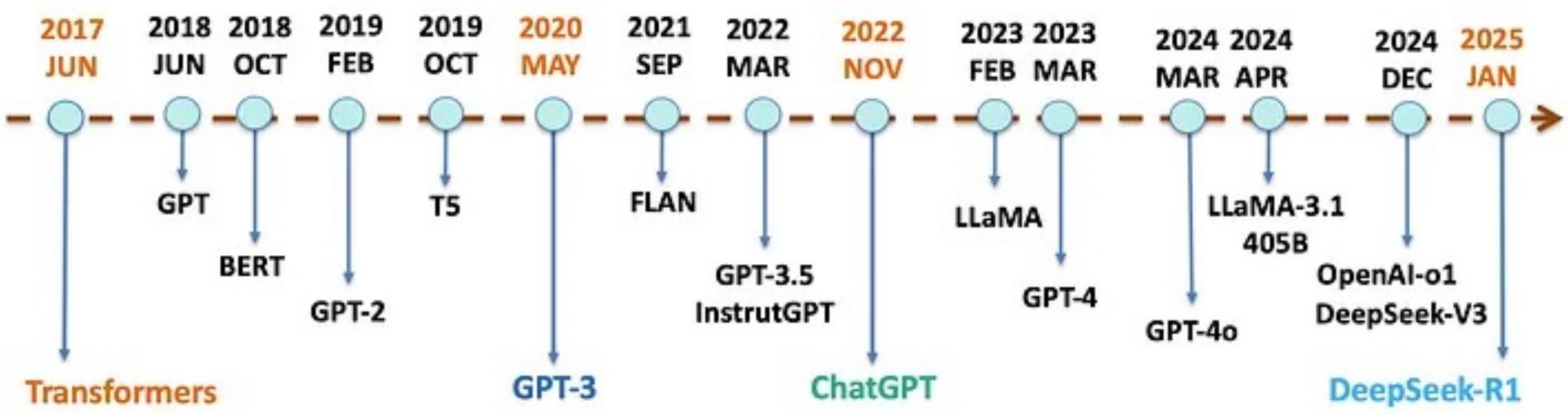
```
from transformers import pipeline
classifier = pipeline("sentiment-analysis")
classifier("I've been waiting for a machine learning system course my whole life.")

[{'label': 'POSITIVE', 'score': 0.9598047137260437}]
```

  - Other pipelines
    - *text-generation, text-classification, summarization, translation, feature-extraction*
    - *image-to-text, image-classification, object-detection*
    - *automatic-speech-recognition, text-to-speech, audio-classification*

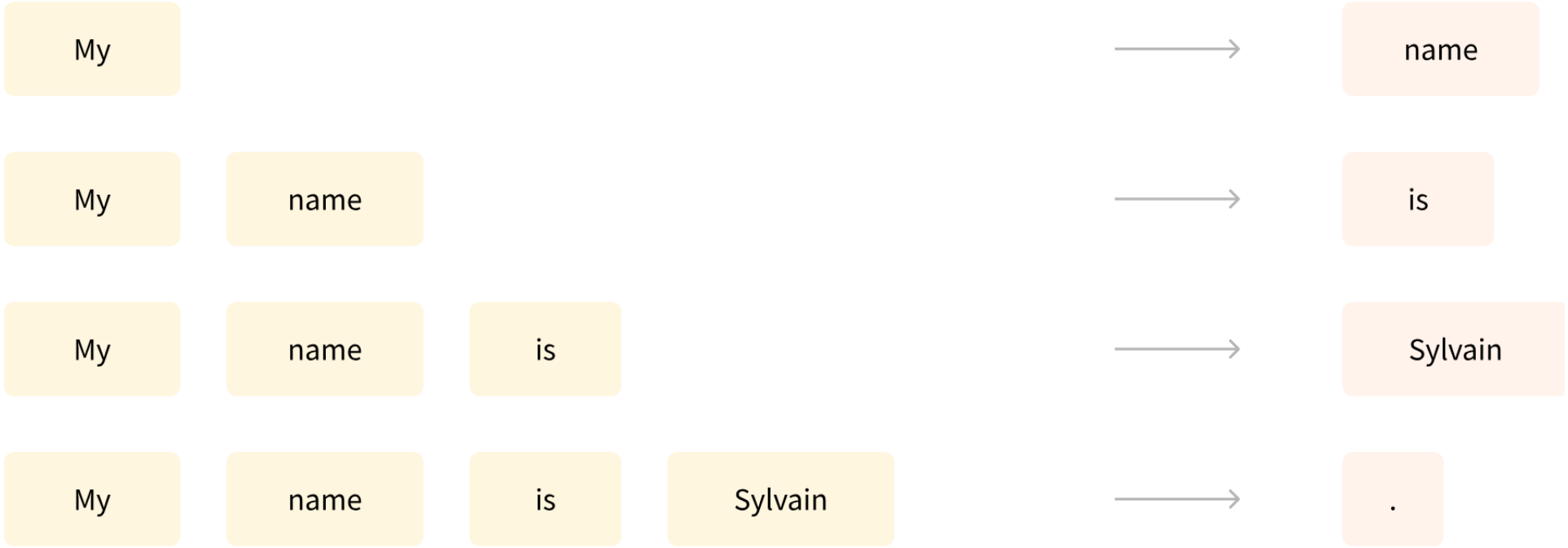
Transformer Overview

- A brief history of Transformer models



Transformer Overview

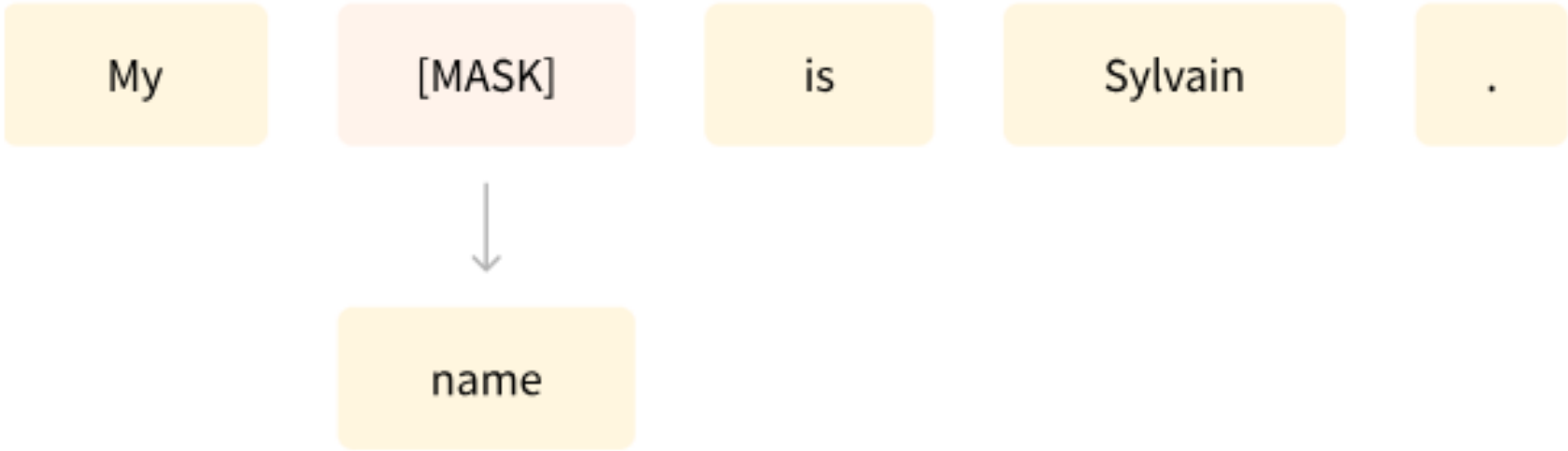
- Causal language model



Predicting the next word in a sentence having read a number of previous words

Transformer Overview

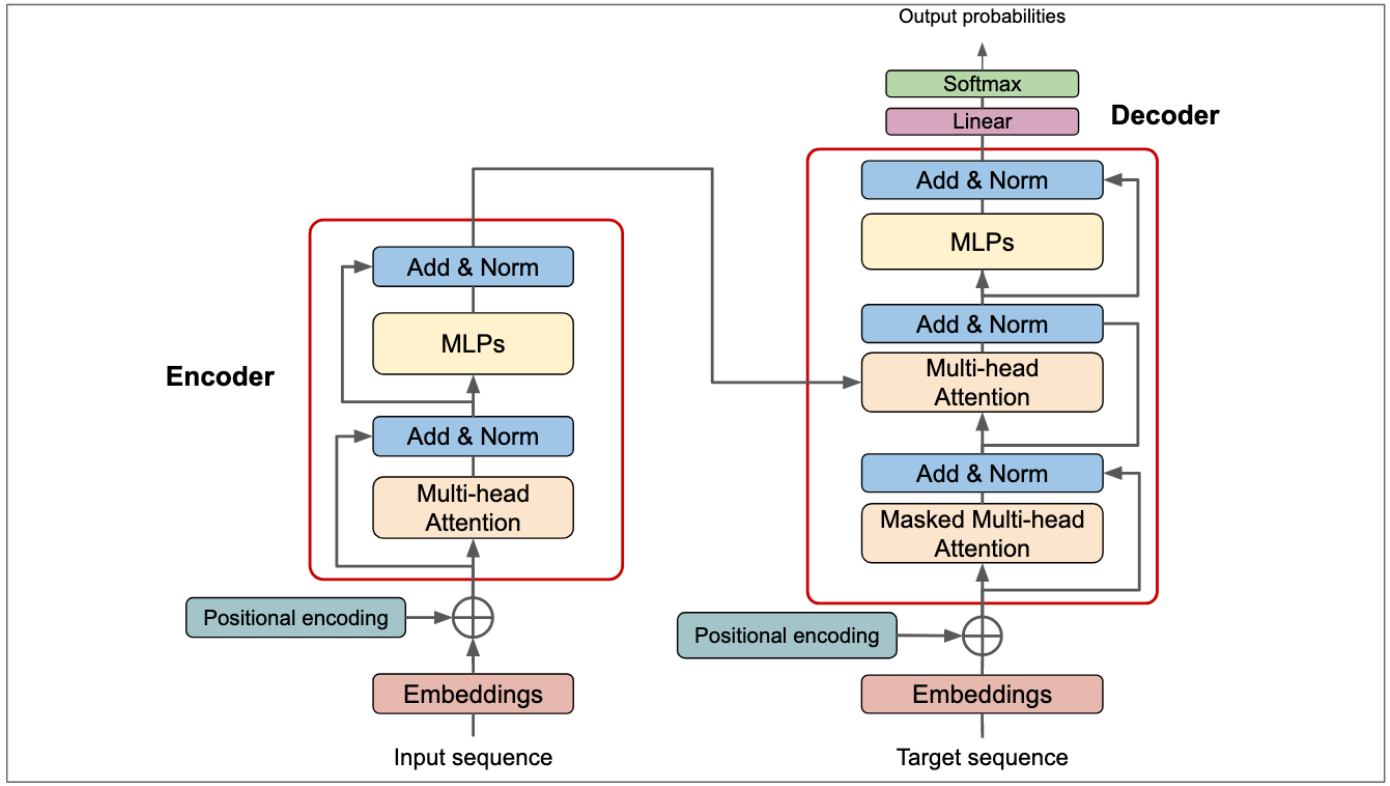
- Non-causal language model



Masked language modeling in which the model predicts a masked word in the sentence

Transformer Overview

- Transformer architecture



A minimalist description of Transformer *

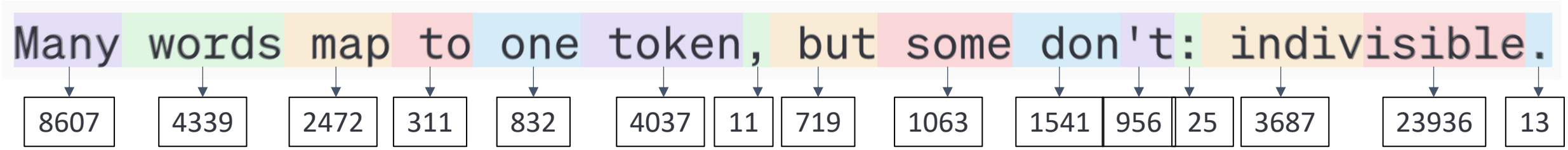
Encoder (left): receiving an input and building a representation of it (its features), i.e. acquire understanding from the input.

Decoder (right): utilizing the encoder's representation along with other inputs to generate a target sequence, i.e. generating outputs.

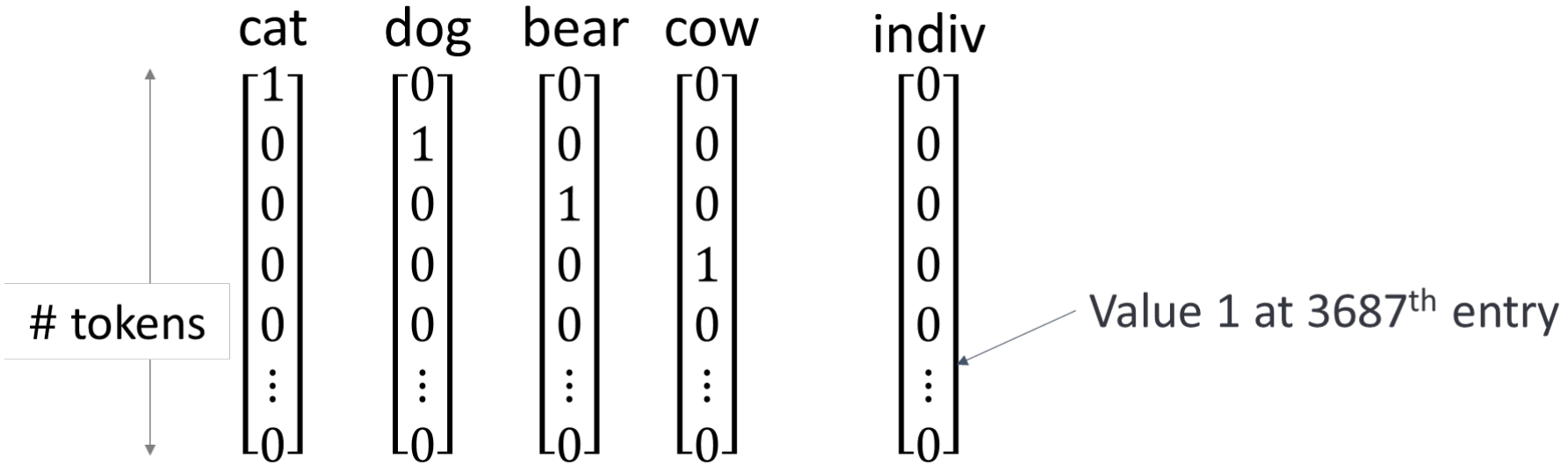
* https://re-cinq.com/blog/llm-architectures

Transformer Overview

- Sequence to tokens

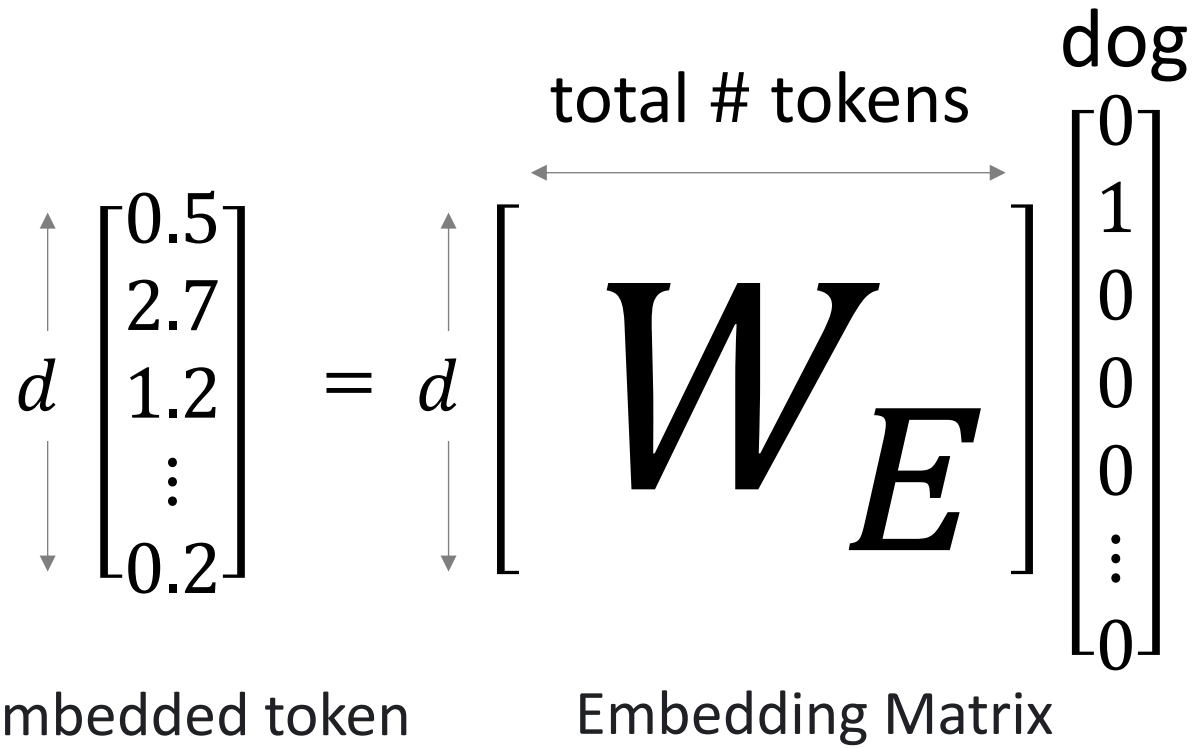


One-hot encoding



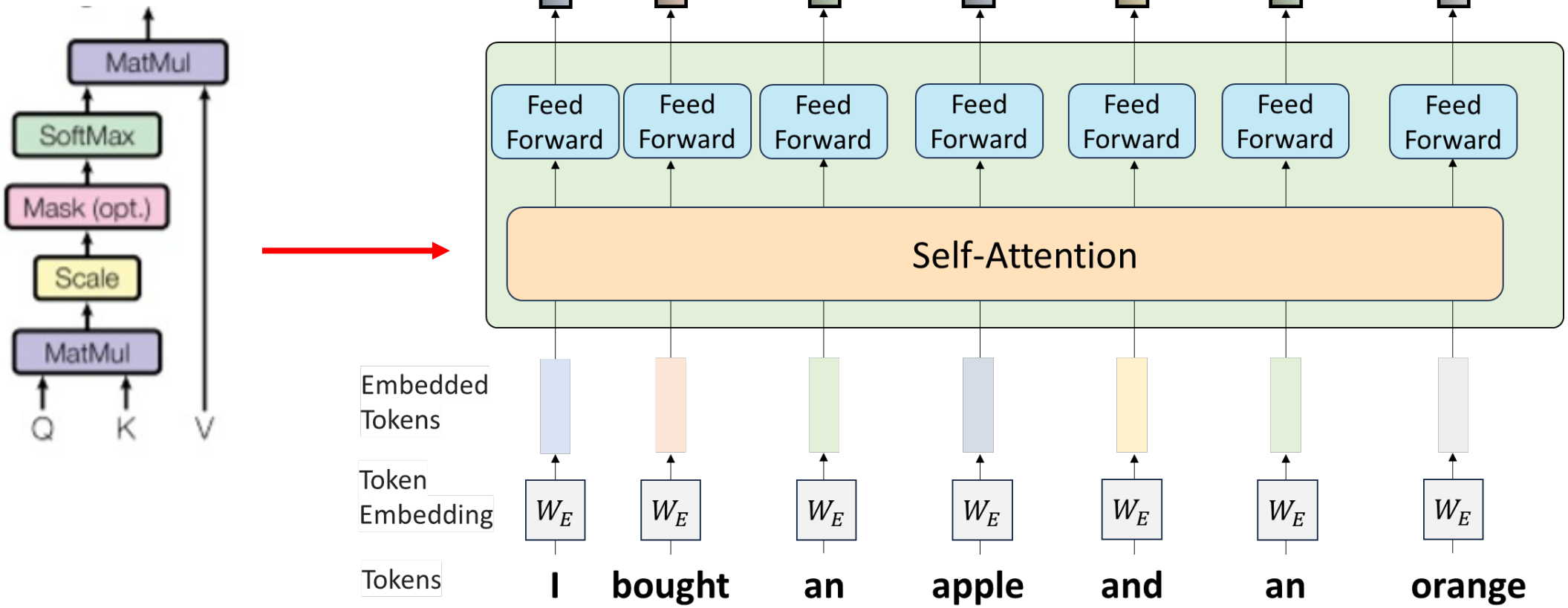
Transformer Overview

- Token embedding
  - Compressing a high dimensional vector into a lower one
  - Multiplying one-hot encoded vector by embedding matrix (index lookup)



Transformer Overview

- Attention computing



Transformer Overview

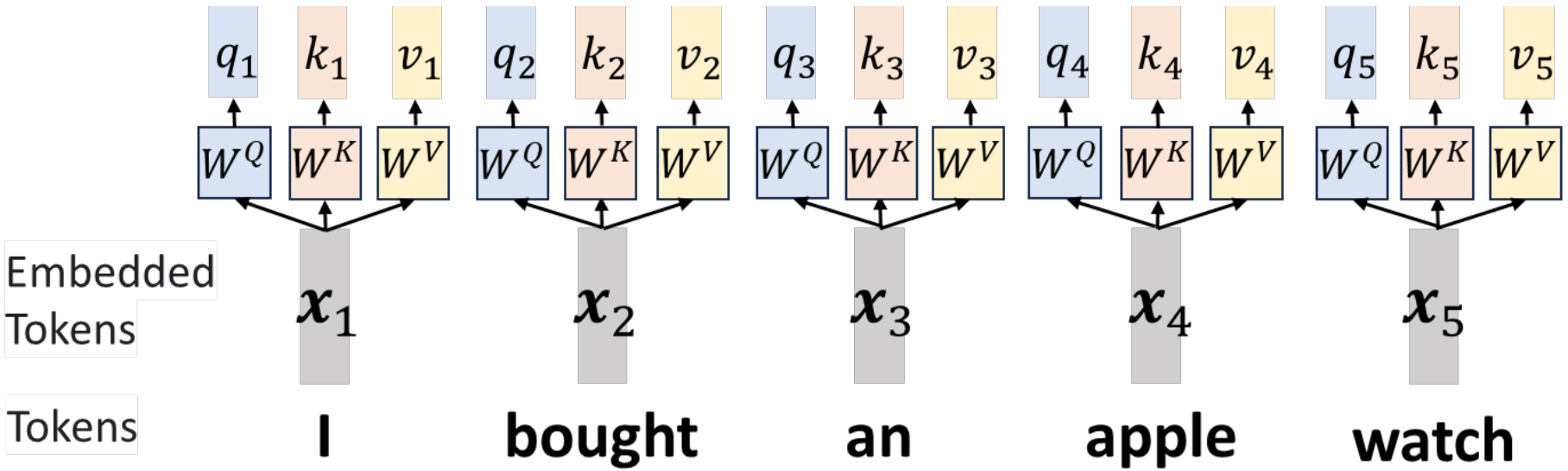
- Attention computing

$W^Q \in R^{d \times d_k}$

$W^K \in R^{d \times d_k}$

$W^V \in R^{d \times d_v}$

$$Q = \begin{matrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{matrix} W^Q \qquad K = \begin{matrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{matrix} W^K \qquad V = \begin{matrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{matrix} W^V$$



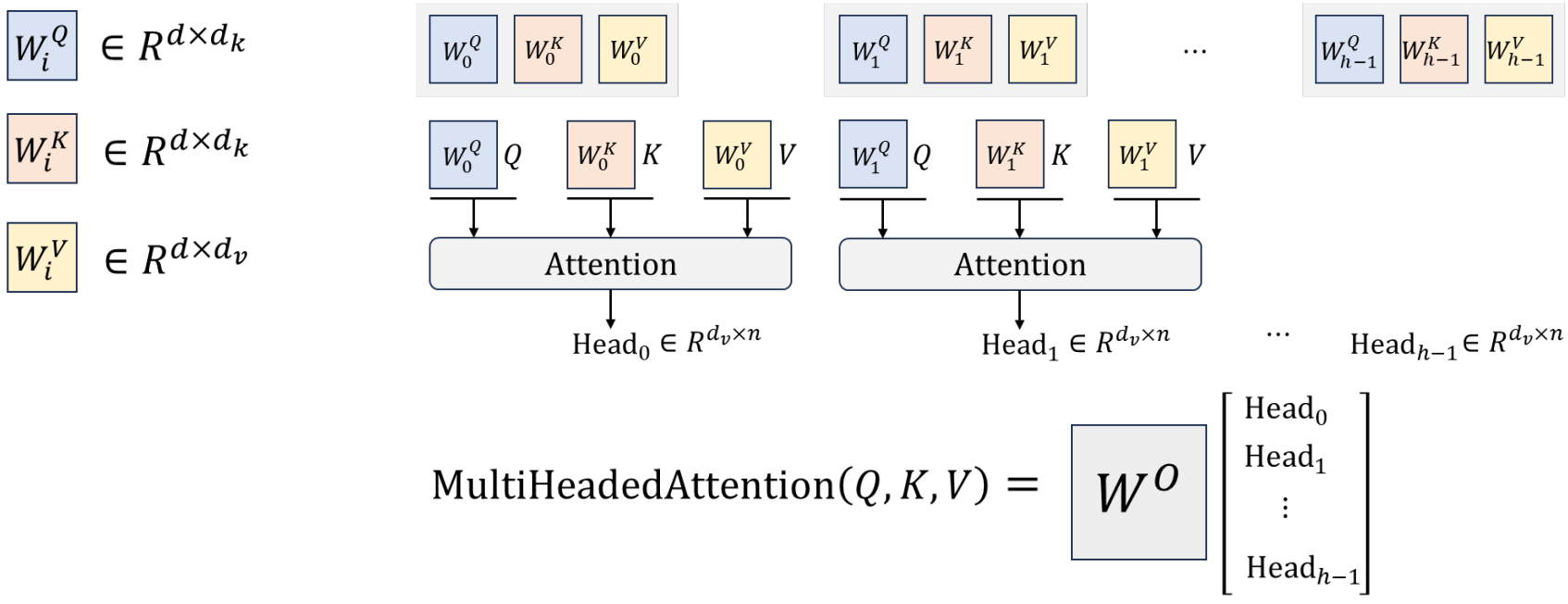
* LayerNorm and residual omitted; Slides from Prof. Kai-Bin Huang

Transformer Overview

- Attention computing
  - Single head ($d_k = d$)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{K^\top Q}{\sqrt{d_k}}\right)V$$

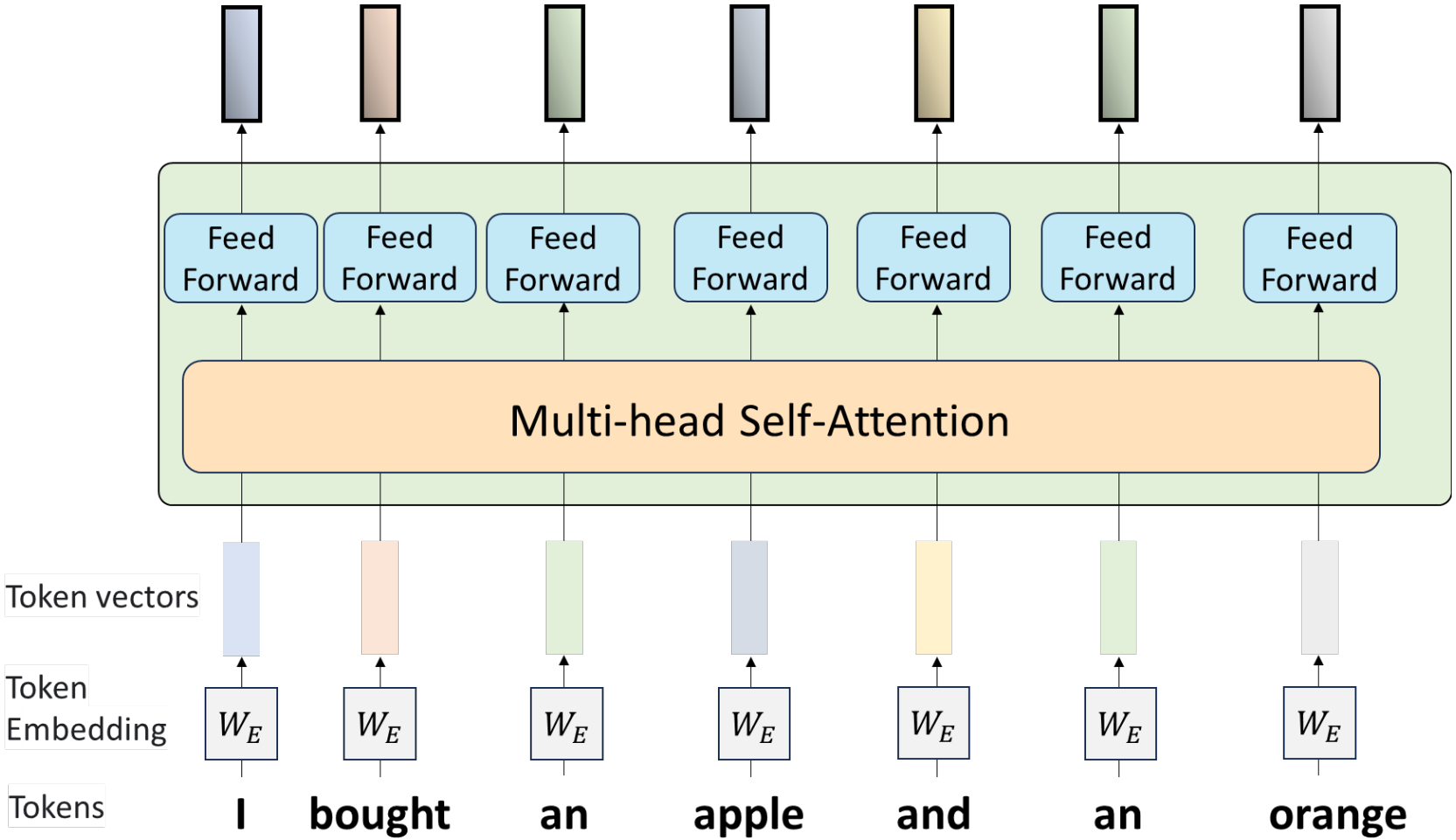
  - Multi-head ($d_k = d/h$)



$$\text{MultiHeadedAttention}(Q, K, V) = W^O \begin{bmatrix} \text{Head}_0 \\ \text{Head}_1 \\ \vdots \\ \text{Head}_{h-1} \end{bmatrix}$$

Transformer Overview

- FFN computing

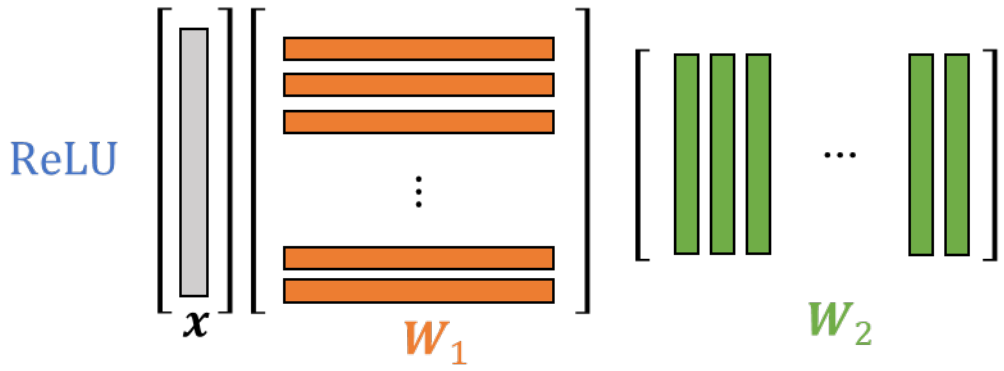


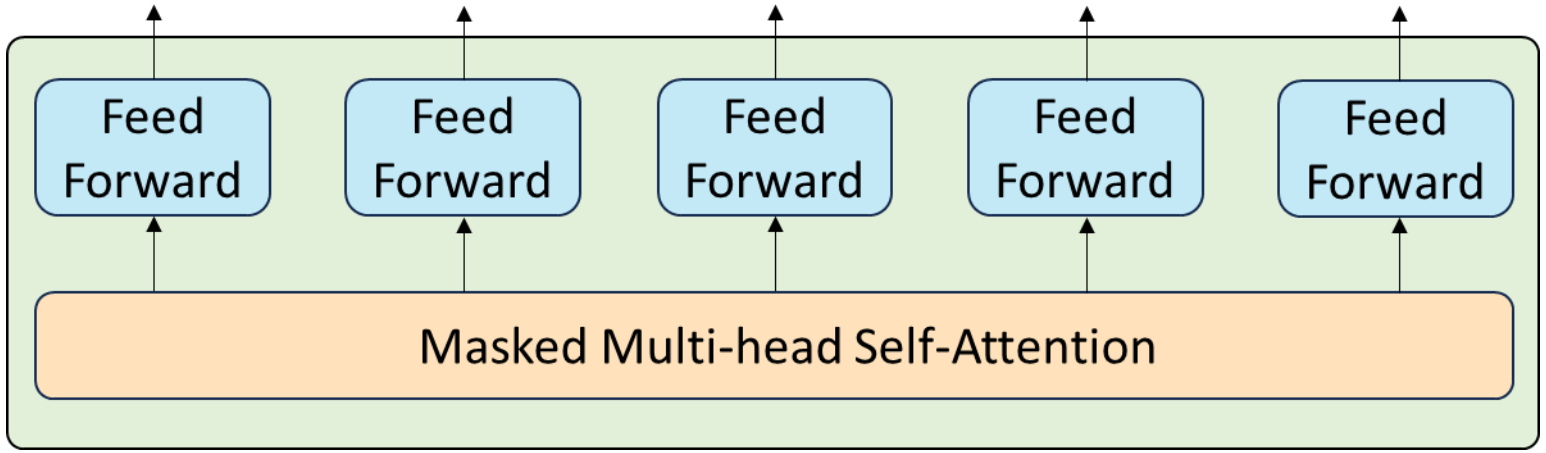
* LayerNorm and residual omitted; Slides from Prof. Kai-Bin Huang

Transformer Overview

- FFN computing

$$FFN(\boldsymbol{x}) = \mathrm{ReLU}(\boldsymbol{x}\boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2$$

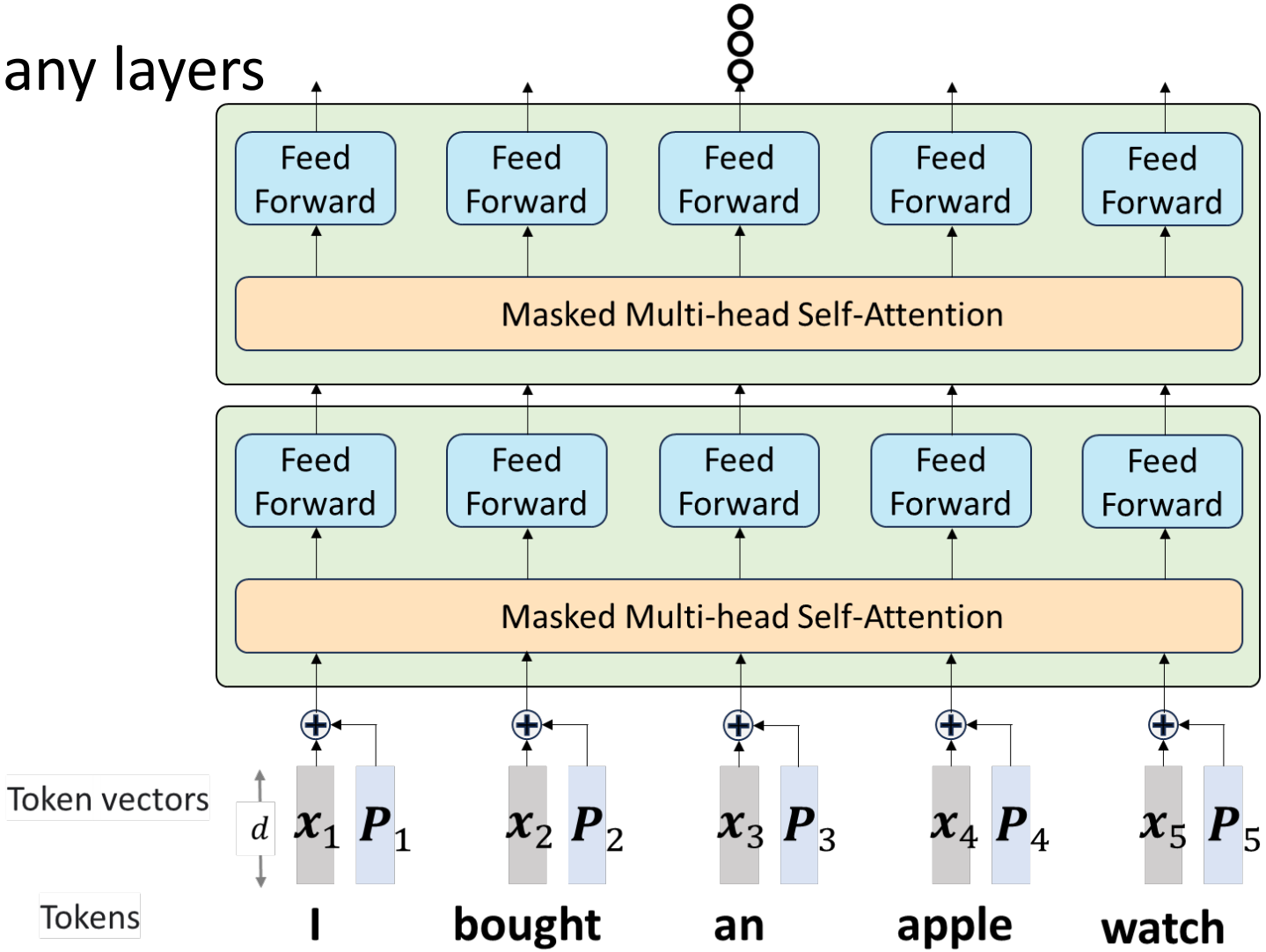




* LayerNorm and residual omitted; Slides from Prof. Kai-Bin Huang

Transformer Overview

- Stacking many layers



* LayerNorm and residual omitted; Slides from Prof. Kai-Bin Huang

Transformer Overview

- Encoder versus decoder

Encoder:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Decoder:

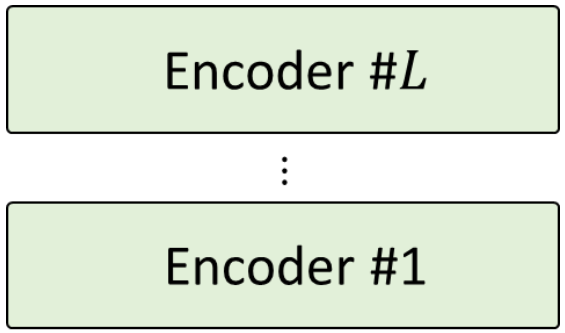
$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

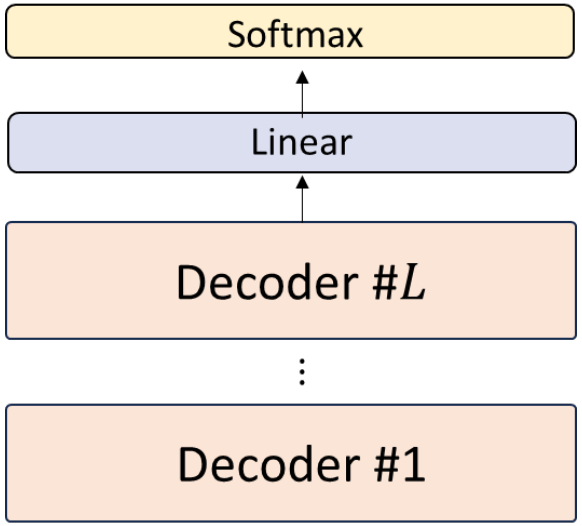
* LayerNorm and residual omitted; Slides from Prof. Kai-Bin Huang

Transformer Overview

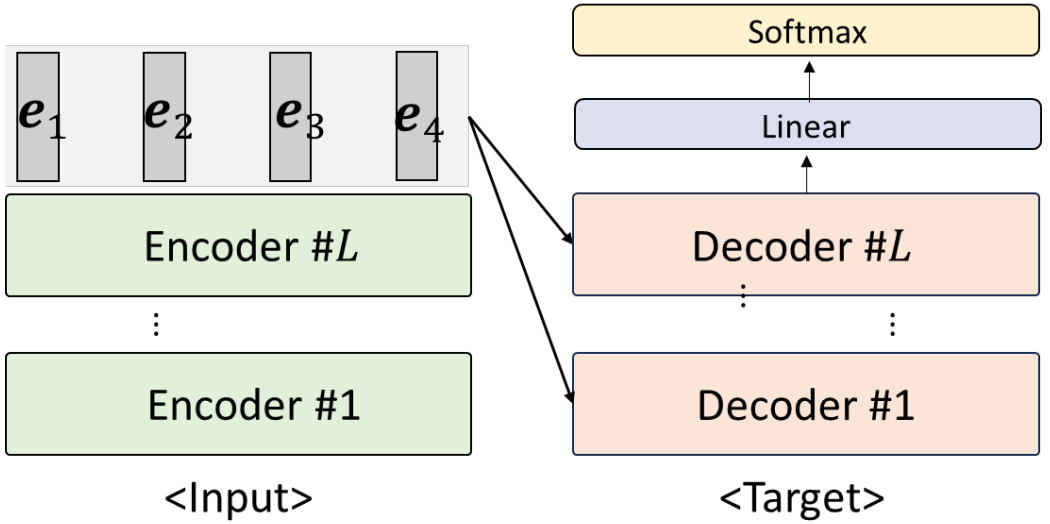
- Encoder-only, decoder-only and encoder-decoder



Encoder-only



Decoder-only



Encoder-decoder

* LayerNorm and residual omitted; Slides from Prof. Kai-Bin Huang

Transformer Overview

- Output
  - Linear projection

$$\text{Logits} = x \cdot W^T + b$$

  - Softmax to probability distribution

$$P(w_i) = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

  - Decoding/Sampling to predict a word

* LayerNorm and residual omitted; Slides from Prof. Kai-Bin Huang

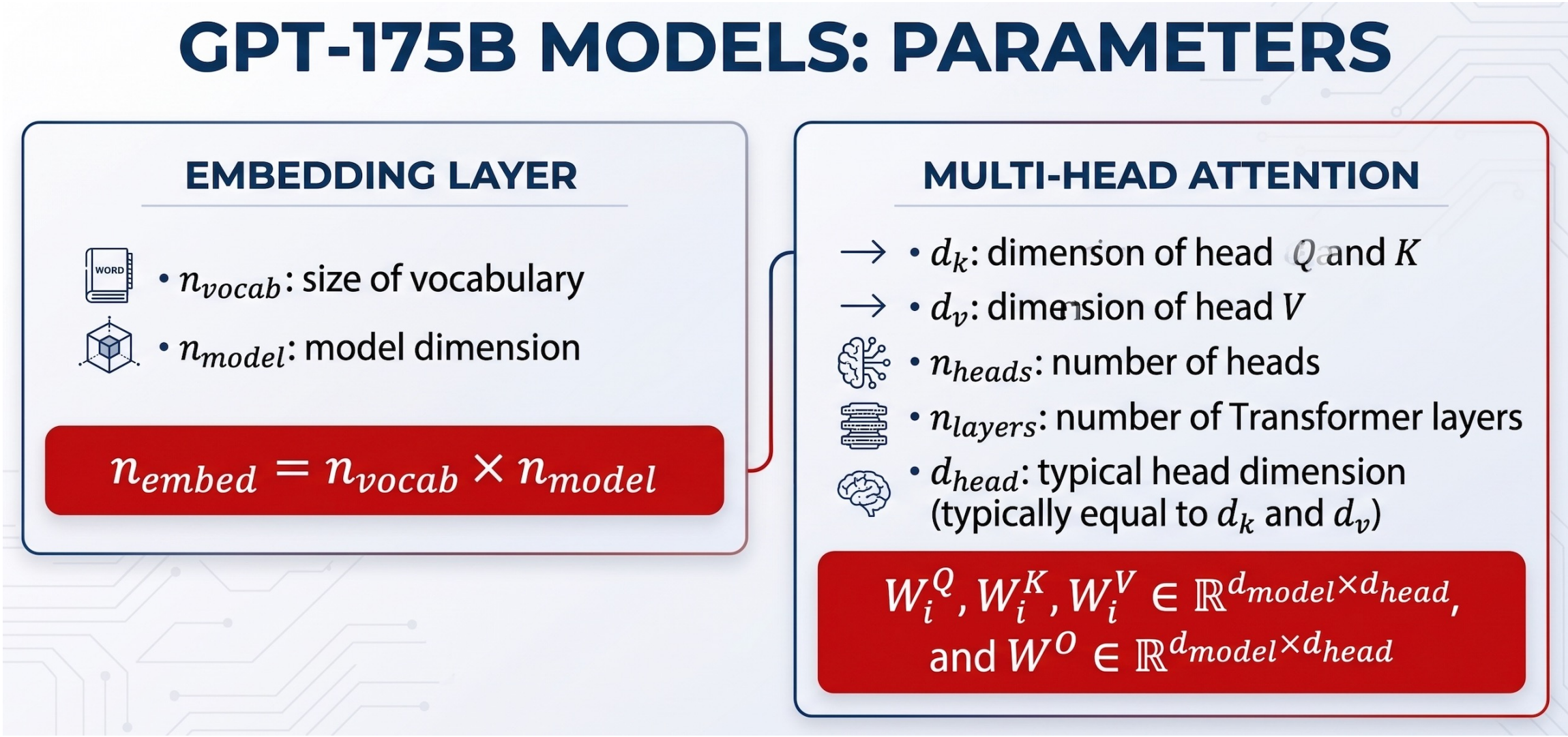
Distributed LLM Training: Outline

- What to memorize
  - Computation flow instead of functionalities of different modules
  - Computational cost and memory consumption

- What to know beforehand
  - Block matrix multiplication
  - Some basic software, hardware and networking knowledge

Why do we need “distributed training”?

Quantitative Analysis

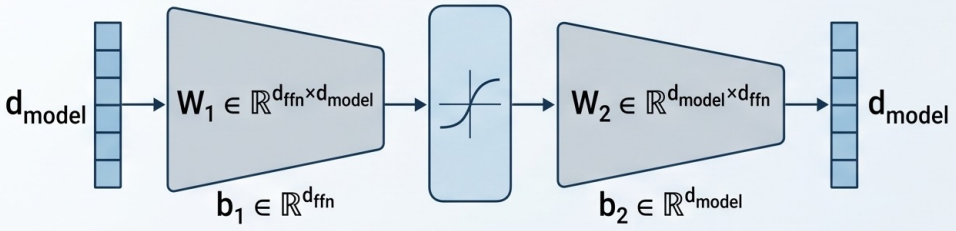


Quantitative Analysis

GPT Models: Parameter Counting

Multi-Layer Perceptron (MLP)

Parameter Breakdown



d_{ffn}: Hidden Dimension of Fully Connected Layer

Parameters: $W_1 \in \mathbb{R}^{d_{ffn} \times d_{model}}$, $b_1 \in$ in $\mathbb{R}^{d_{morel}}$, $b_2 \in \mathbb{R}^{d_{model}}$

$$n_{MLP} = 2d_{ffn} \times d_{model} + d_{ffn} + d_{model}$$

Simplified Total Parameter Count

(Excluding Biases)

$$n_{total} = n_{vocab} d_{model} + n_{layers} (4d^2_{model} + 2d_{ffn} d_{model})$$

*This formula counts key parameter matrices and input token embeddings, excluding all biases and positional embeddings.

Simplified Total Parameter Count

(Excluding Biases)

(Variable d_{model} is uofl hidden dimensiout for consistency)

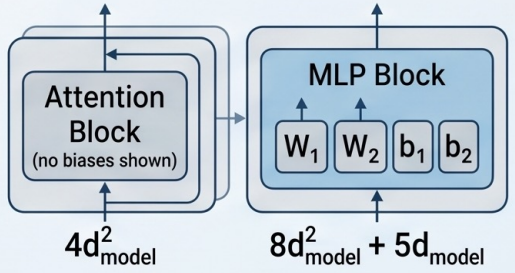
$$n_{total} = n_{vocab} d_{model} + n_{layers} (4d^2_{model} + 2d_{ffn} d_{model})$$

*This formula counts key parameter matrices and input token embeddings, excluding all biases and positional embeddings.

Parameter Estimation for Standard GPT Models

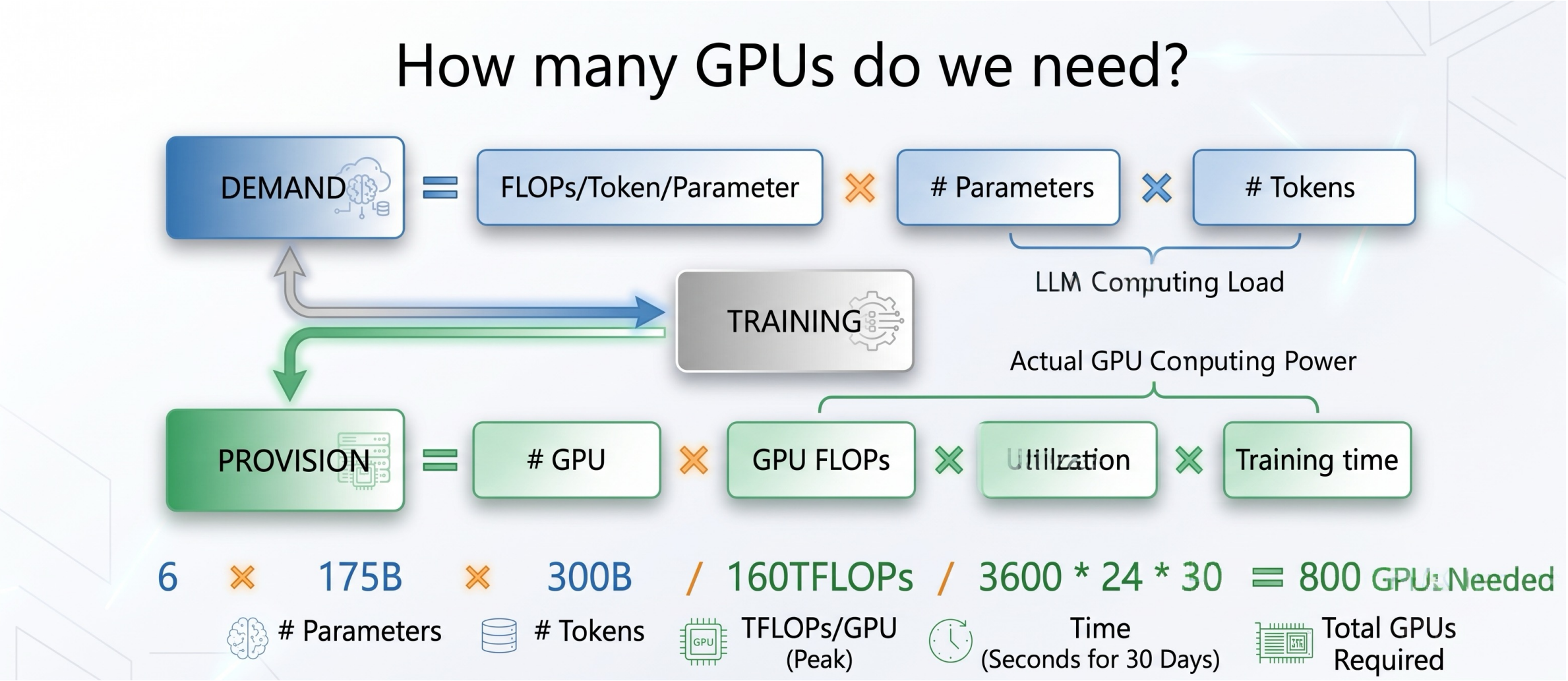
- $d_{model} = d_{head} \times n_{heads}$
- $d_{ffn} = 4\, d_{model}$

Note: Today's largest models often use different ratios.



$$n_{total} = n_{vocab} d_{model} + n_{layers} (12d^2_{model} + 5d_{model})$$

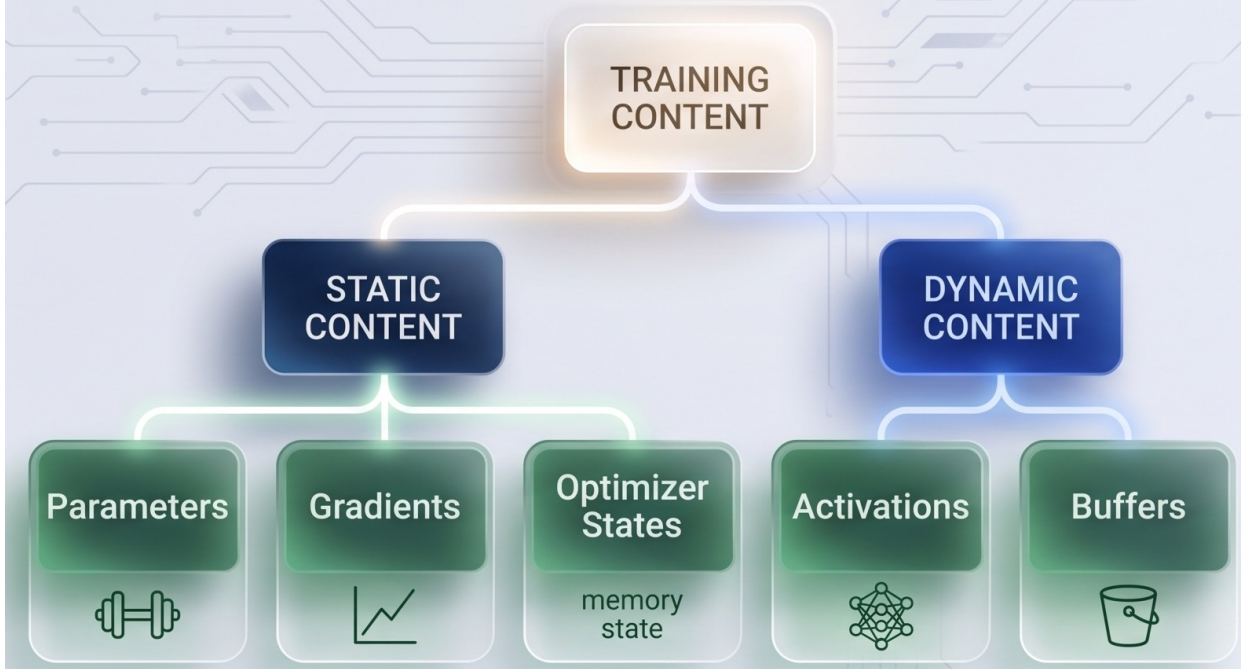
Quantitative Analysis



《Language Models are Few-Shot Learners》
《Efficient large-scale language model training on GPU clusters using megatron-LM》

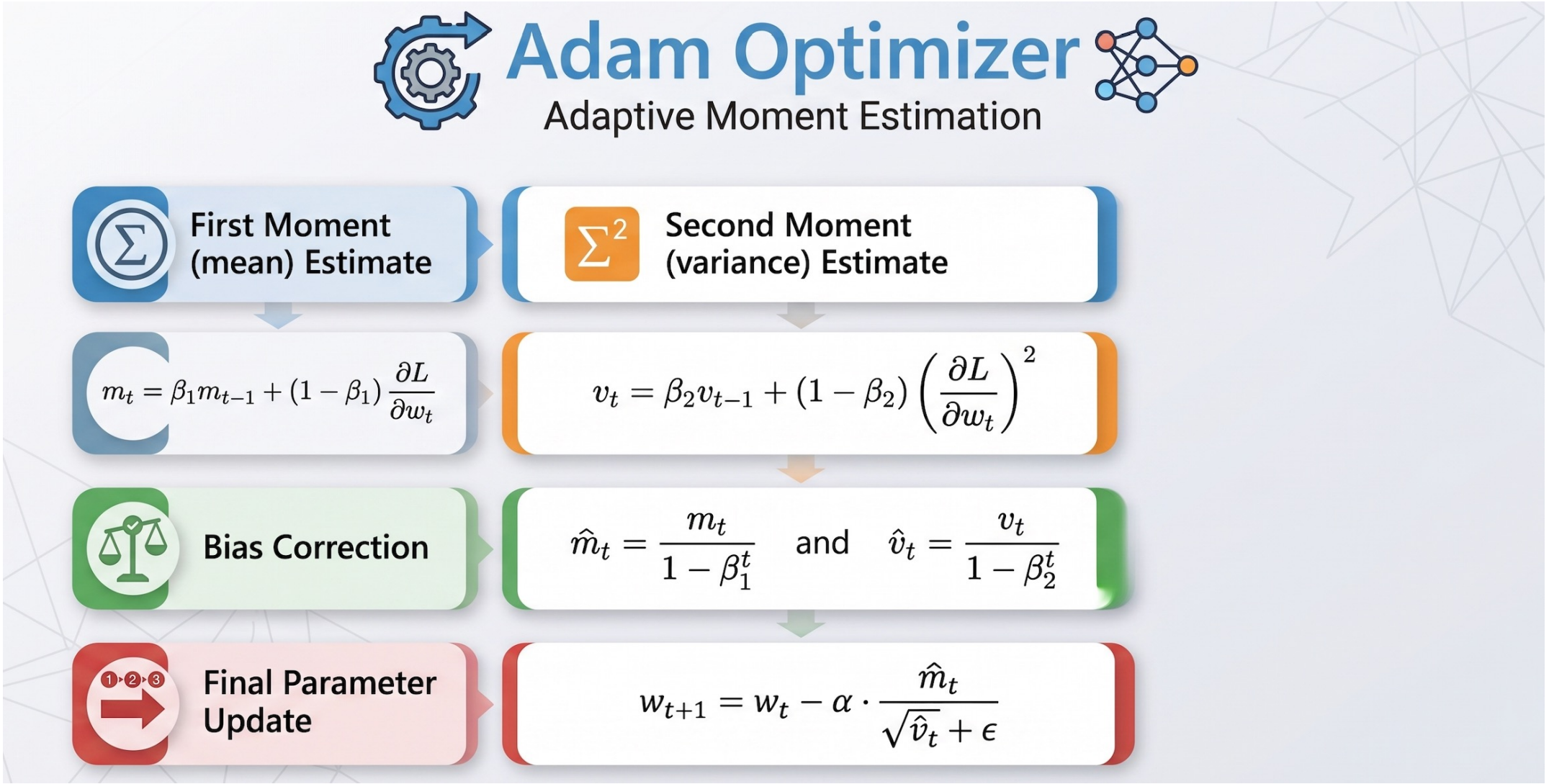
Quantitative Analysis

- GPU HBM Content During Training



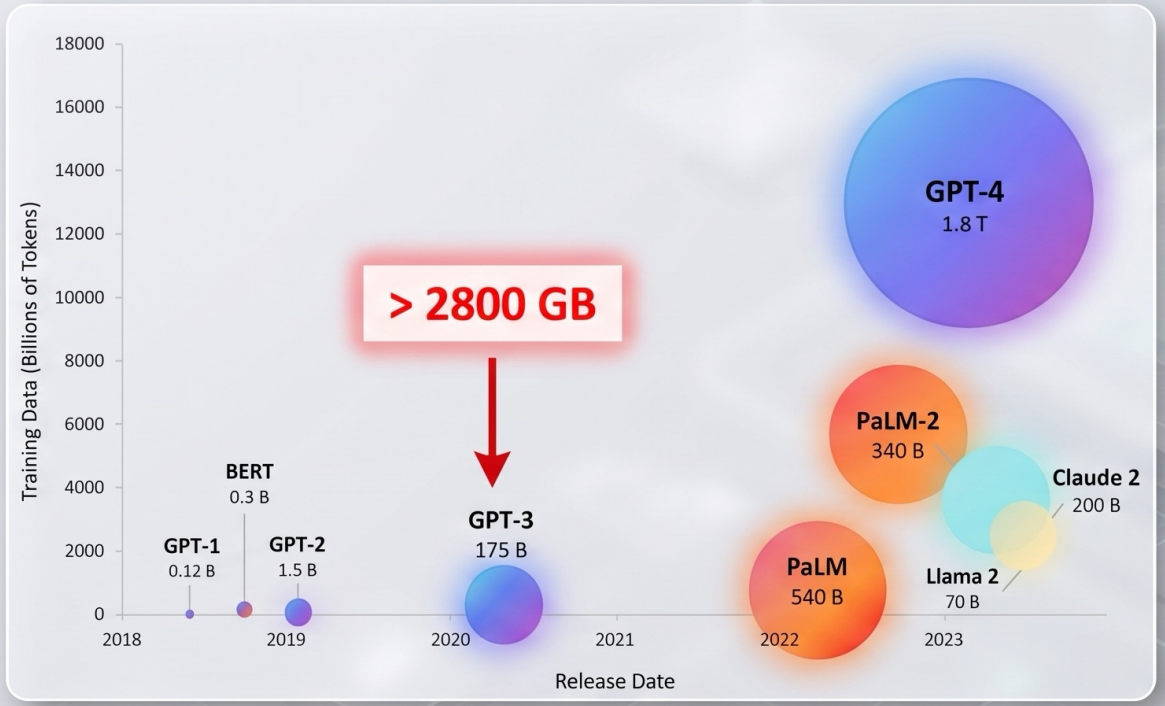
- An example of GPT-3 175B
  - **OPTIMIZER STATES**
    - 32-bit Parameter **700 GB**
    - Adam Moment **700 GB**
    - Adam Variance **700 GB**
  - 16-bit Parameter **350 GB**
  - 16-bit Gradient **350 GB**
  - Activations (depending on batch size)
  - Buffer and Fragmentation

Quantitative Analysis



Quantitative Analysis

- **GPT-175B models: storage**



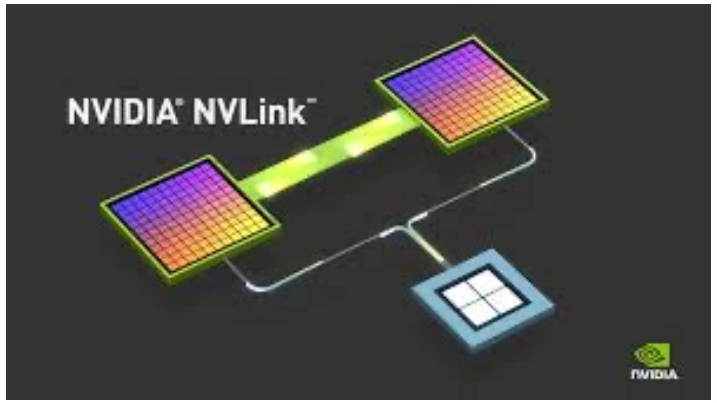
Models become larger and larger

GPU	Memory	Type
Tesla P100	16GB	HBM2
Tesla V100	32GB	HBM2
Tesla A100	40/80GB	HBM2E
Tesla A800	80GB	HBM2E
Tesla H100	80GB	HBM3
Tesla H800	80GB	HBM3

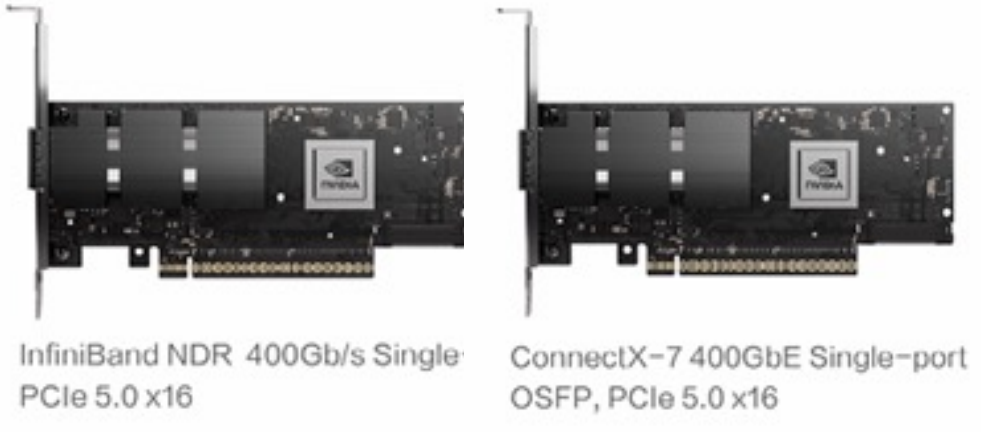
GPU HBM size remains small

Distributed Training

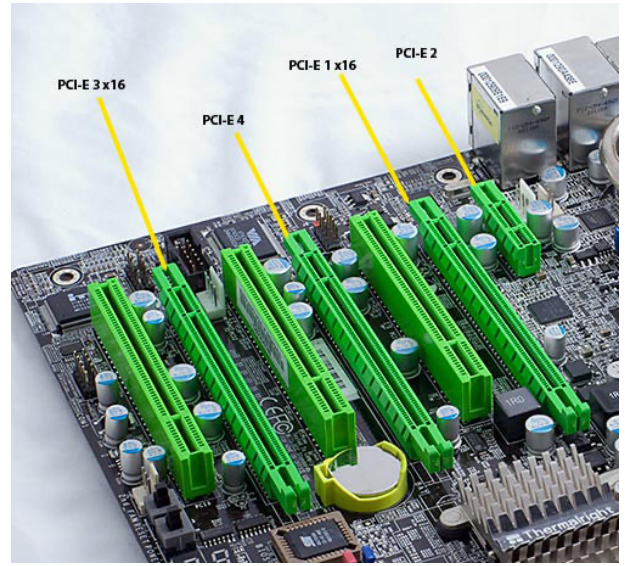
- Multiple GPUs compute collaboratively
  - Data Parallel, Pipeline Parallel, Tensor Parallel, Expert Parallel, Sequence Parallel, Context Parallel
- GPU interconnection



NVLink 5.0 (e.g. 1.8TB/s)



RDMA (RoCE/InfiniBand, e.g. 400Gbps)



PCIe 5.0 (e.g. 128GB/s, 16lanes)

Distributed Training

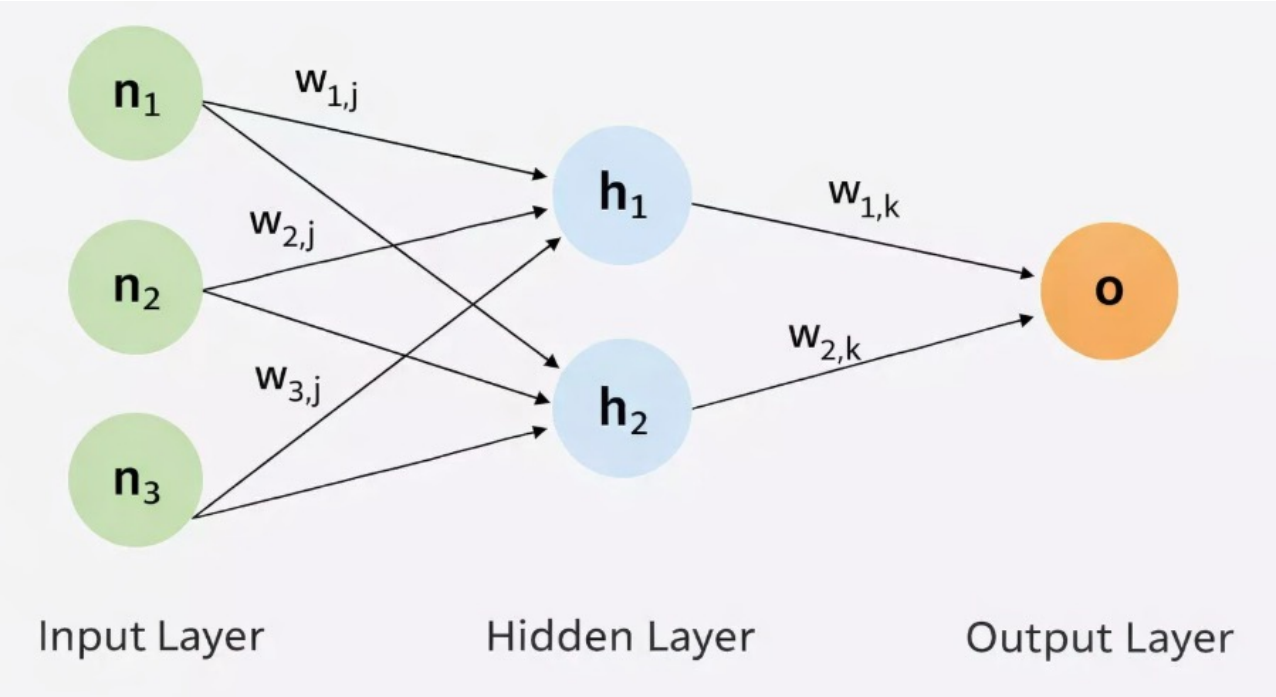
- Importance of Communication
  - Traffic volume
    - 16-bit gradient of GPT3-175B: 350GB needs to be transmitted in every iteration
    - Even more data communication if model is partitioned
  - Imbalanced technology upgrading
    - Ampere A100 (FP16 312TFLOPS, released in 2020) → Blackwell B300 (FP8 72PFLOPS, 2025) → Rubin (2026)
    - NVLink 3.0 (600GB/s, 2020) → NVLink 4.0 (900GB/s, 2022) → NVLink 5.0 (1.8TB/s, 2025) → NVLink 6.0 (3.6TB/s, 2026)

Distributed LLM Training: Outline

- Transformer: A Quick Overview
- Data Parallelism
  - **Parameter-Server**
  - All-Reduce
  - Memory Optimization

DNN Training

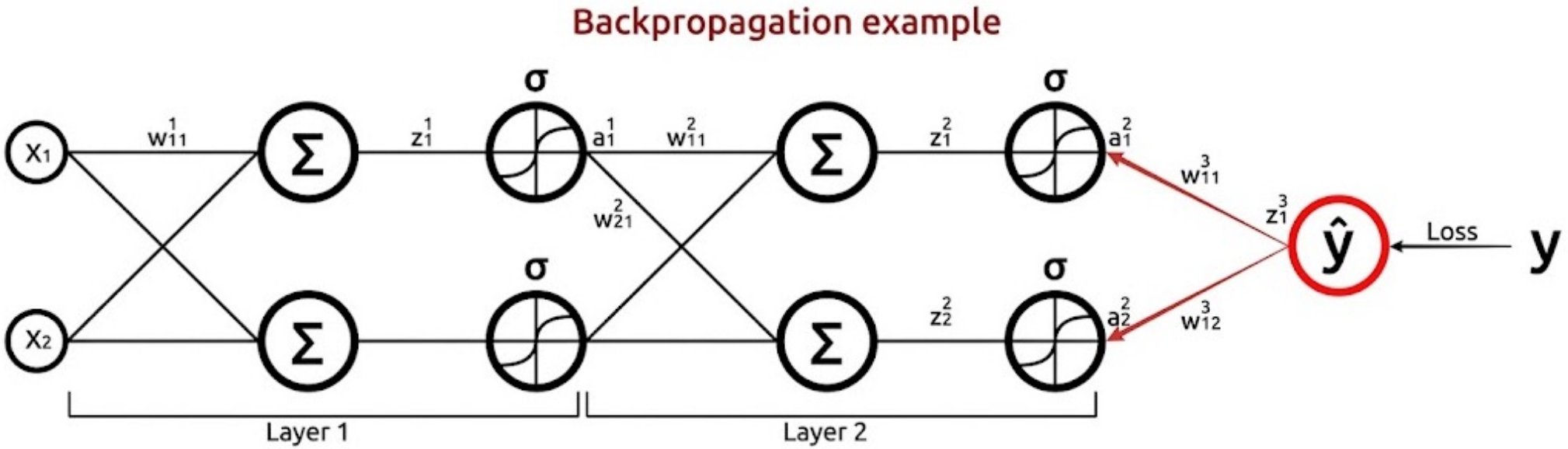
- Forward propagation



Forward propagation

DNN Training

- Backward propagation



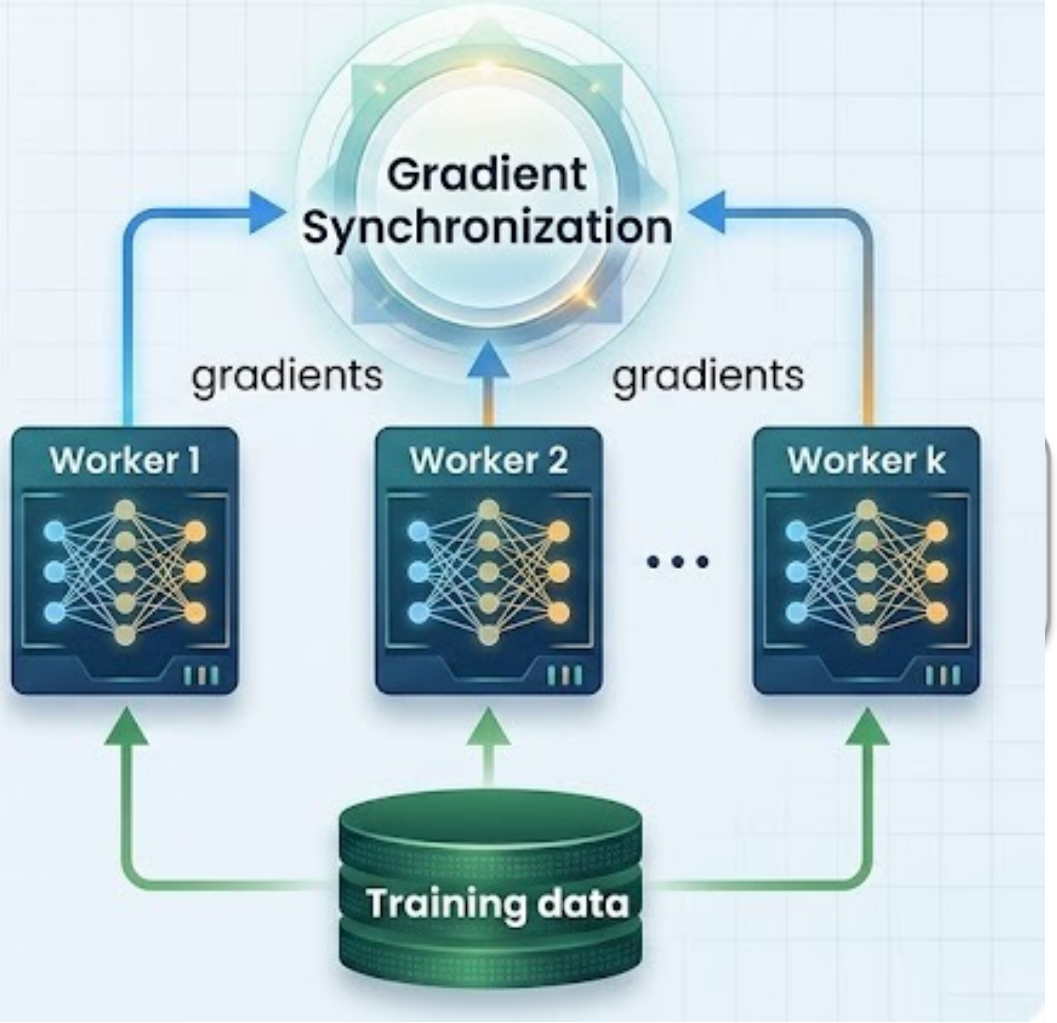
$$\frac{\partial L}{\partial w_{11}^1} = \frac{\partial L}{\partial z_1^3}\left[\frac{\partial z_1^3}{\partial a_1^2}\frac{\partial a_1^2}{\partial z_1^2}\frac{\partial z_1^2}{\partial a_1^1} + \frac{\partial z_1^3}{\partial a_2^2}\frac{\partial a_2^2}{\partial z_2^2}\frac{\partial z_2^2}{\partial a_1^1}\right]\frac{\partial a_1^1}{\partial z_1^1}\frac{\partial z_1^1}{\partial w_{11}^1}$$

Distributed LLM Training: Outline

Data Parallelism

- Distribute data to workers
- Each worker work independently
- Synchronize gradients via different approaches
- Repeat the above procedures until model convergence

Aggregate compute power



Parameter Server Architecture

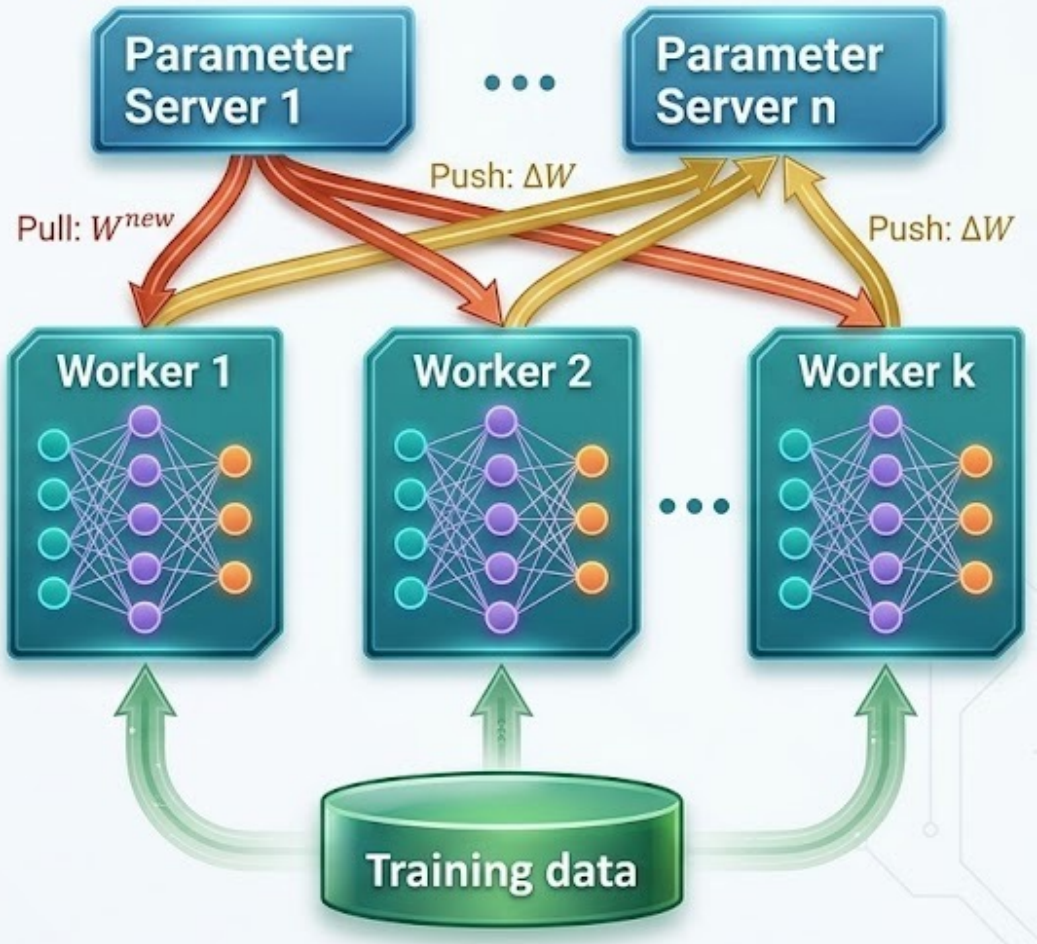
Update: $W^{new} = W - \gamma \Delta W$

5. Pull: W^{new}

3. Push: ΔW

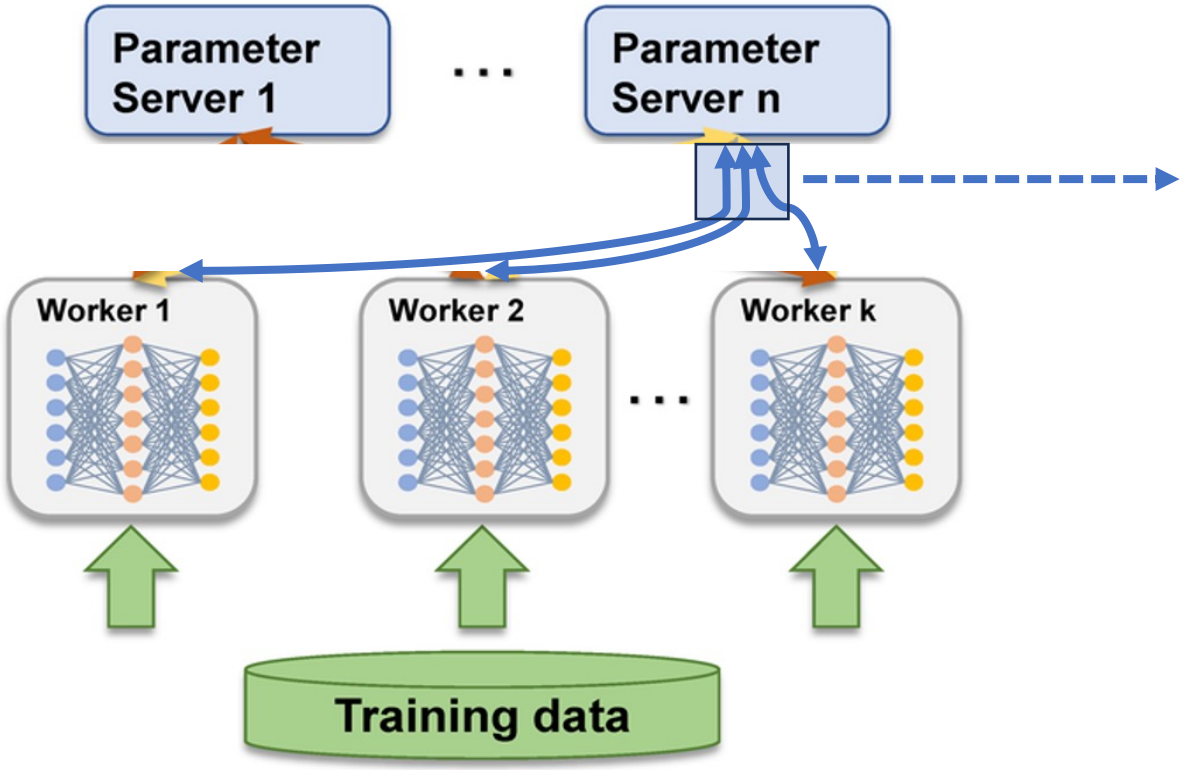
2. back propagation

1. forward propagation



Distributed Parameter Server Architecture: Augmenting Bandwidth

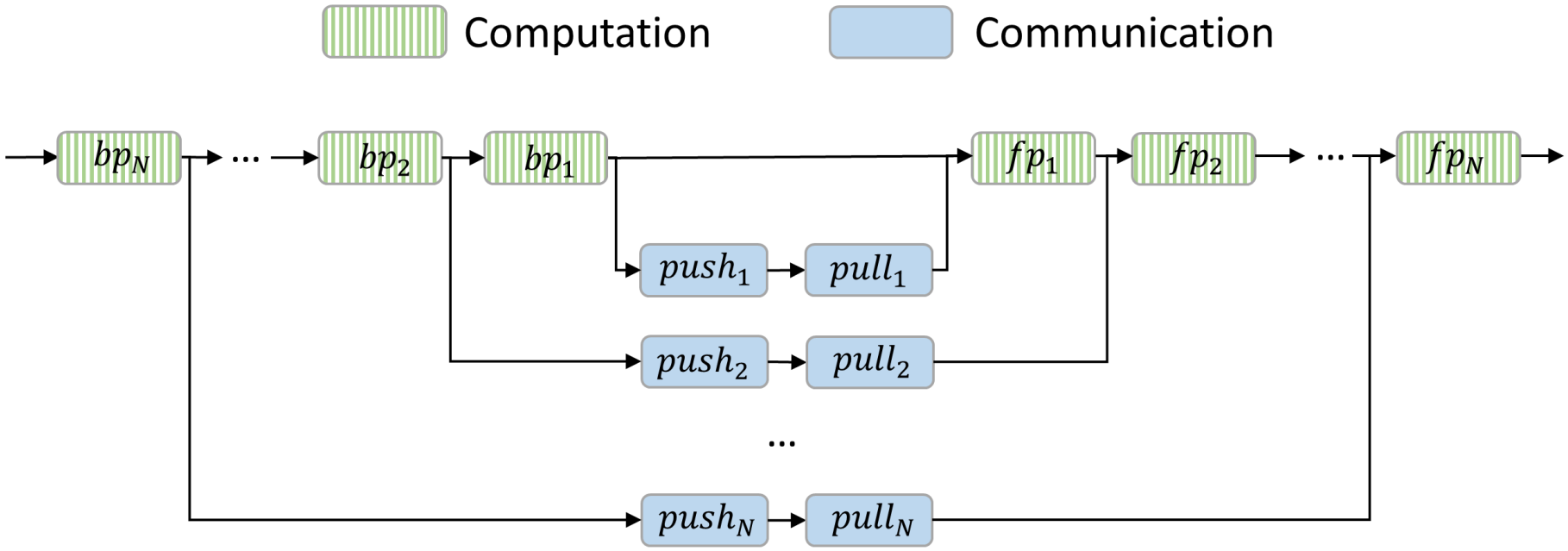
Parameter Server Architecture



- Shared bottleneck
  - Communication throttles computation
  - Reduced bandwidth efficiency due to multi-flow competition
  - Difficult to overlap comp. and comm., push and pull

Network Bottleneck

Optimizing PS Architecture via Scheduling

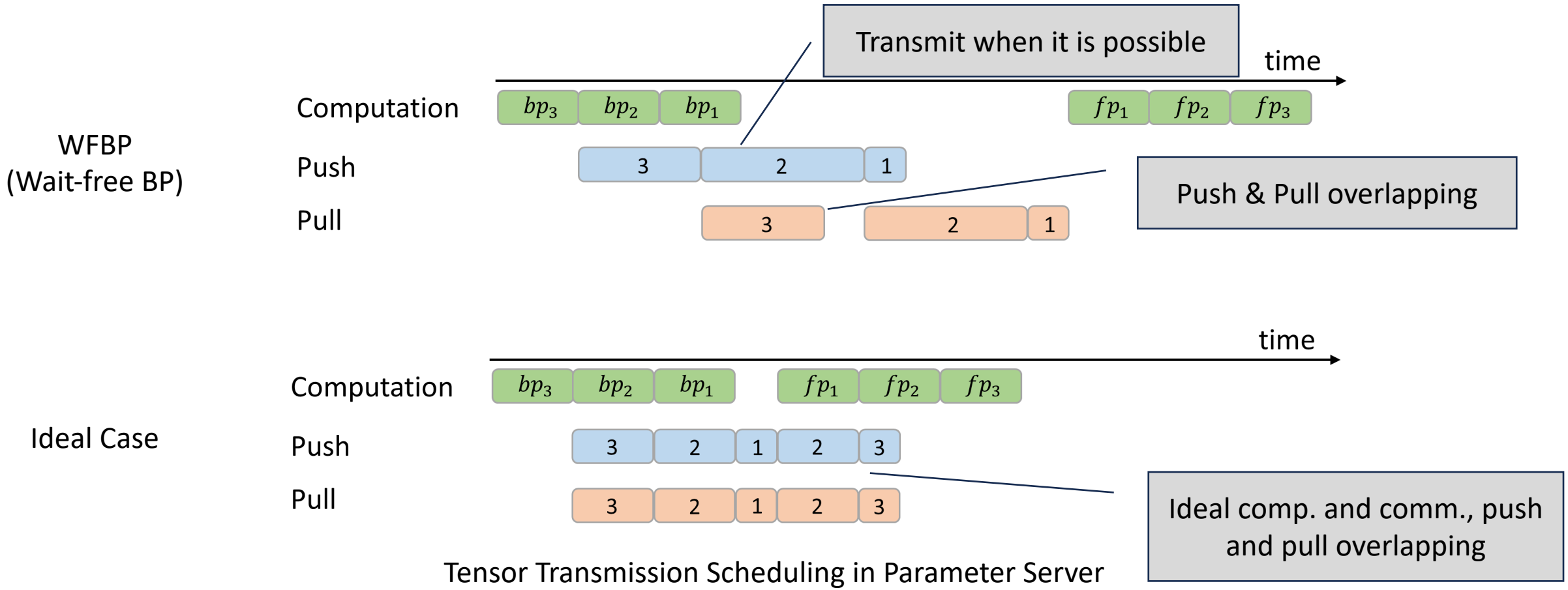


Computation order: bp_N → bp_{N-1} → … → bp_2 → bp_1 → fp_1 → fp_2 → … → fp_N

Data availability order: $gradient_N$ → $gradient_{N-1}$ → … → $gradient_2$ → $gradient_1$

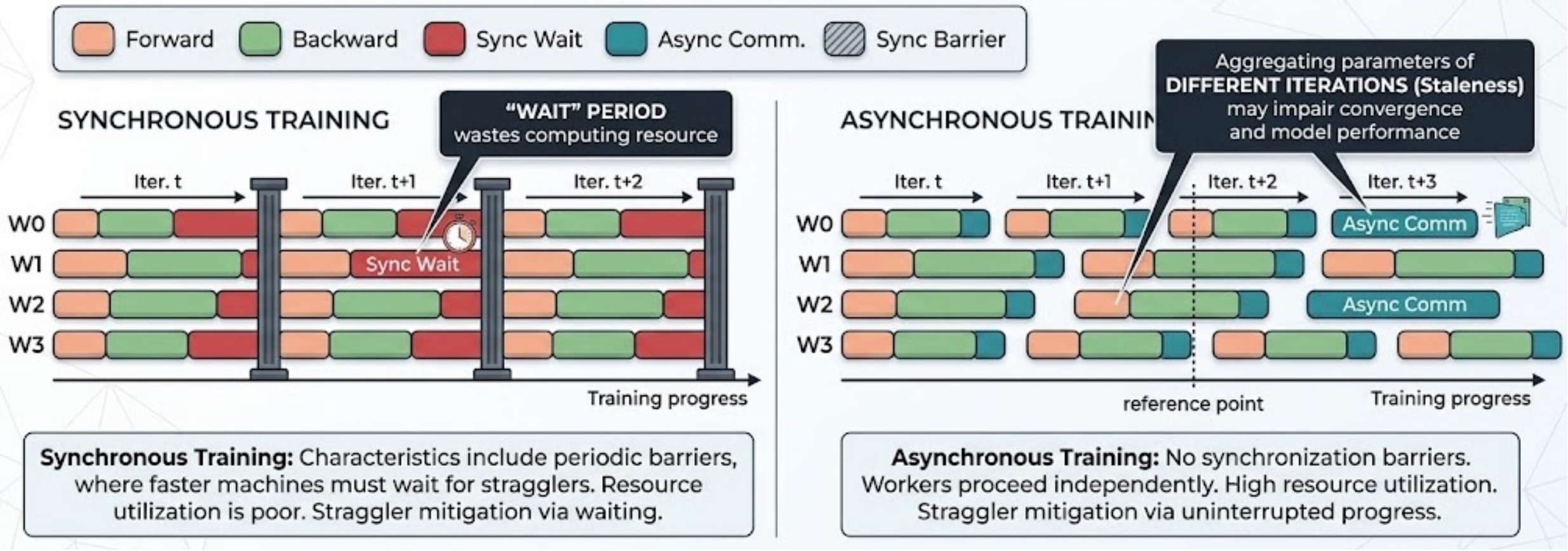
Layer-wise Computation and Communication for a DNN with three layers

Optimizing PS Architecture via Scheduling



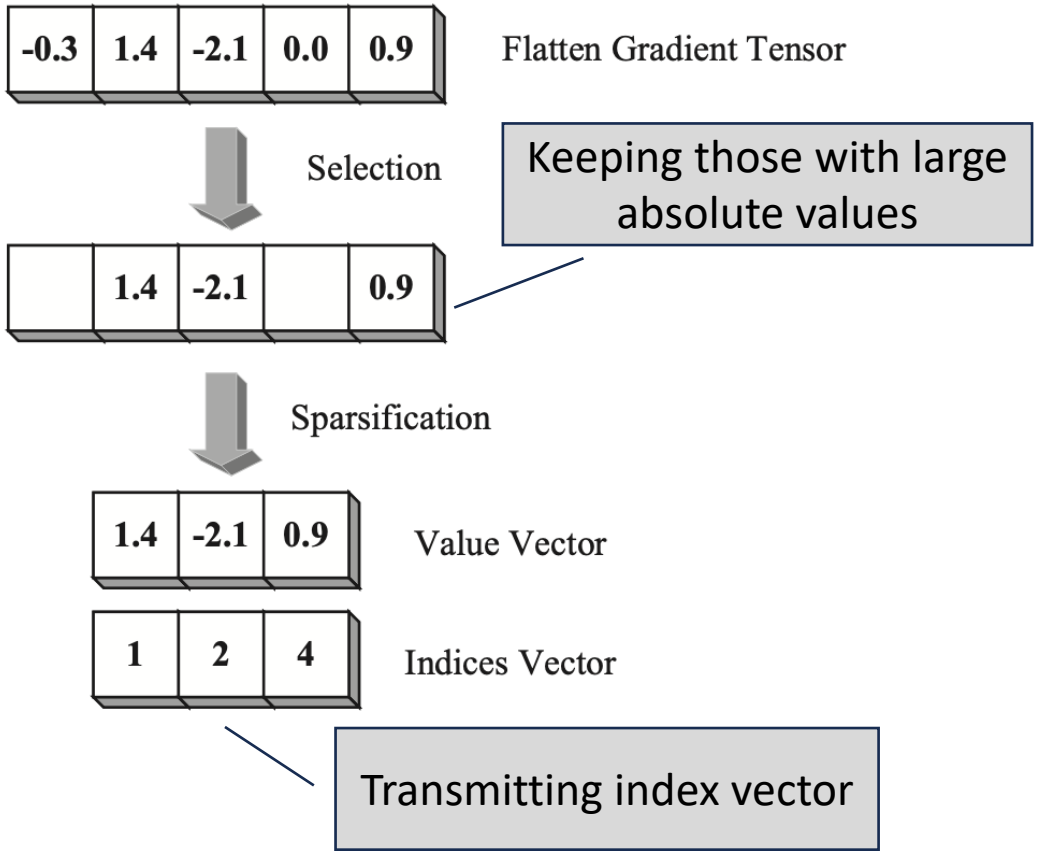
Tensor Transmission Scheduling in Parameter Server

Optimizing PS Architecture via Asynchronism

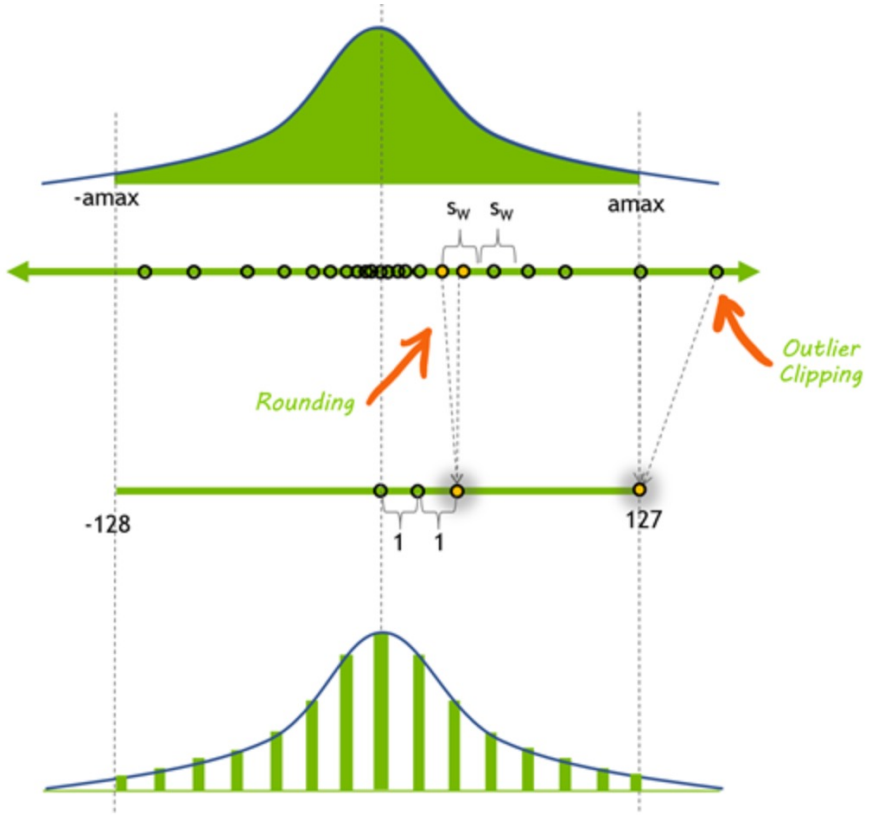


Asynchronous Training: Mitigating Stragglers

Optimizing PS Architecture via Compression



Sparsification: Reducing # of parameters

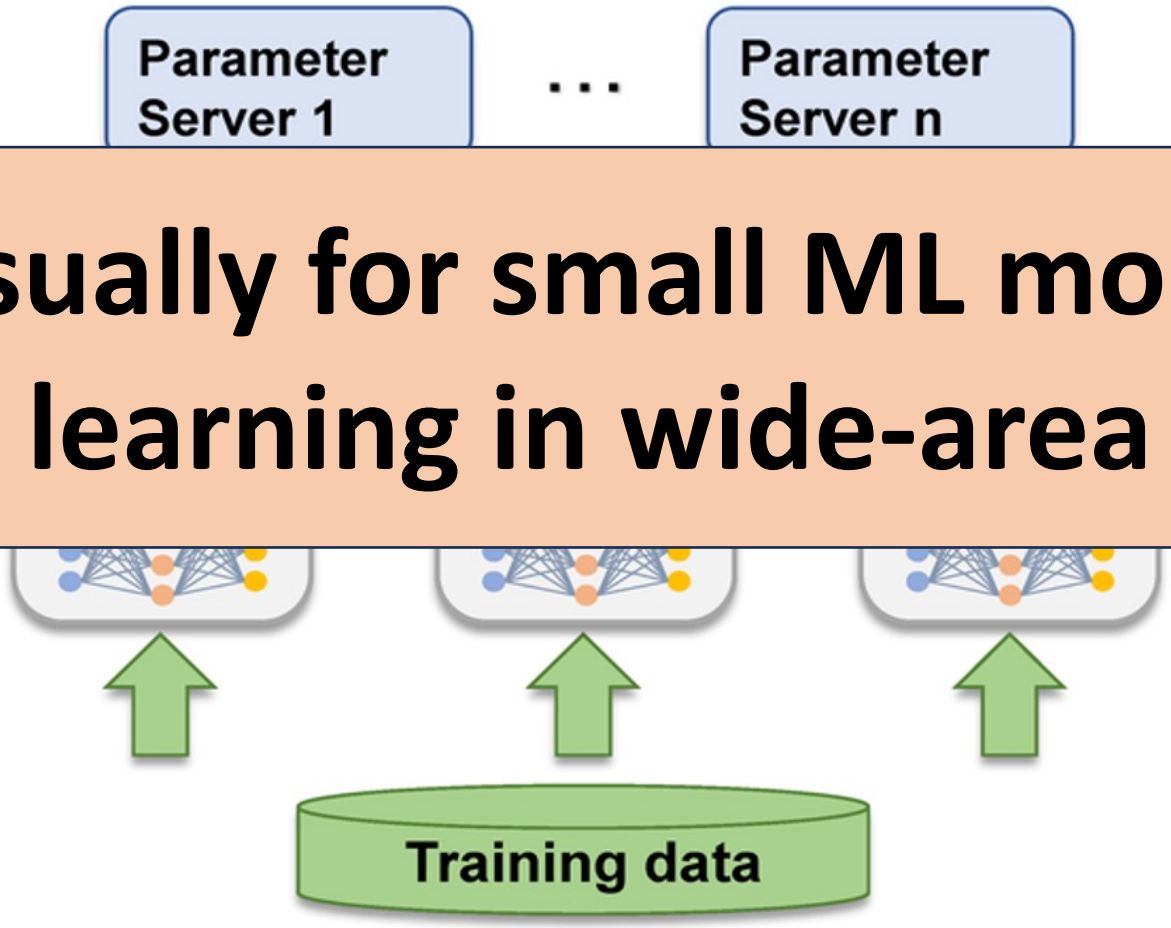


Mapping gradients into low-precision ones w/wo the consideration of value distribution

Sparsification and Quantization

《Communication Compression Techniques in Distributed Deep Learning: A Survey》

Parameter Server Architecture



Simple, usually for small ML models and distributed learning in wide-area networks

Distributed LLM Training: Outline

- Data Parallelism
  - Parameter-Server
  - **All-Reduce**
  - Memory optimization

Collective Communication Basics

- Collective Communication

> *"**Collective communication** is communication that involves a group of processing elements (termed nodes in this entry) and affects a data transfer between all or some of these processing elements. Data transfer may include the application of a reduction operator or other transformation of the data." 《Encyclopedia of Parallel Computing》*

集合通信是指一个进程组的所有进程都参与的全局通信操作。

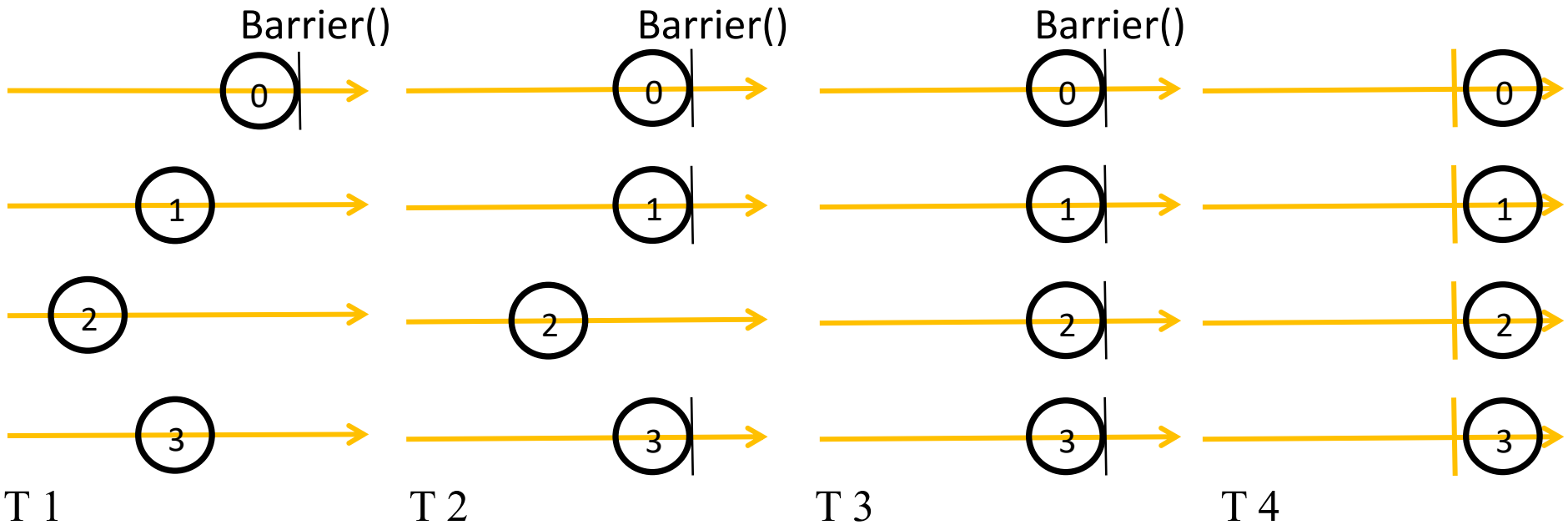
全部点对点通信完成才算集合通信完成

Collective Communication Basics

- Why Collective Communication
  - Simplified Programming Interface
    - Developers don't need to manually code complex synchronization and data distribution logic for every scenario.
  - Scalability for Large-Scale Systems
    - As systems grow to thousands of nodes or GPUs (e.g., in supercomputers or cloud clusters), managing communication becomes exponentially complex. These libraries support sparse data handling, fault tolerance, and observability features to ensure reliable operation at scale.
  - Decompose Compute and Communication
    - Allowing machine learning researchers and system engineers to work on their own.

Collective Communication Basics

- Collective Communication: Basic Operations
  - **SEND**, **RECEIVE**, COPY, BARRIER, SIGNAL+WAIL (in Message Passing Interface, i.e. MPI)

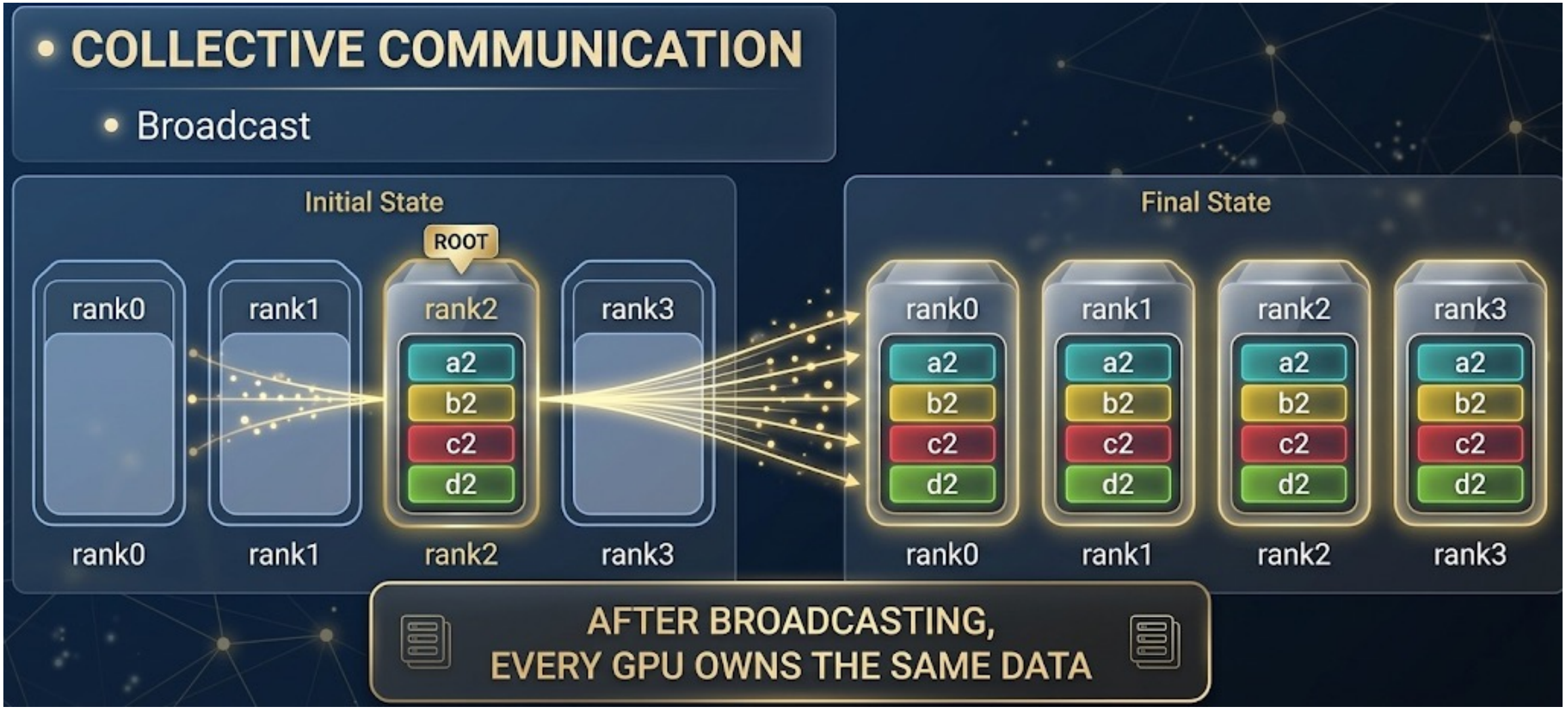


* Pictures replotted, but originated from Mengyuan @ Zhihu

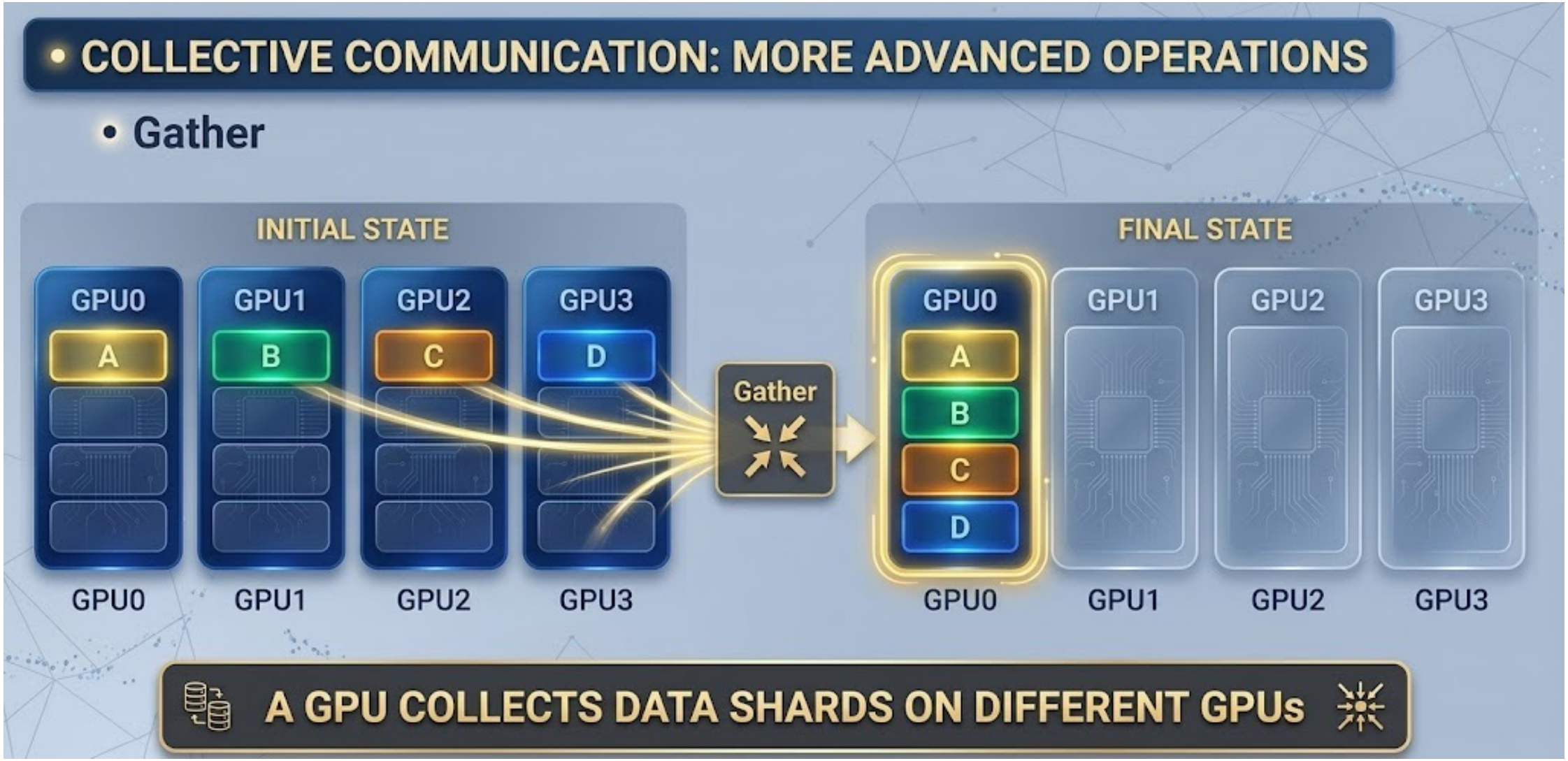
Collective Communication Basics

- Collective Communication: More Advanced Operations
  - Broadcast: one-to-many
  - Gather: many-to-one, and All-Gather: many-to-many
  - Scatter: one-to-many
  - Reduce: many-to-one, and All-Reduce: many-to-many
  - Reduce-Scatter: aggregate data and then transmit
  - All-to-All: many-to-many
  - AllReduce: ?

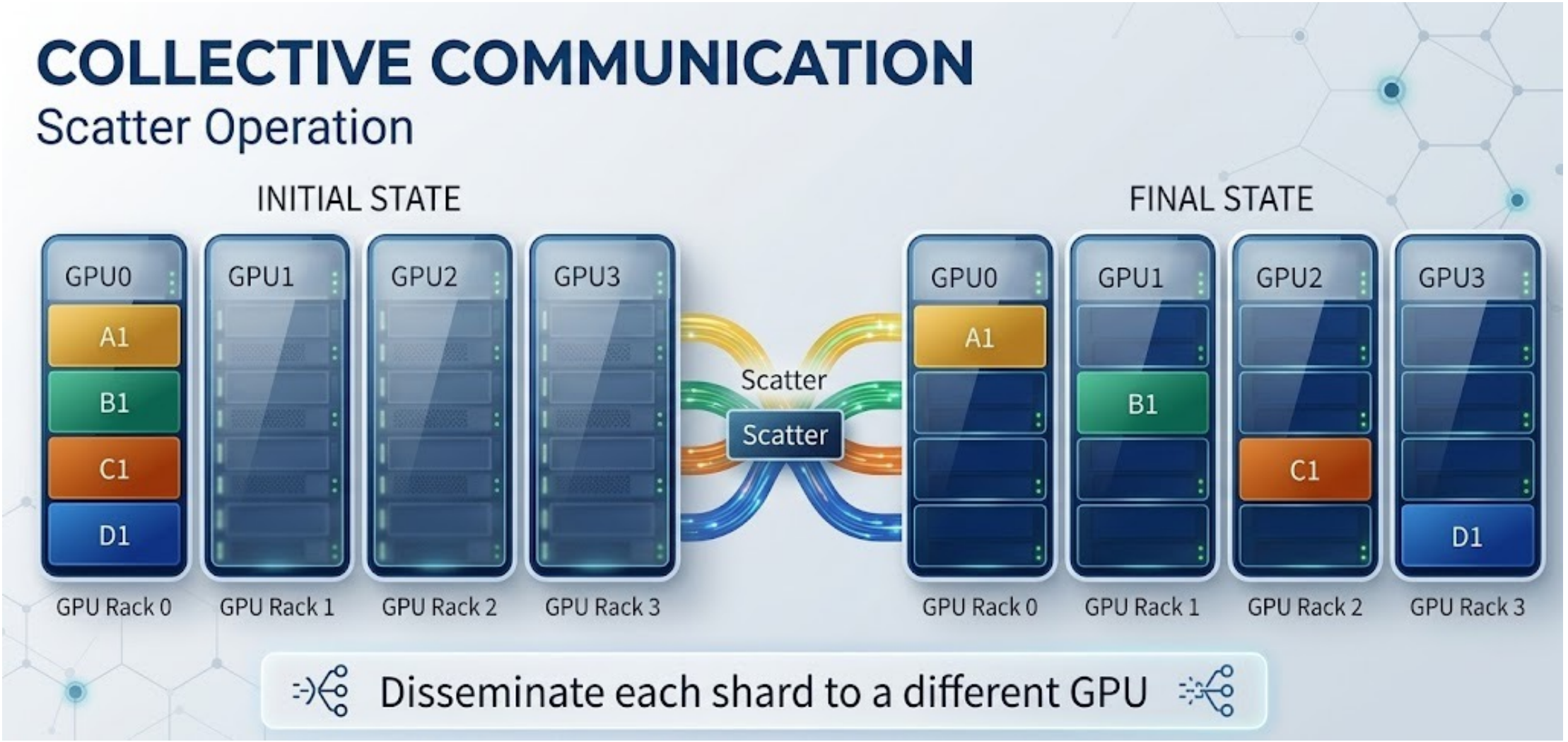
Collective Communication Basics



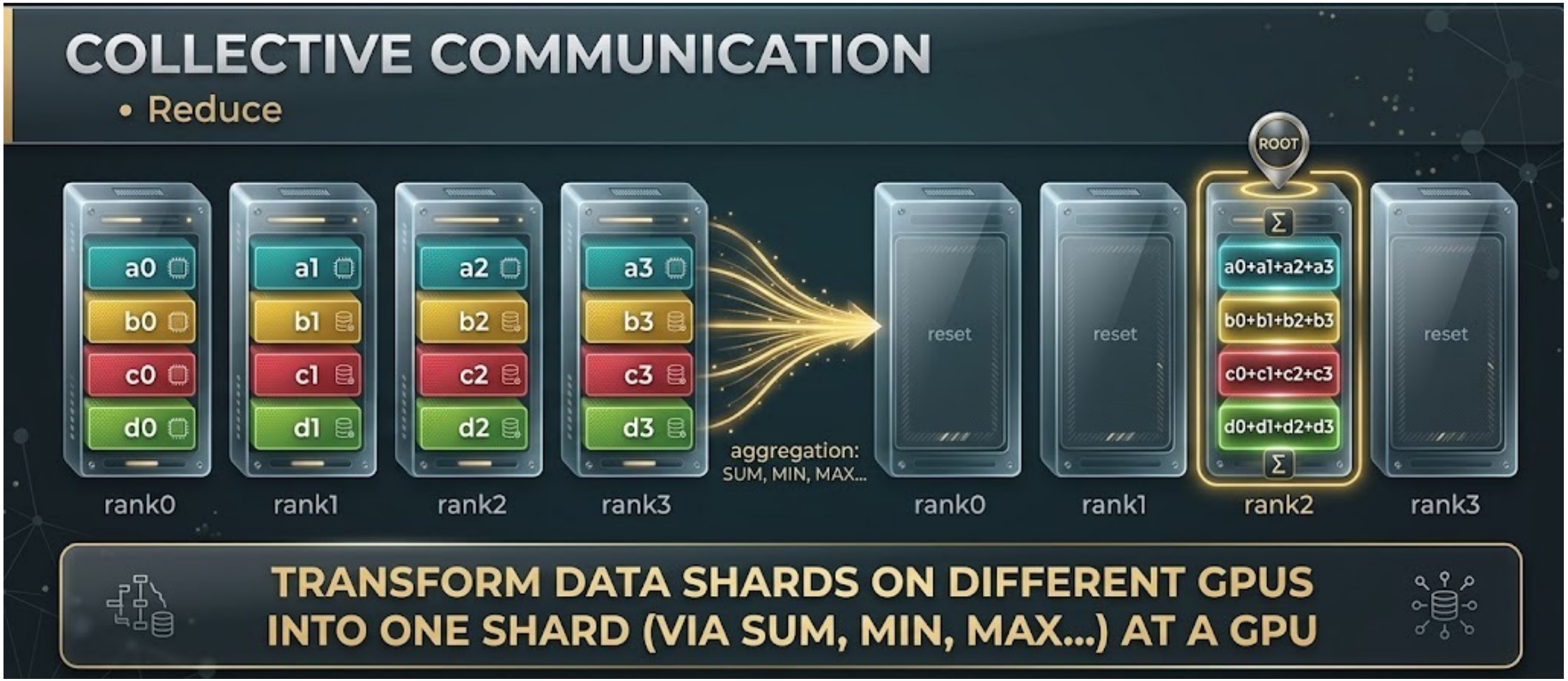
Collective Communication Basics



Collective Communication Basics



Collective Communication Basics



Collective Communication Basics

Collective Communication Basics

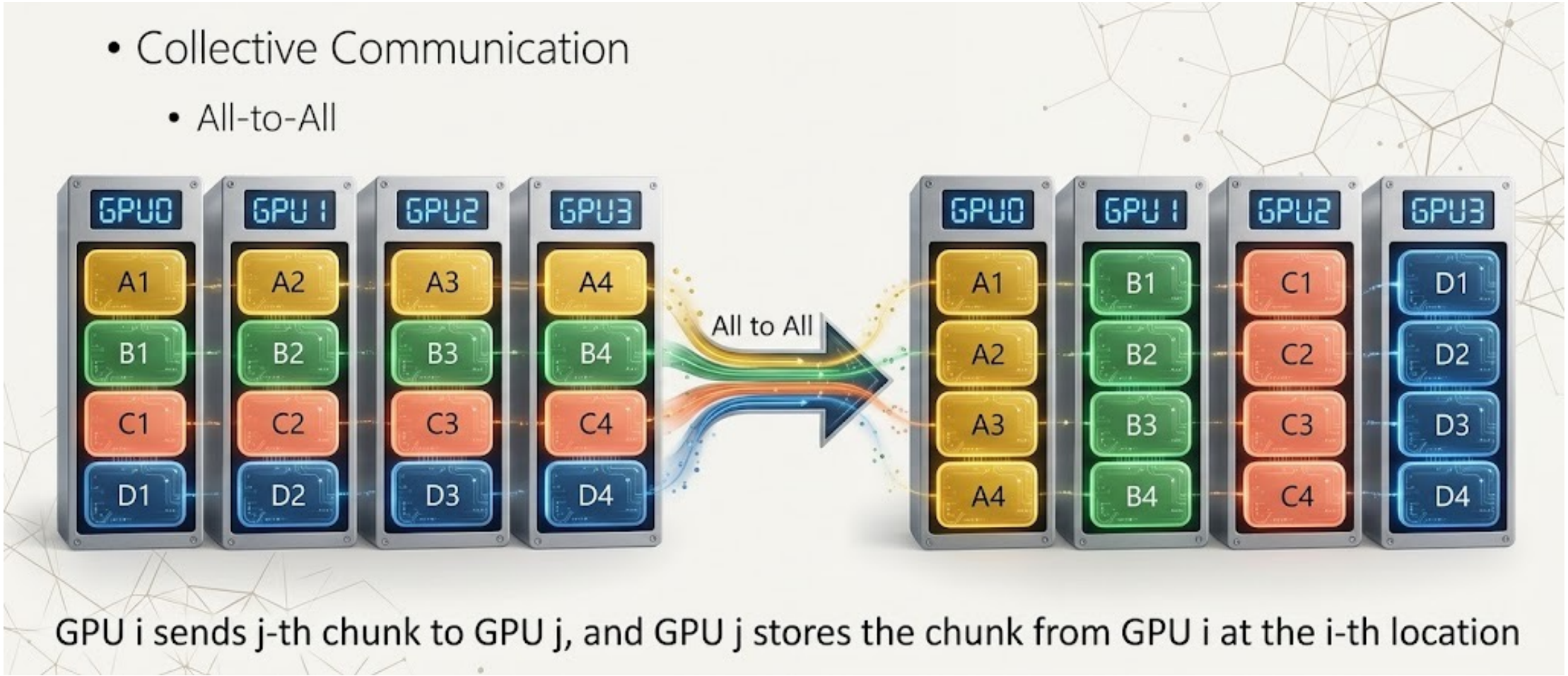
- Collective Communication
  - All-Gather

Collective Communication Basics

- **Collective Communication**
  - Reduce-Scatter

Aggregate data chunks and store one shard at a GPU

Collective Communication Basics



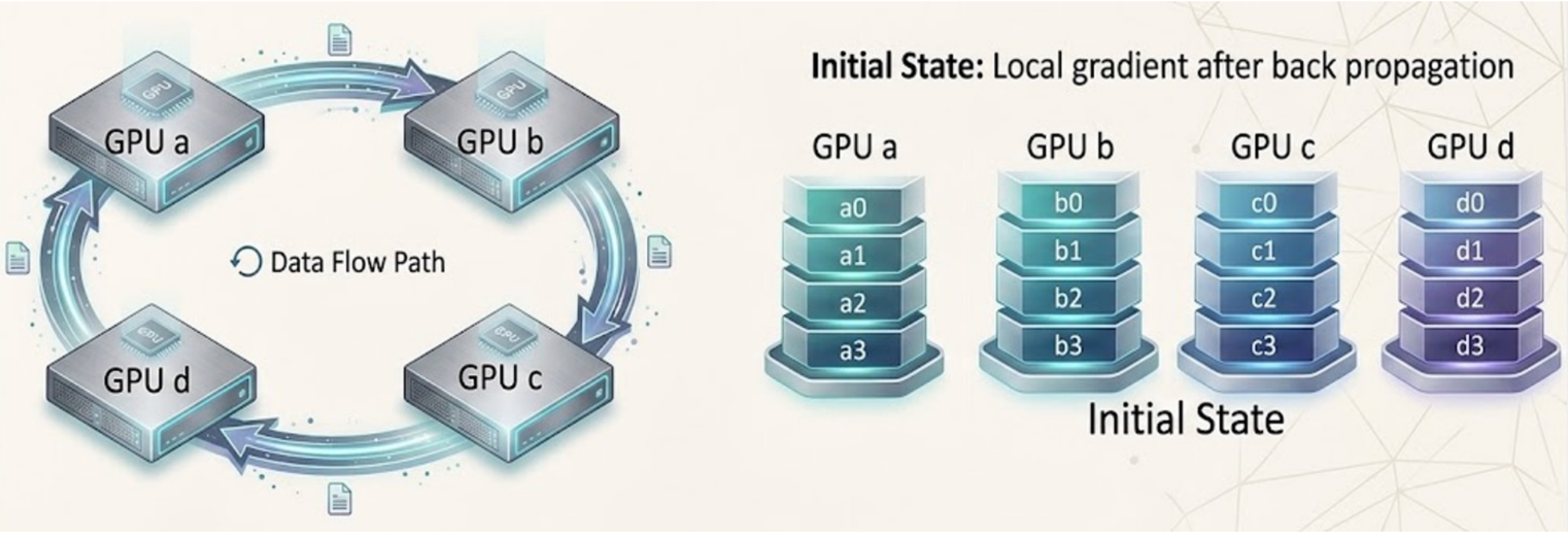
All-to-All is purely a data routing and memory transposition operation

Starting from the most important “All-Reduce”

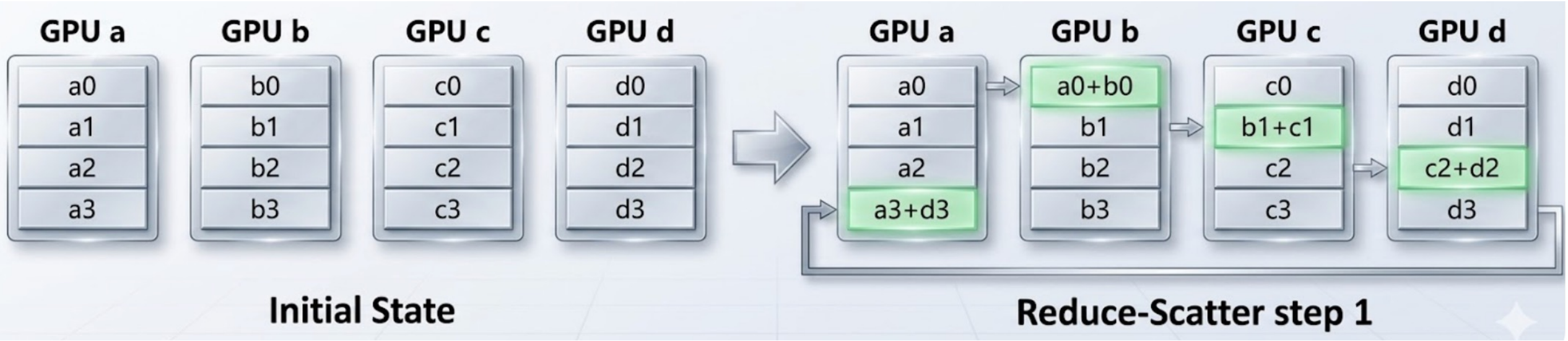
Ring All-Reduce

- Ring All-Reduce
- Tree All-Reduce
- Topology-aware All-Reduce

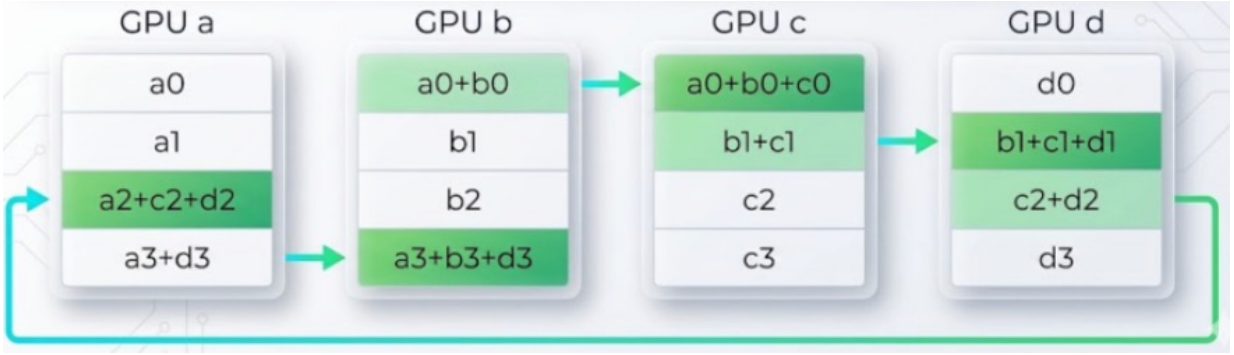
Ring All-Reduce



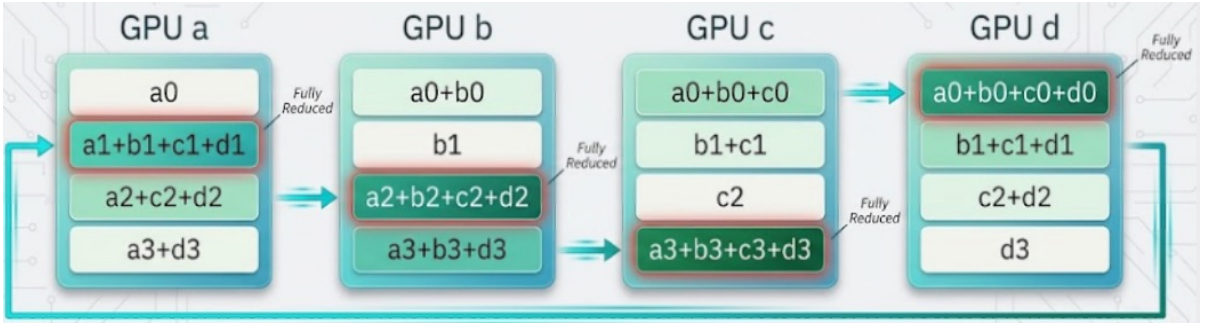
Ring All-Reduce



Ring All-Reduce



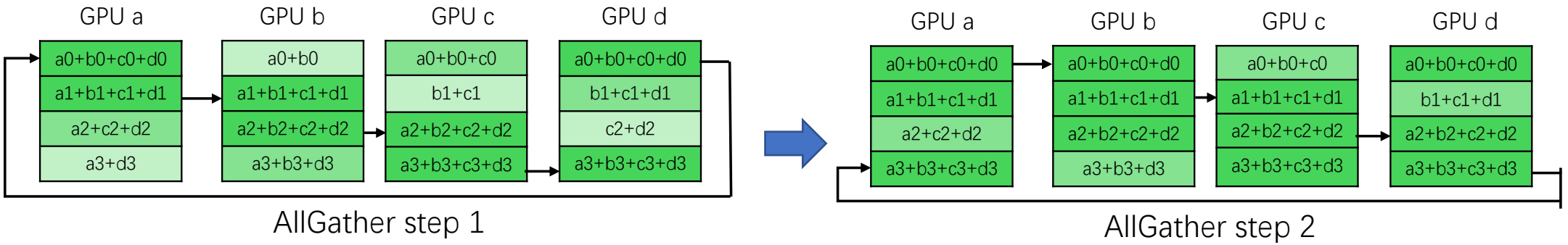
Reduce-Scatter Step 2



Reduce-Scatter Step 3

Ring All-Reduce

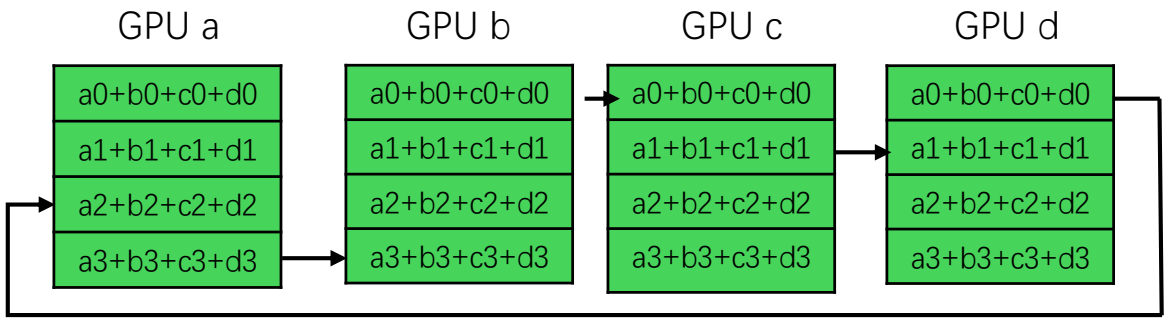
- Ring All-Reduce



All-Gather

Ring All-Reduce

- Ring All-Reduce



AllGather

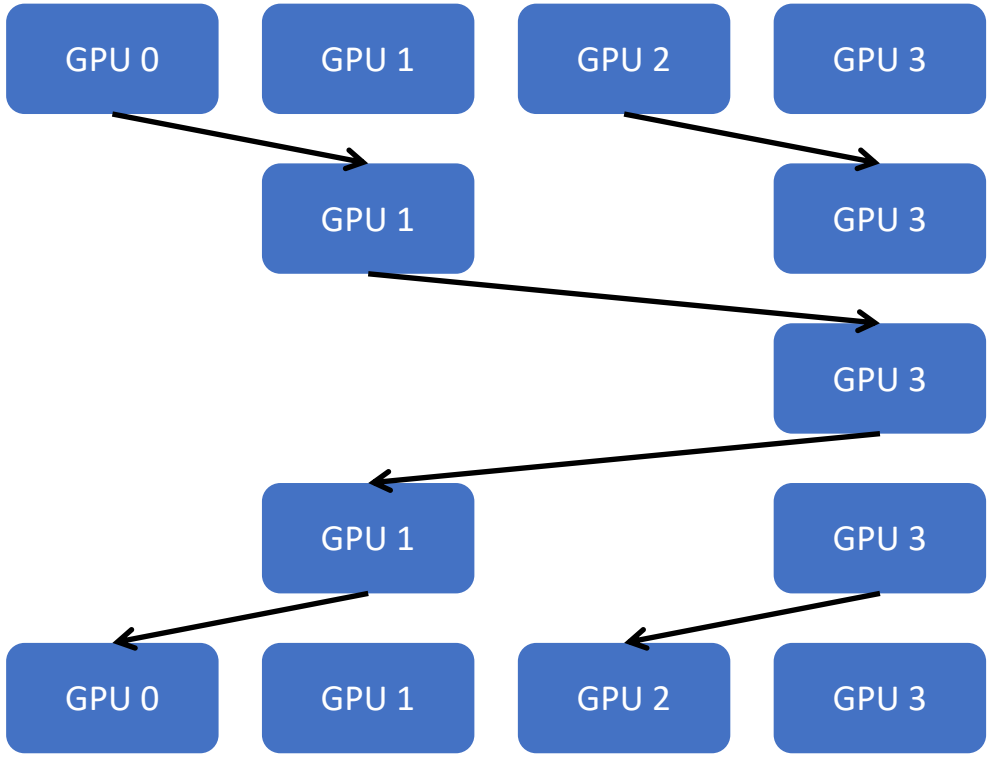
- Reduce-Scatter
  - N: # of GPUs, S: per-GPU data volume
  - $N-1$ rounds
  - S/N data transmission per round
- All-Gather
  - One round boradcasting or $N-1$ clockwise rounds
  - Data volume: $(N-1)*S/N$
- Total traffic per GPU
  - $\boldsymbol{2*(N-1)*S/N}$

Ring All-Reduce

“All-Reduce = Reduce-Scatter + All-Gather”

Tree All-Reduce

- Tree All-Reduce

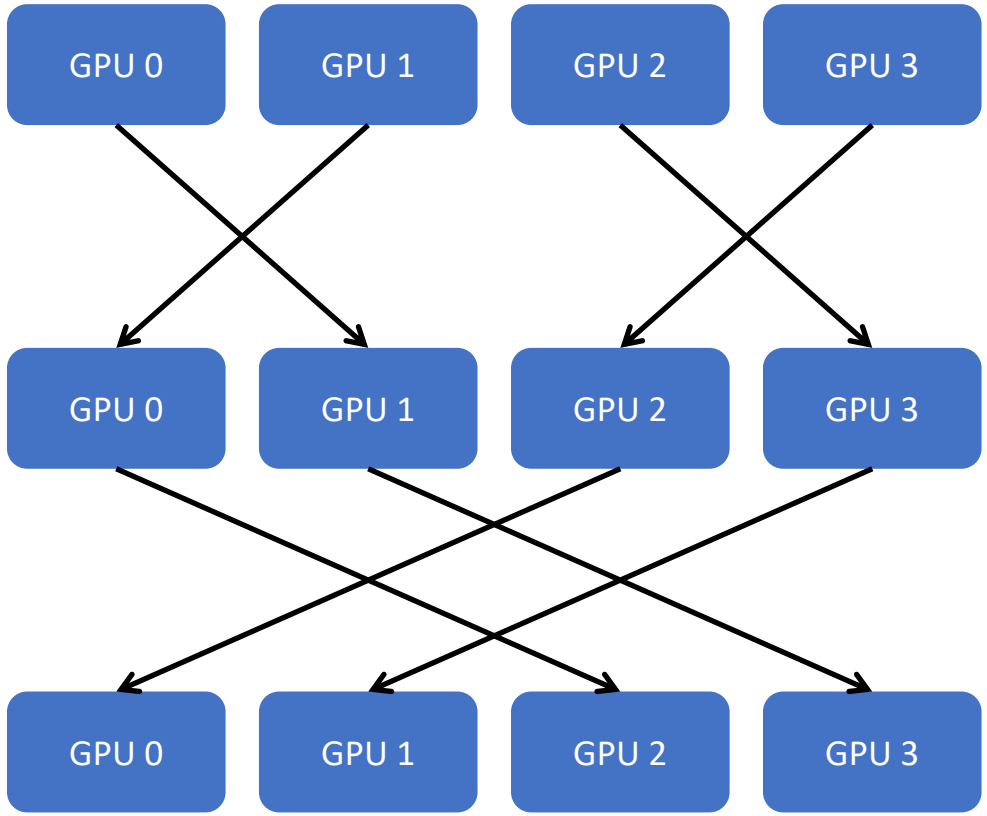


Data communication

- Upward
  - $\log_2 N$ rounds
  - At most S data per round
- Downward
  - $\log_2 N$ rounds
  - At most S data per round
- Total traffic per GPU
  - $2 * S * \log_2 N$ (naïve transmission without overlapping)

Butterfly All-Reduce

- Butterfly All-Reduce

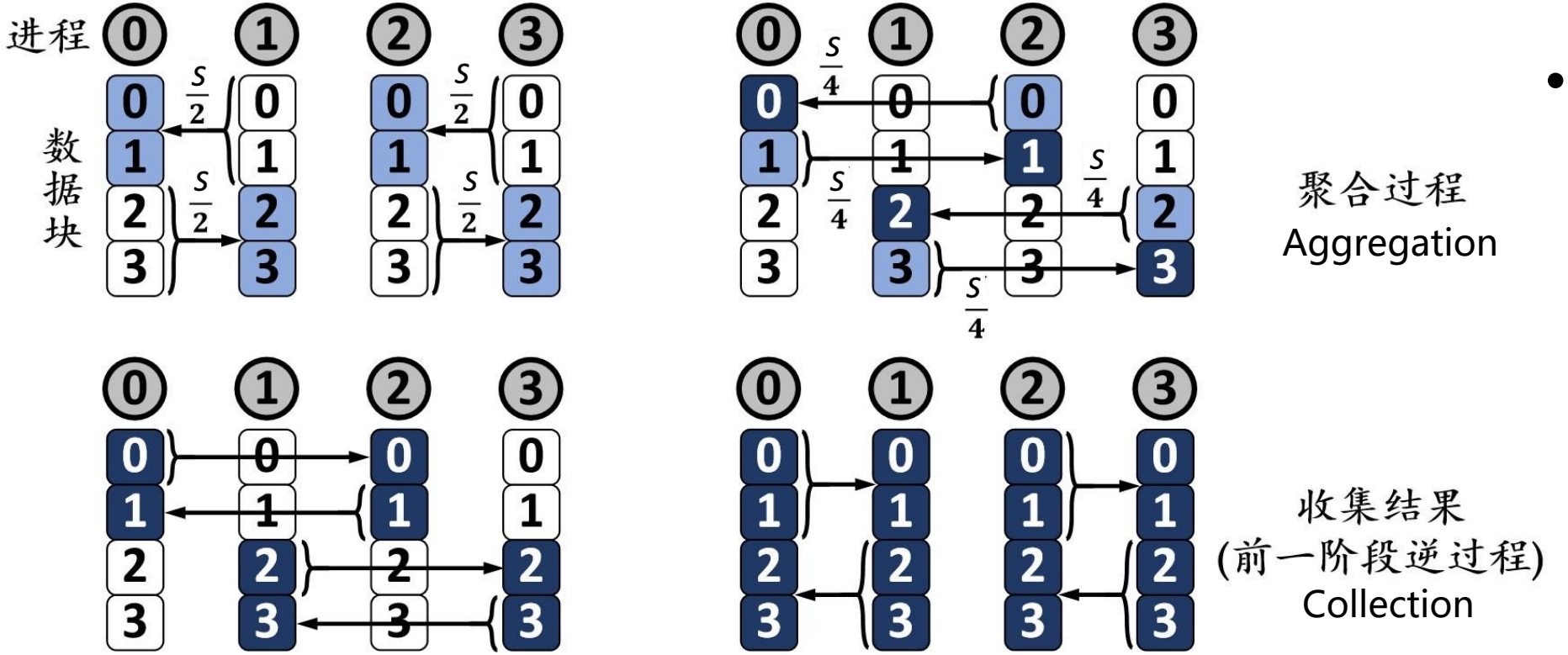


Butterfly Reduce: Utilizing bidirectional bandwidth

- Butterfly Reduce
  - $\log_2 N$ rounds
  - S data per round
- Total Traffic per-GPU
  - $\log_2 N * S$

Rabenseifner All-Reduce

- Rabenseifner Reduce



Rabenseifner Algorithm (~ recursive halving-doubling)

- Total Traffic per-GPU
  - $2\log_2 N$ rounds
  - Different traffic load at every round
  - Total traffic volume: $2\left(\frac{S}{2}+\frac{S}{4}+\cdots+\frac{S}{N}\right) \approx 2\frac{N-1}{N}S$

《Allreduce算法及其硬件加速方法介绍》

Cross-Comparison

- Communication Cost
  - Assumption: each GPU can send and receive data simultaneously
  - Classical α-β model: latency = $\alpha + \beta \cdot \frac{S}{B}$

"start-up" delay

transmission efficiency

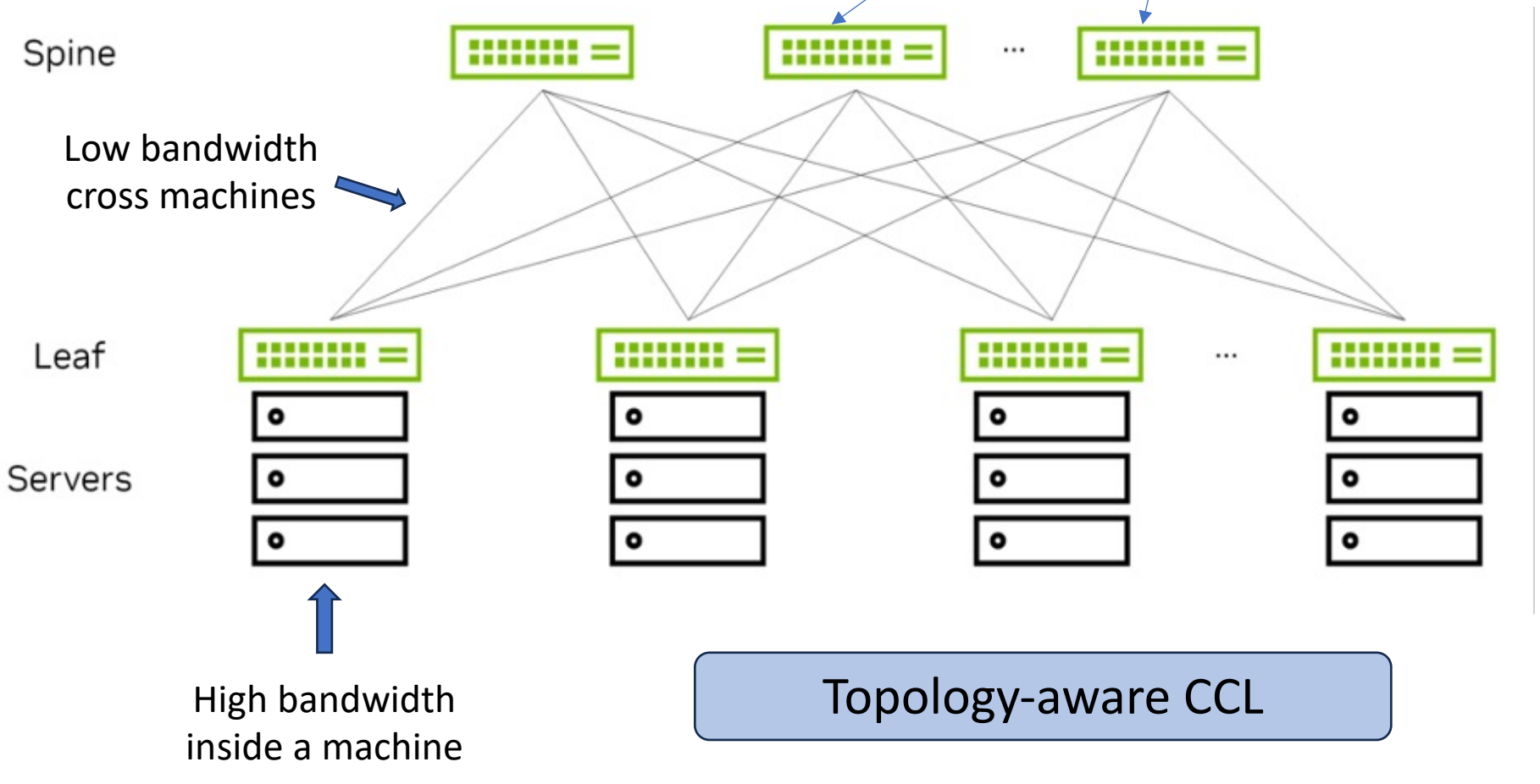
Message size/bandwidth

Algorithm	Rounds	Traffic Load	Cost	Pros and Cons
Ring	2(N-1)	2S(N-1)/N	*2(N-1)*(α + β∗S/N/B)*	▪ Large data chunk aggregation ▪ Relatively small # of GPUs ▪ Not suitable for short chunks ▪ Ease of implementation ▪ Utilizating bidirectional links
Rabenseifner	2 $\lceil \log N \rceil$	2S(N-1)/N	2$\lceil \log N \rceil\alpha$ + 2$(N-1)$ $\beta * S/N$/B	▪ Relatively smaller data chunk ▪ Large # of GPUs ▪ Periodically changing communication pairs

Challenge of Scaling Out

Network congestion, load balancing

- Two-tier spine-leaf topology



Topology-aware CCL

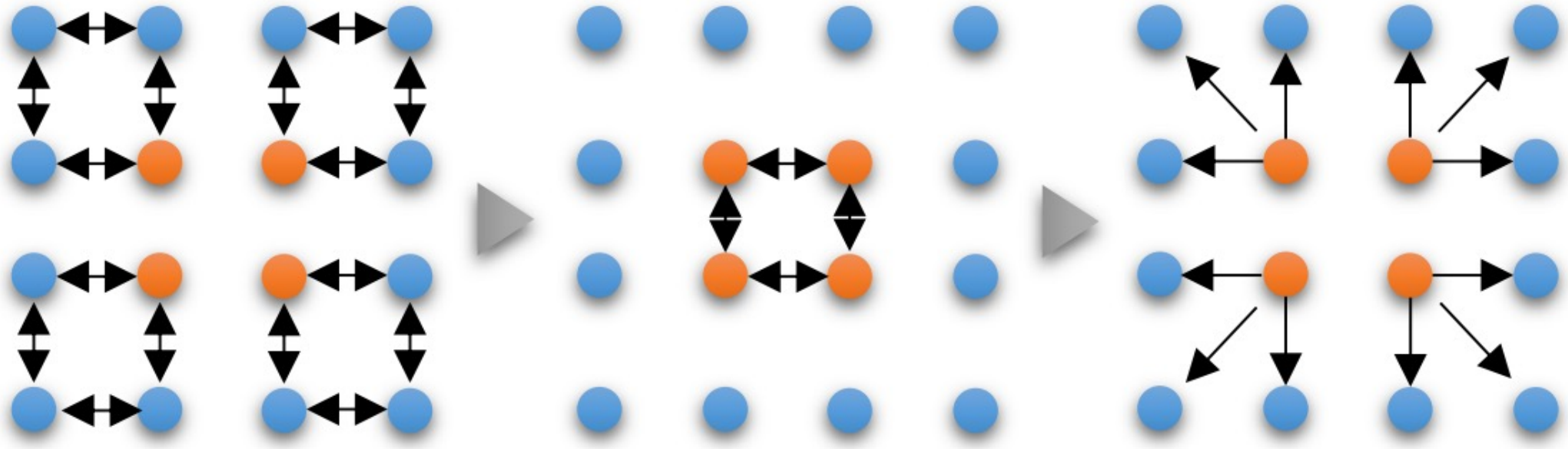
All-Reduce for LLM training

- Each company develops its own collective communication libraries
  - NCCL (NVIDIA CCL)
  - MSCCL (Microsoft CCL)
  - Gloo
  - HCCL (Huawei CCL)
  - ACCL (Alibaba CCL)
  - TCCL (Tencent CCL)
  - Many to be added

Tailored for their own harware, datacenter network topology, etc.

All-Reduce for LLM training

- Hierarchical All-Reduce (for reference)

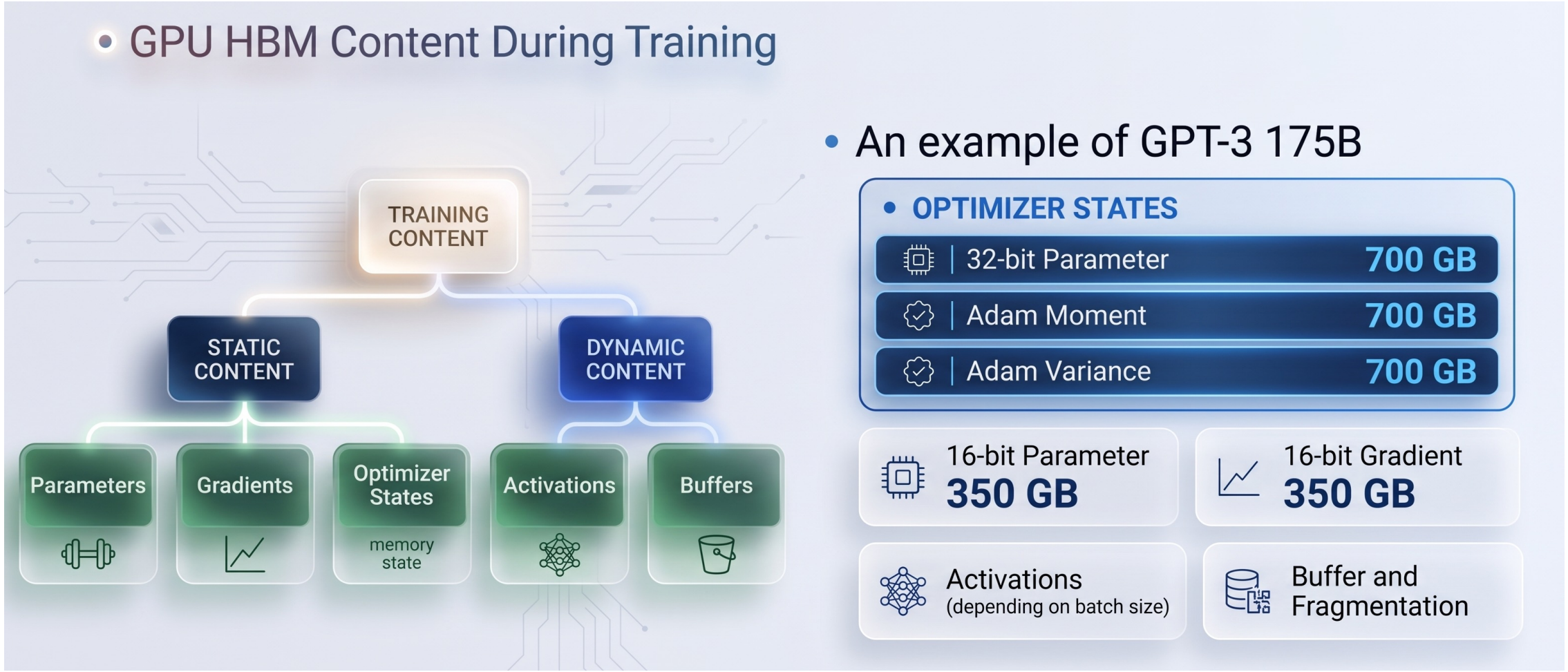


《Highly Scalable Deep Learning Training System with Mixed-Precision: Training ImageNet in Four Minutes》

Distributed LLM Training: Outline

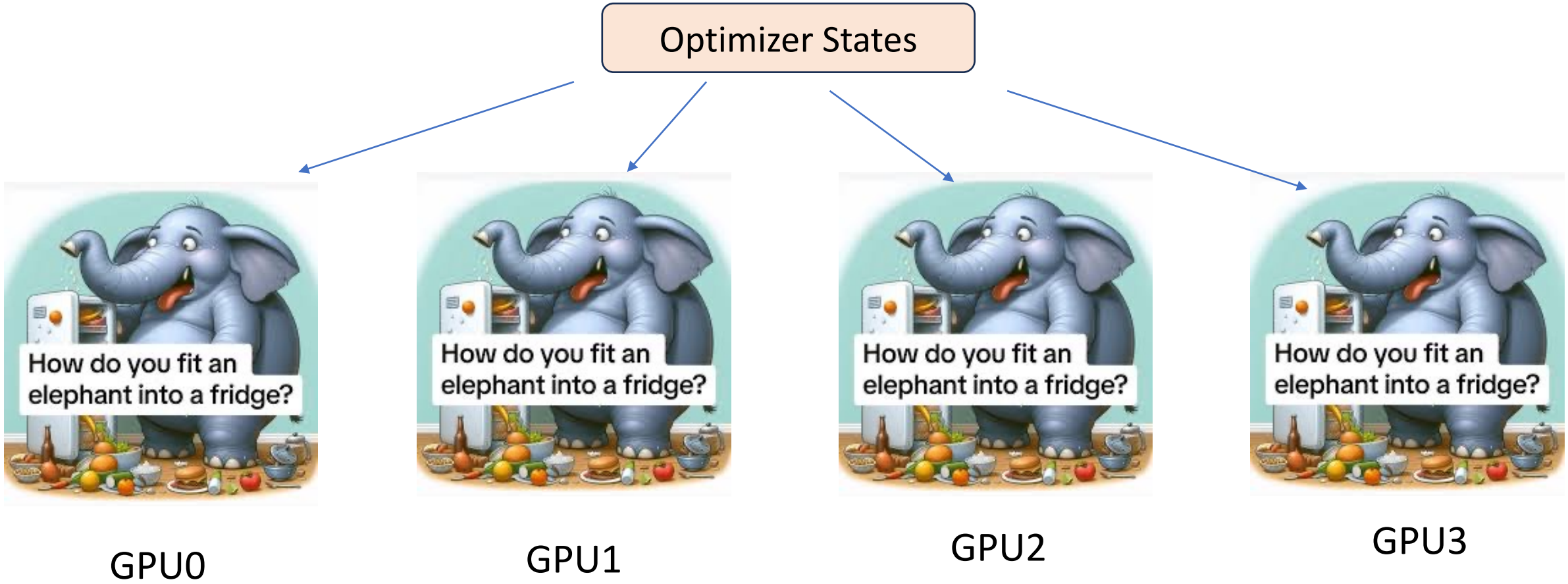
- Data Parallelism
  - Parameter-Server
  - All-Reduce
  - **Memory optimization**

LLM training: DP with Memory Optimization



LLM training: DP with Memory Optimization

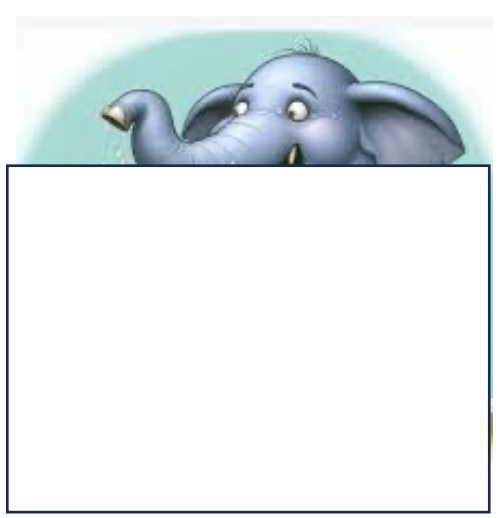
- How to put an elephant into a fridge?



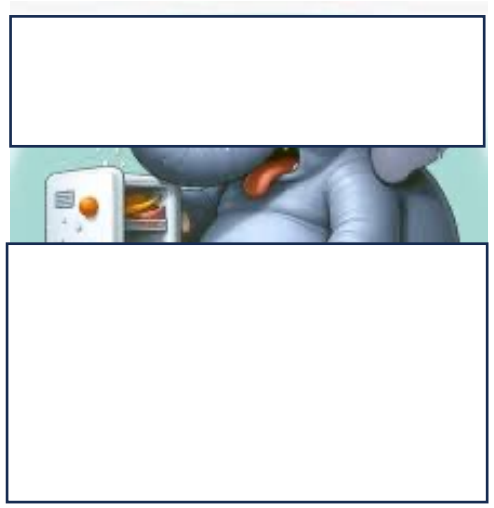
LLM training: DP with Memory Optimization

- How to put an elephant into a fridge?

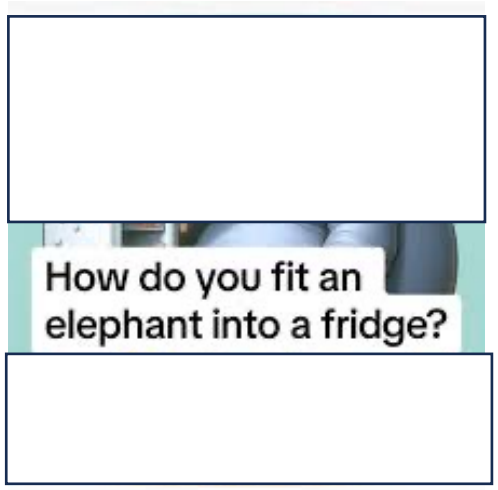
Sharding the elephant into four partitions, and each GPU HBM holds one chunk!



GPU0



GPU1

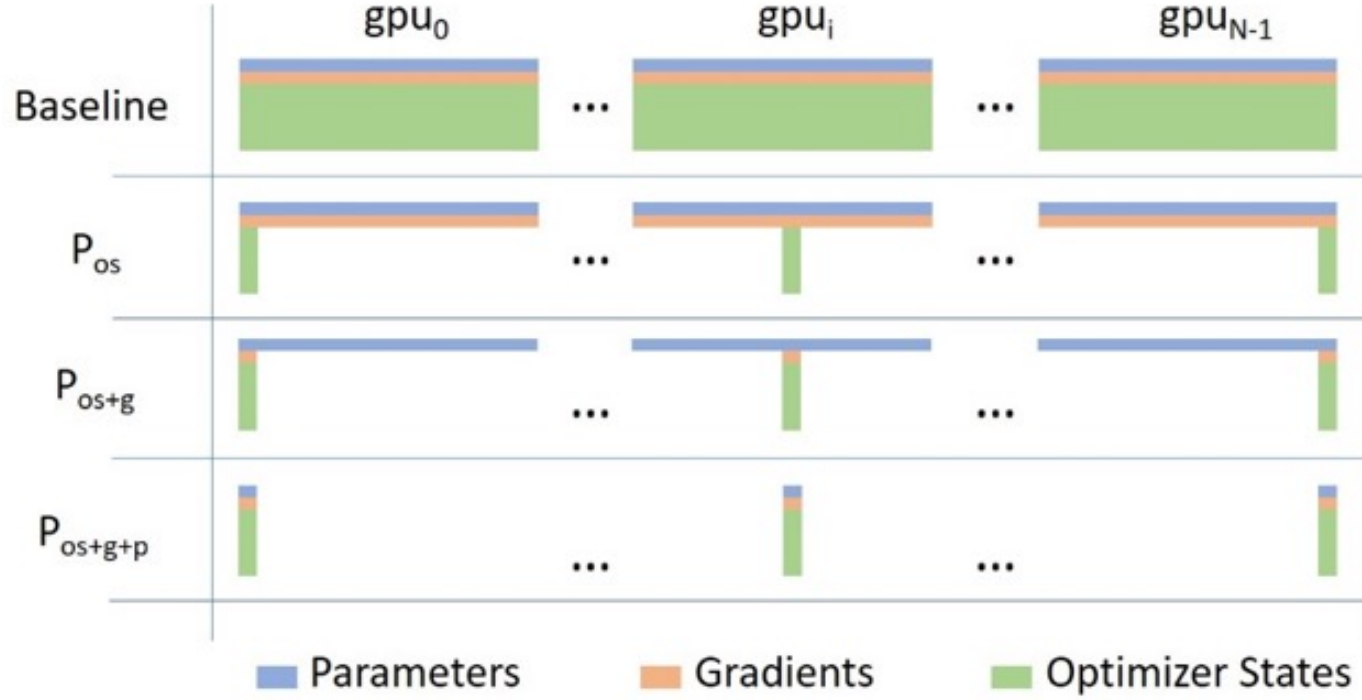


GPU2

GPU3

LLM training: DP with Memory Optimization

- ZeRO (Zero Redundancy) optimization: an overview



	Memory Consumption		Comm Volume
	Formulation	Specific Example K=12 Ψ=7.5B N_d=64	
Baseline	$(2+2+K)*\Psi$	120GB	1x
ZeRO-1 (P_{os})	$2\Psi+2\Psi+\frac{K*\Psi}{N_d}$	31.4GB	1x
ZeRO-2 (P_{os+g})	$2\Psi+\frac{(2+K)*\Psi}{N_d}$	16.6GB	1x
ZeRO-3 (P_{os+g+p})	$\frac{(2+2+K)*\Psi}{N_d}$	1.9GB	1.5x

HBM usage with DP degree 64

《Fit More and Train Faster With ZeRO via DeepSpeed and FairScale》

LLM training: DP with Memory Optimization

- How to compute the storage stage
  - ZeRO-1 (P_{OS})

$$\text{Model States (bytes)}|P_{os+g} = \overbrace{2 \times \Psi}^{\text{Param}} + \overbrace{\frac{14 \times \Psi}{N_d}}^{\text{Gradient + Optimizer}}$$
$$\approx 2\Psi | N_d \to \infty$$

  - ZeRO-2 (P_{OS+g})

$$\text{Model States (bytes)}|P_{os+g} = \overbrace{2 \times \Psi}^{\text{Param}} + \overbrace{\frac{14 \times \Psi}{N_d}}^{\text{Gradient + Optimizer}}$$
$$\approx 2\Psi | N_d \to \infty$$

  - ZeRO-3 (P_{OS+g+p})

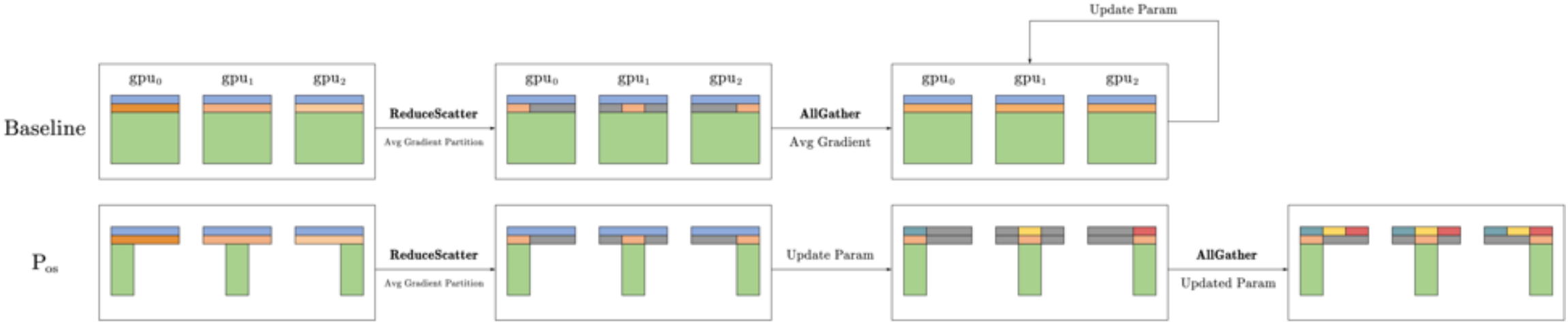
$$\text{Model States (bytes)}|P_{os+g+p} = \overbrace{\frac{16 \times \Psi}{N_d}}^{\text{Param + Gradient + Optimizer}}$$
$$\approx 0 | N_d \to \infty$$

LLM training: DP with Memory Optimization

- ZeRO-1&2
  - *Forward* is OK because each GPU holds the complete *parameter* ✓
  - *Backward* is OK because of the same reason ✓
  - *AllReduce* = ReduceScatter + AllGather
    - *ReduceScatter is OK* ✓
    - *Update optimizer state shards* ✓
      - *Incomplete optimizer states at every GPU*
    - *Update parameter shards* ✓
    - *AllGather remaining parameter shards to assembly the complete parameter* ✓

LLM training: DP with Memory Optimization

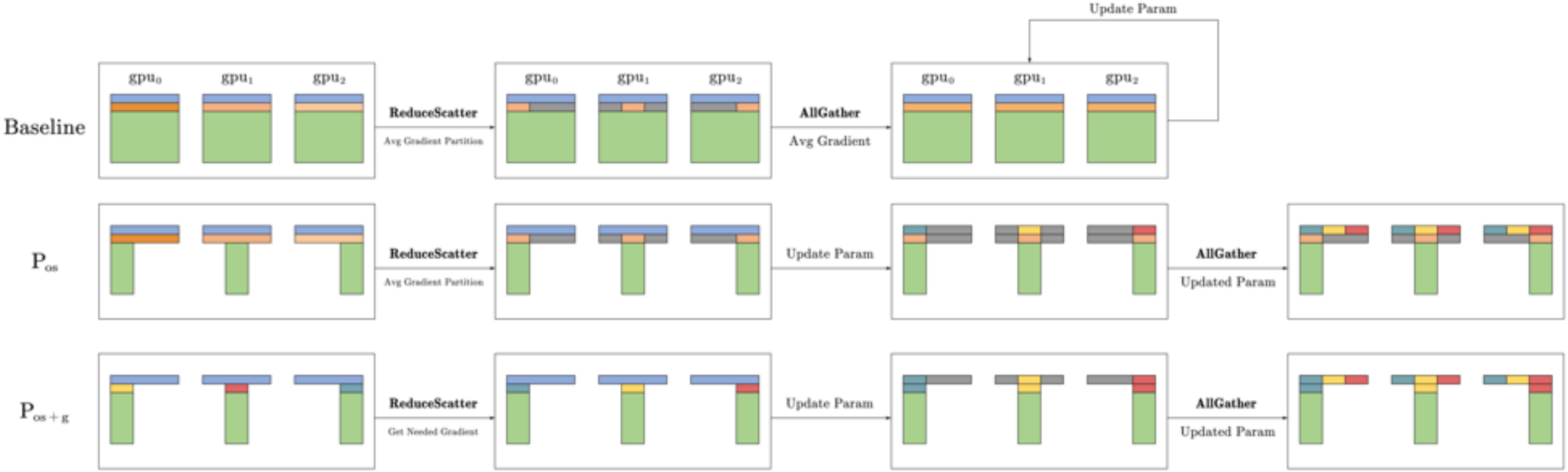
- ZeRO-1



- Using scatter-reduce and all-gather to obtain *global gradient shard*, and using global gradient shard to update the parameter and optimizer shard
- Using all-gather to obtain the complete global parameter

LLM training: DP with Memory Optimization

- ZeRO-2



- Similar to ZeRO-1, except not conserving all the global gradient shards

LLM training: DP with Memory Optimization

- ZeRO-3 (parameter, gradient and optimizer sharding)
  - *Forward* is ***NOT*** OK because of incomplete *parameter*
    - *Need to fetch parameter shards from other GPUs via BROADCAST*
  - *Backward* is ***NOT*** OK because of incomplete parameter
    - *Need to fetch parameter shards from other GPUs via BROADCAST again*
  - *Aggregating local gradient shards to obtain global gradient shards*
    - *Reduce Scatter is OK*
  - *Updating optimizer shards and parameter shards are OK*
    - *No "AllGather" operation afterwards*

LLM training: DP with Memory Optimization

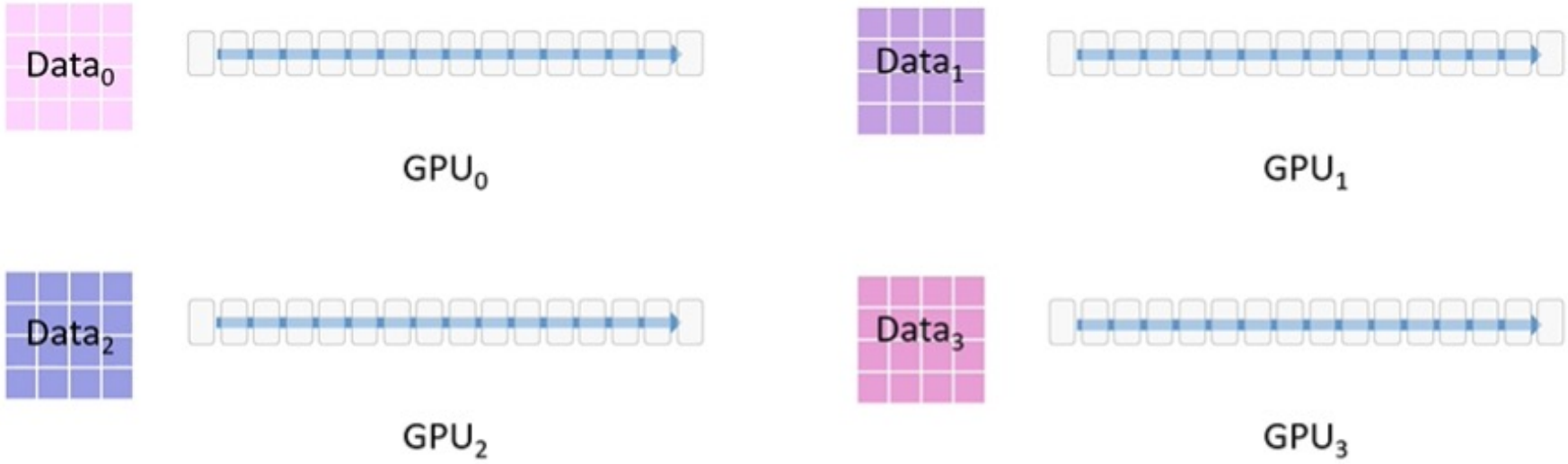
- ZeRO-3: An animation (Implementation on DeepSpeed could be somewhat different)

- All-Gather
  - Collecting parameters for forward propagation
- All-Gather
  - Collecting parameters for back propagation
- Reduce-Scatter
  - Obtaining global gradient
- Update OS and param.

Source: microsoft official site

LLM training: DP with Memory Optimization

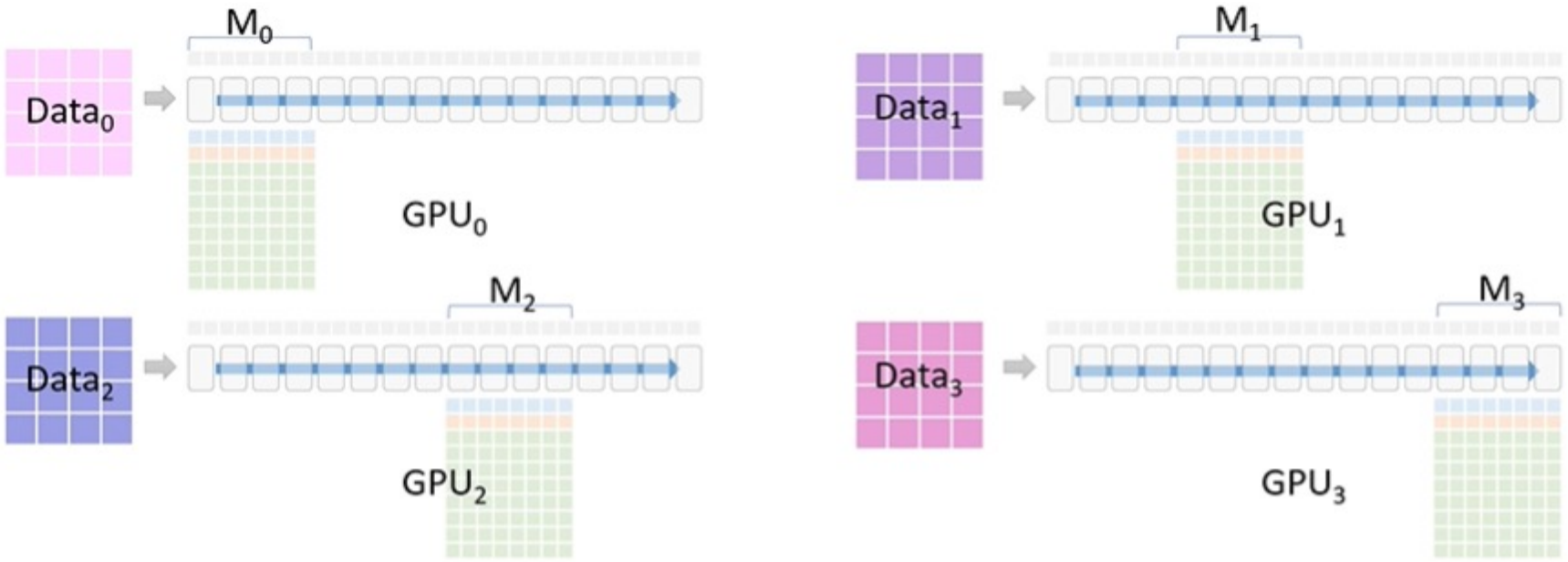
- ZeRO-3: An animation



Dataset is partitioned into four pieces, each of which stored at a GPU

LLM training: DP with Memory Optimization

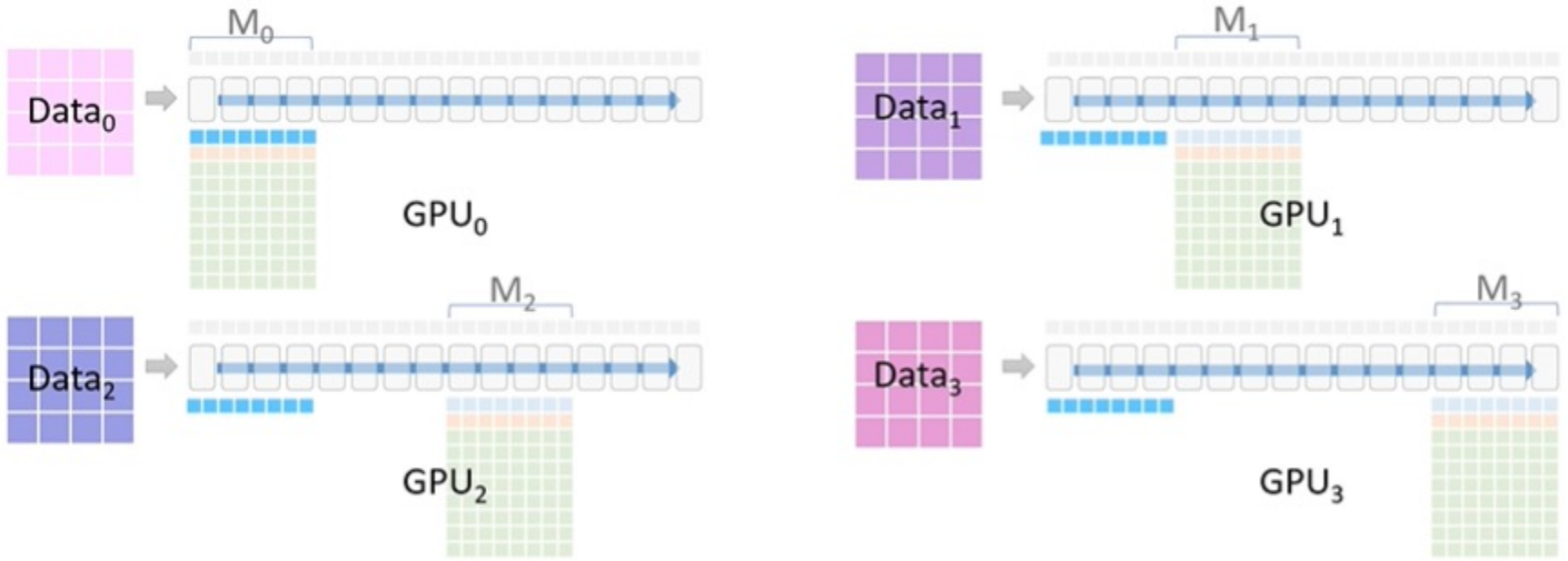
- ZeRO-3: An animation



Each GPU is responsible for one piece of the final model

LLM training: DP with Memory Optimization

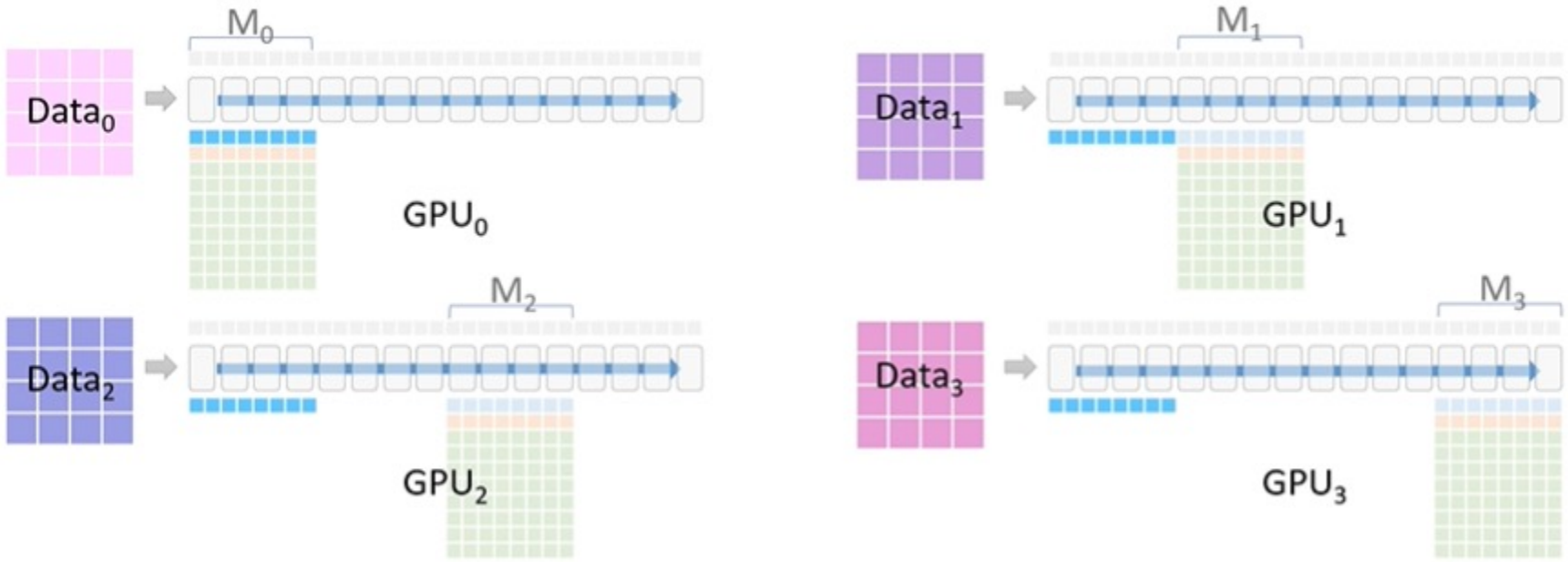
- ZeRO-3: An animation



Only GPU0 initially has the model parameters for M0, so it broadcasts them to GPU-1-2-3

LLM training: DP with Memory Optimization

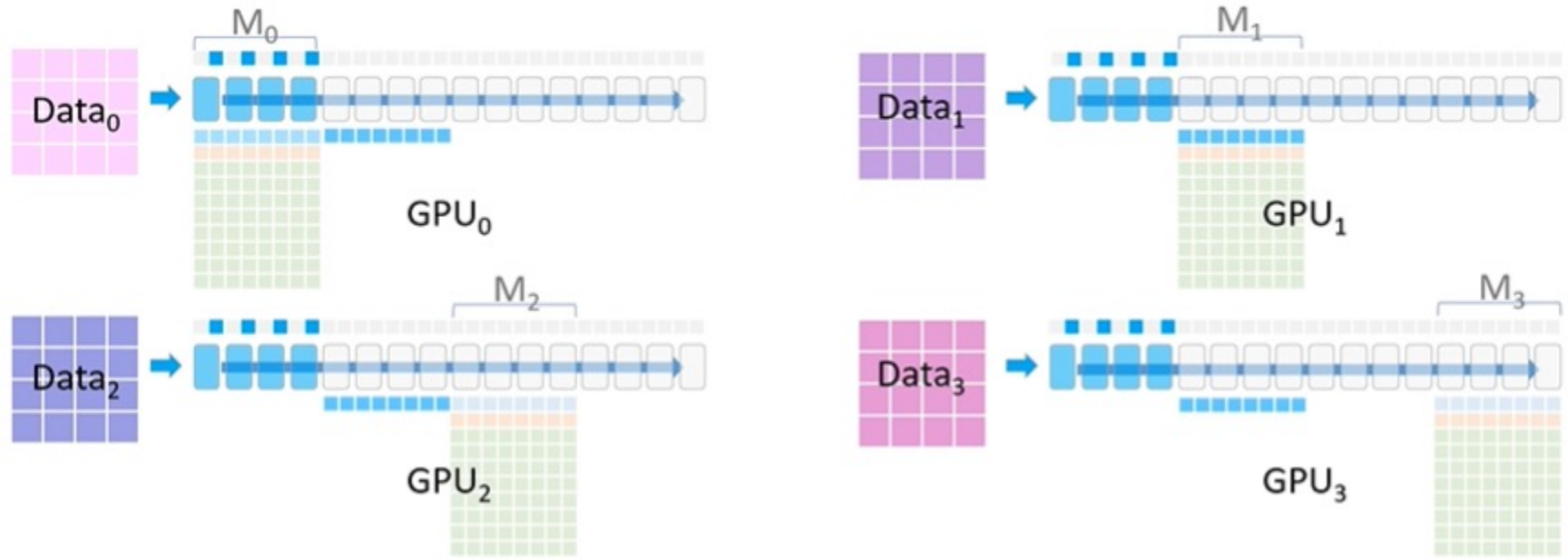
- ZeRO-3: An animation



GPU-1-2-3 store them in the temporary buffer and begin the forward propagation

LLM training: DP with Memory Optimization

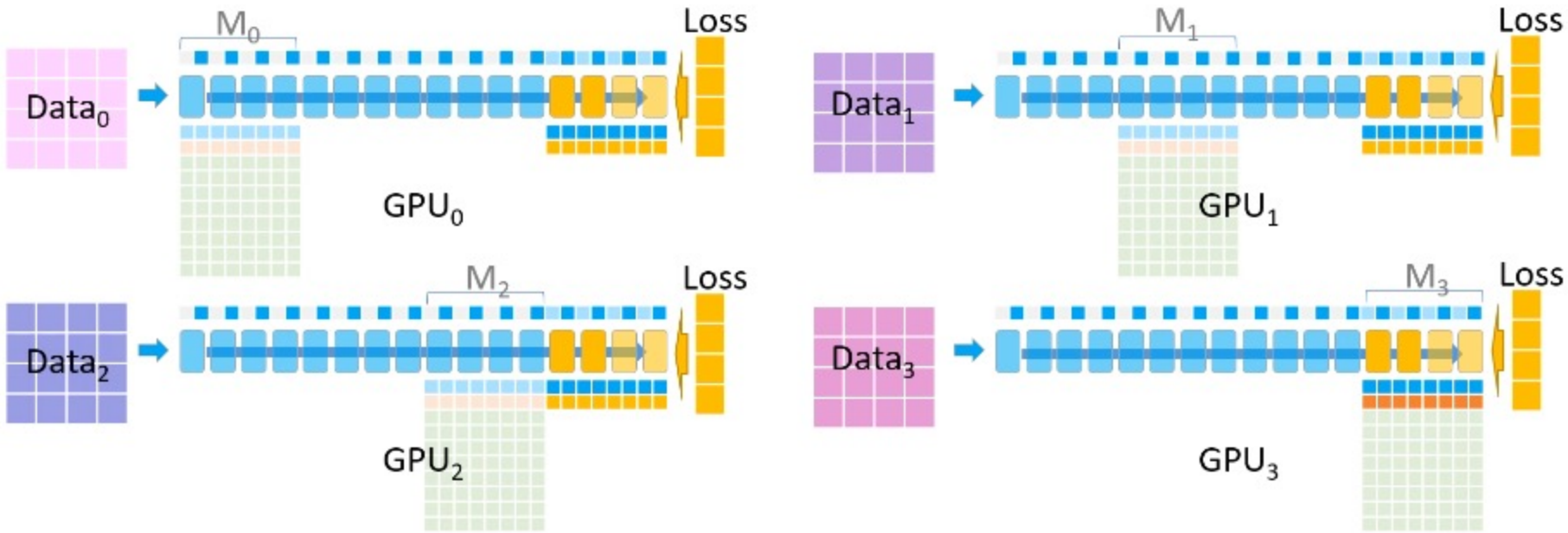
- ZeRO-3: An animation



GPU1 does the same thing, e.g. broadcasting M1 to GPU-0-2-3

LLM training: DP with Memory Optimization

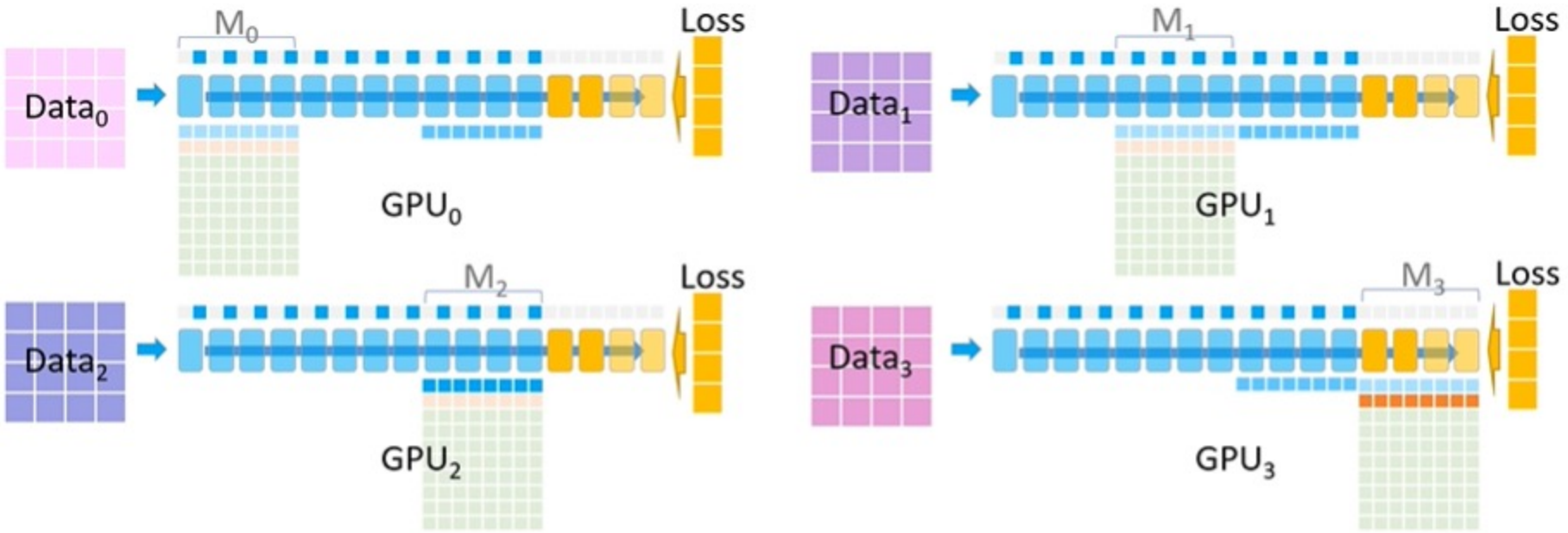
- ZeRO-3: An animation



Backward pass: GPU-0-1-2 pass their M3 gradient to GPU3, and GPU3 aggregate the gradients to obtain the global gradient shard so to generate the final M3 for all data

LLM training: DP with Memory Optimization

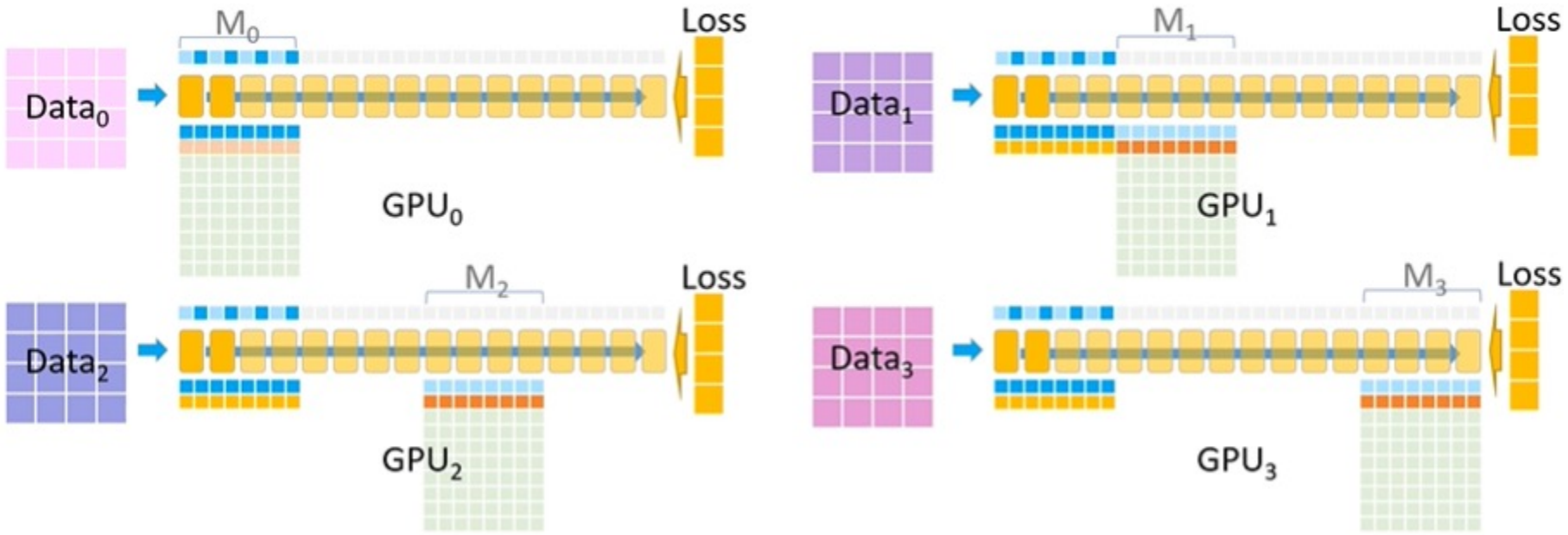
- ZeRO-3: An animation



Backward pass: GPU2 pass its M2 parameter to GPU-0-1-3, so that they can proceed backpropagation. GPU2 aggregates the gradient shards from GPU-0-1-3 and obtain the global gradient shard for M2.

LLM training: DP with Memory Optimization

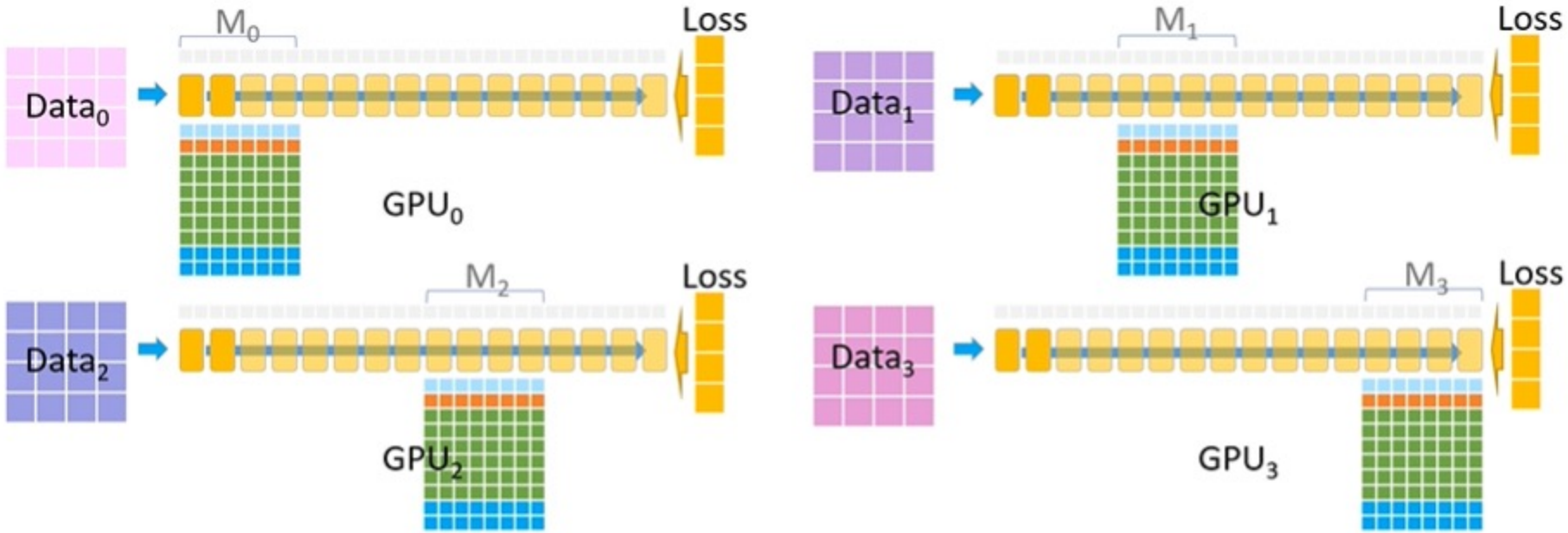
- ZeRO-3: An animation



Backward pass: The above steps repeat until GPU0 pass its M0 parameter to GPU-1-2-3 so that they can proceed the backpropagation. GPU0 aggregated the gradient shard for M0 parameter.

LLM training: DP with Memory Optimization

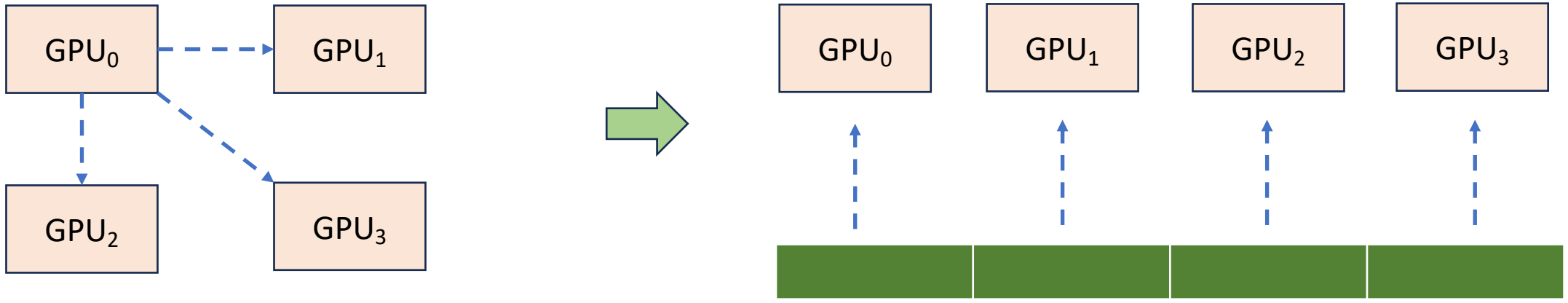
- ZeRO-3: An animation



Update phase: all the GPUs update their optimizer states and parameters in parallel.

LLM training: DP with Memory Optimization

- ZeRO-3: DeepSpeed Implementation
  - Replacing *broadcast* by *allrgather (why?)*
  - Intra-layer partitioning instead of inter-layer partitioning

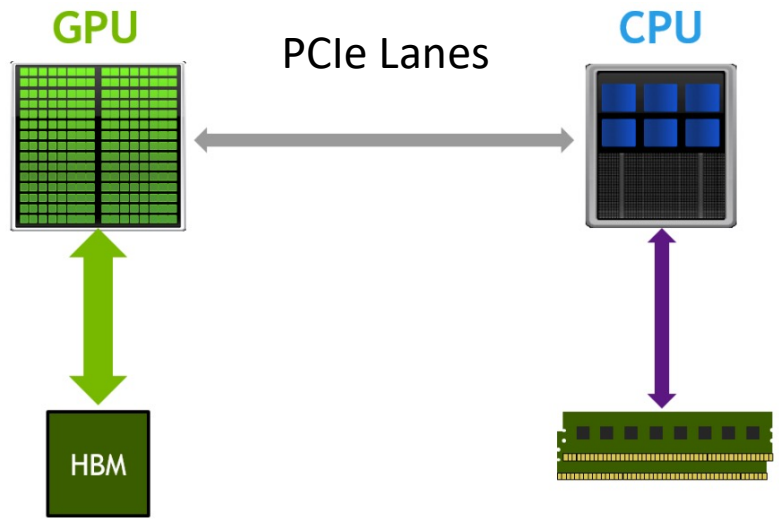


Broadcast

Intra-layer partitioning, and AllGather

LLM training: DP with Memory Optimization

- ZeRO Offload
  - When GPU HBM size is still insufficient to store partitions of optimizer states, parameter and gradients, can we still use it for training?

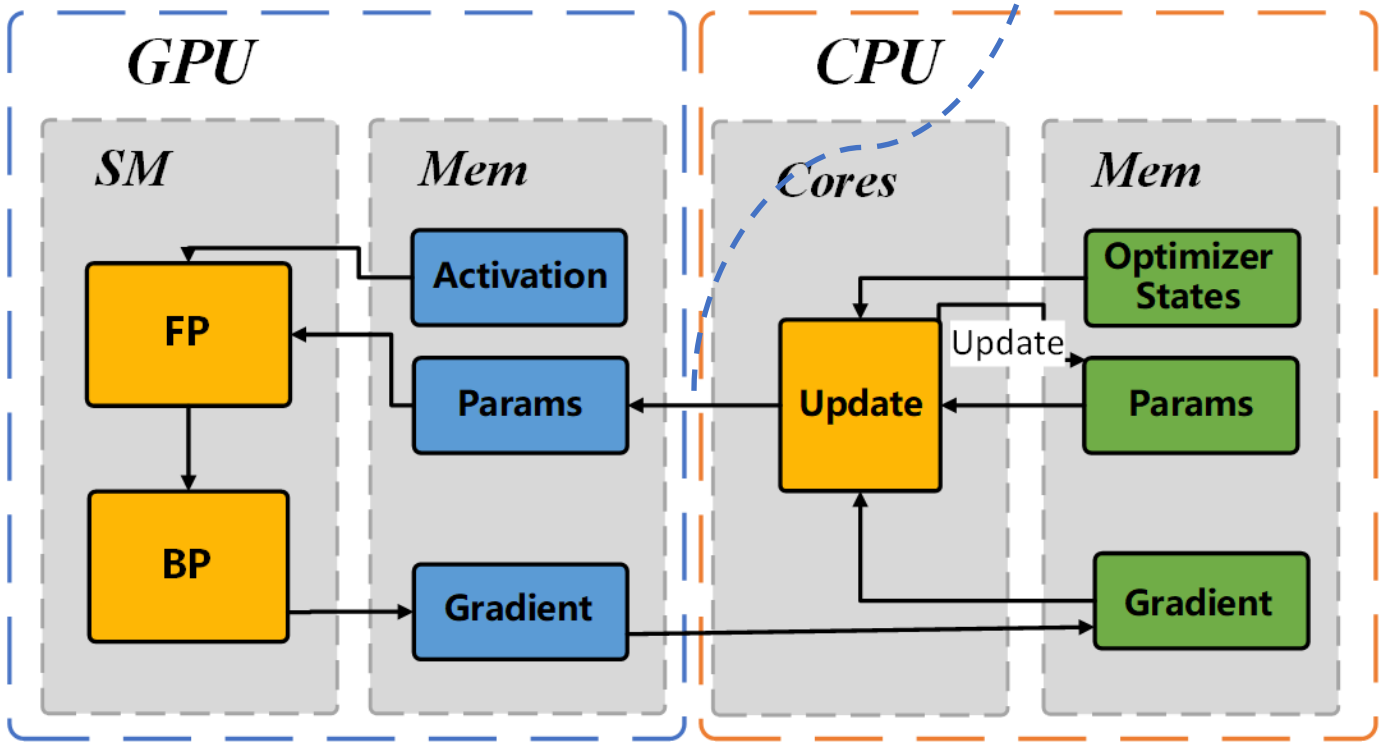


- CPU memory
  - ~1TB vs 80GB HBM
- Connection speed
  - PCIe 5.0 (16 lanes) ~ 64GB/s
  - NVIDIA H100 NVLink 4.0 ~ unidirectional 450GB/s
- GPU←→CPU bottleneck

LLM training: DP with Memory Optimization

- ZeRO Offload

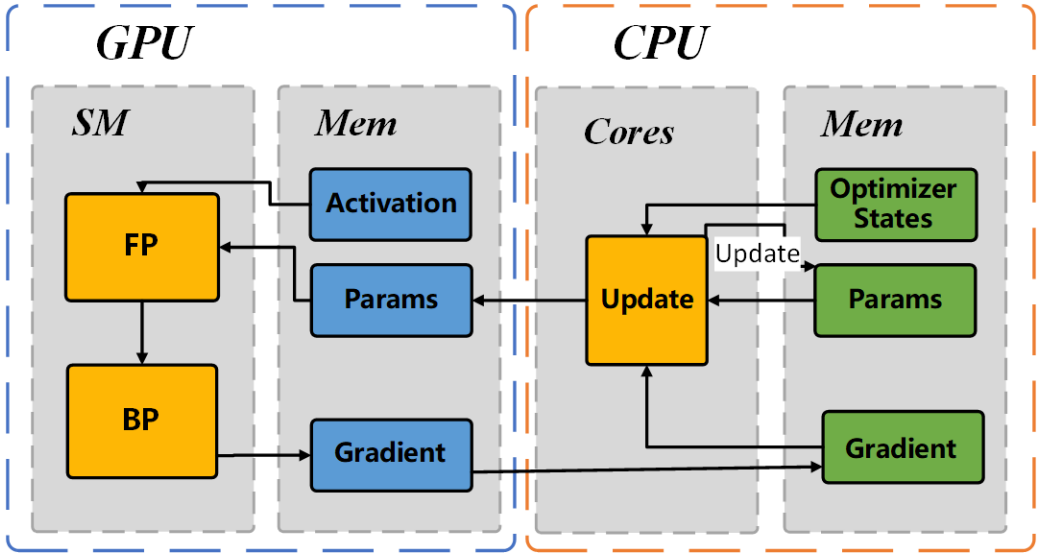
PCIe communications back and forth



- GPU HBM
  - Parameter
  - Gradient
  - Activations
- CPU Memory
  - Optimizer States
  - Parameter
  - Gradient

LLM training: DP with Memory Optimization

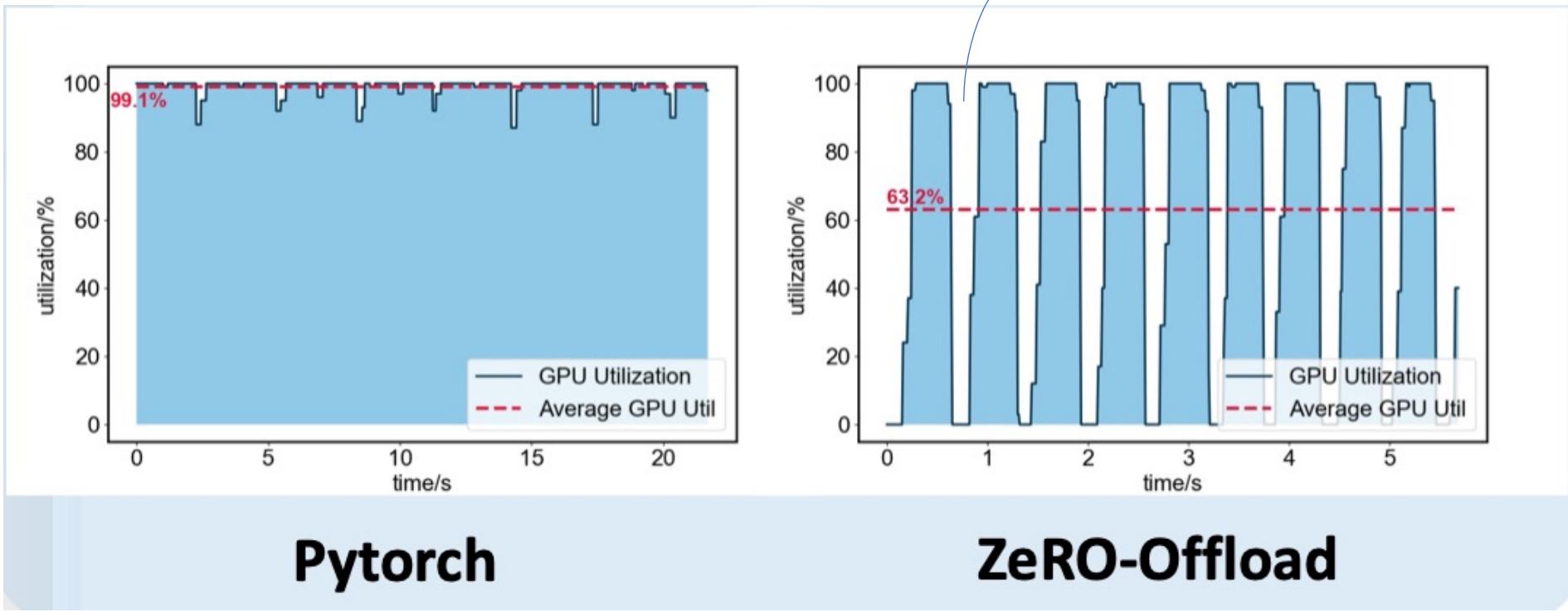
- ZeRO Offload: workflow
  - Forward propagation at **GPU HBM**
  - Backward propagation at **GPU HBM**
  - Optimizer state is at CPU main memory
  - Transmit gradient to from GPU to CPU
  - Update optimizer states at CPU memory (which is relatively slow)
  - Update parameter at CPU memory
  - Transmit new parameter from CPU to GPU, and repeat



LLM training: DP with Memory Optimization

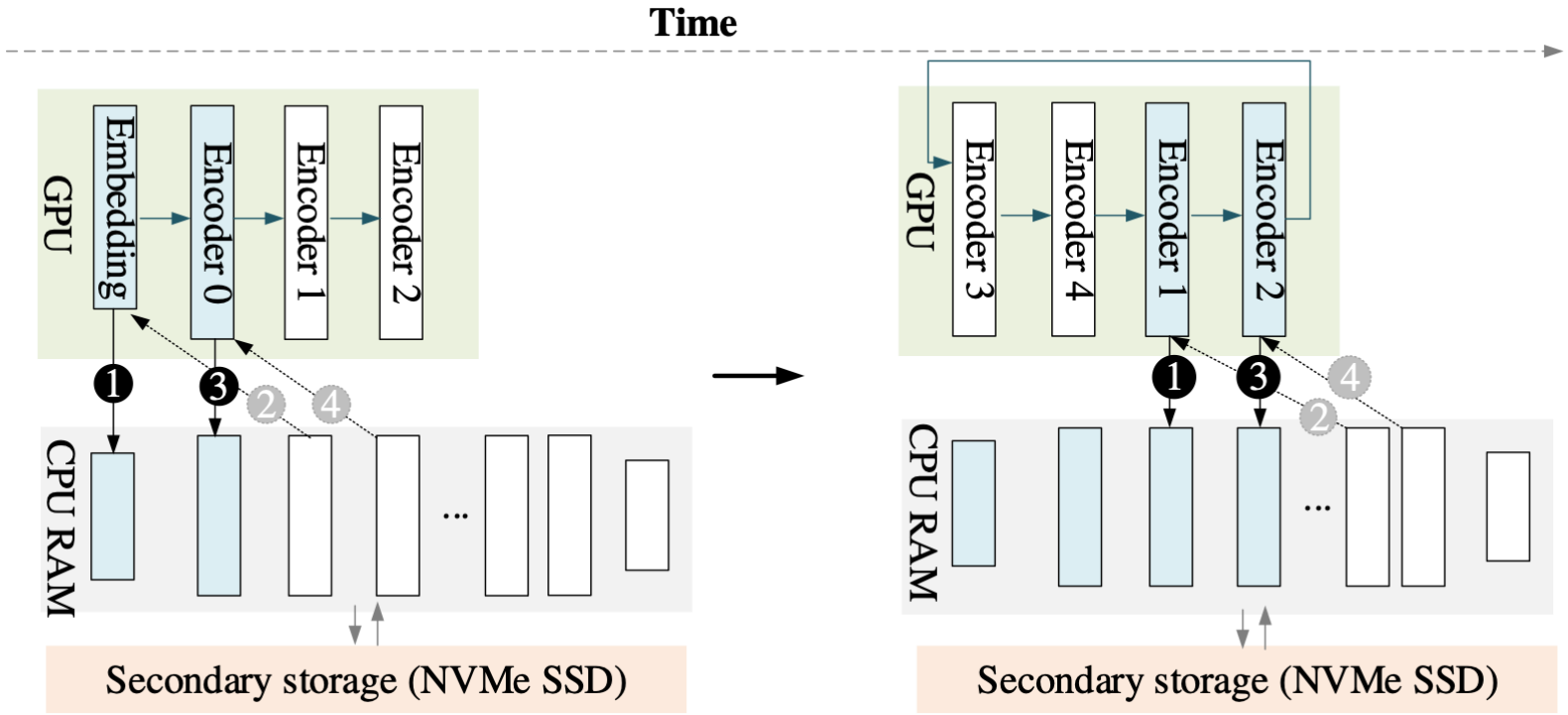
- Pros and Cons
  - Training GPT2 on A100: reduced GPU utilization

Where are these bubbles coming from?



LLM training: DP with Memory Optimization

- ZeRO Offload Variants

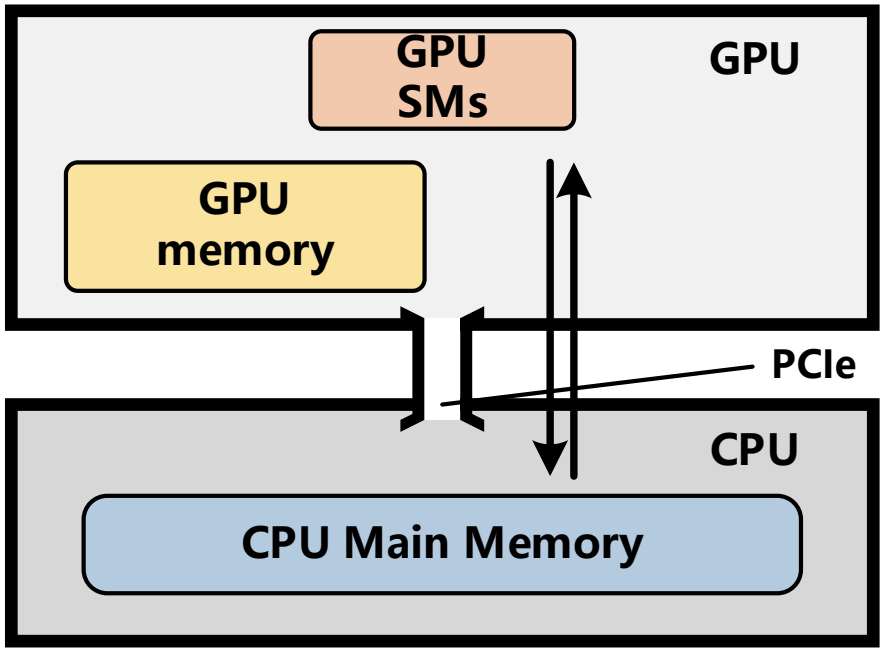


STRONGHOLD stores some DNN layers in the GPU memory and swapping out the finished layer states to the CPU RAM.

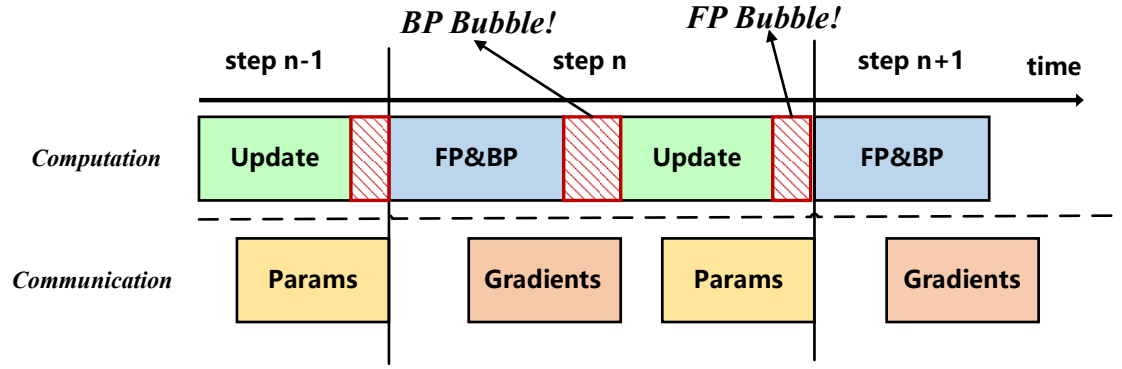
《STRONGHOLD: Fast and Affordable Billion-Scale Deep Learning Model Training》

LLM training: DP with Memory Optimization

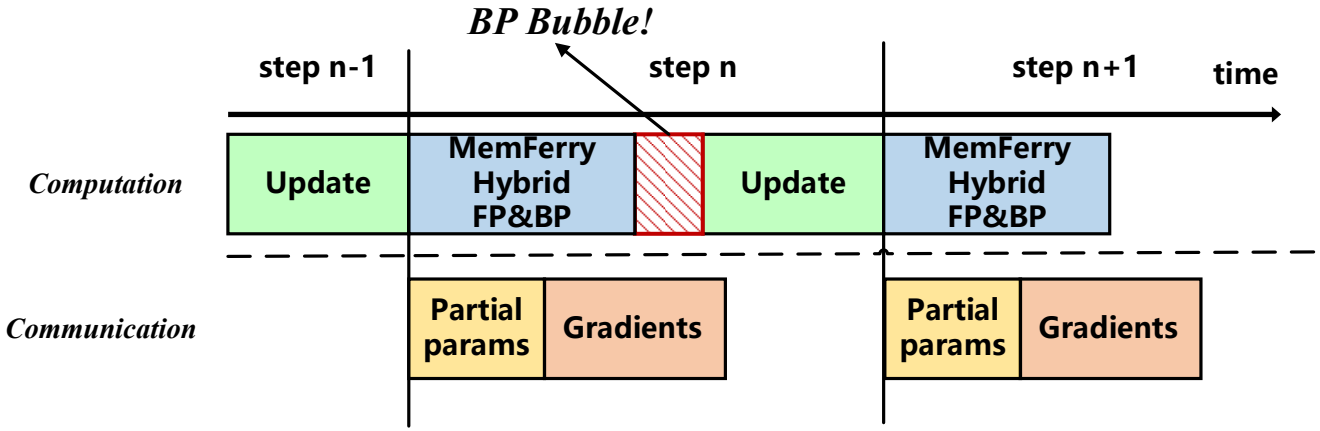
- ZeRO Offload Variants



DHA mode: GPU SMs compute tensors stored in CPU memory



Eliminating bubbles

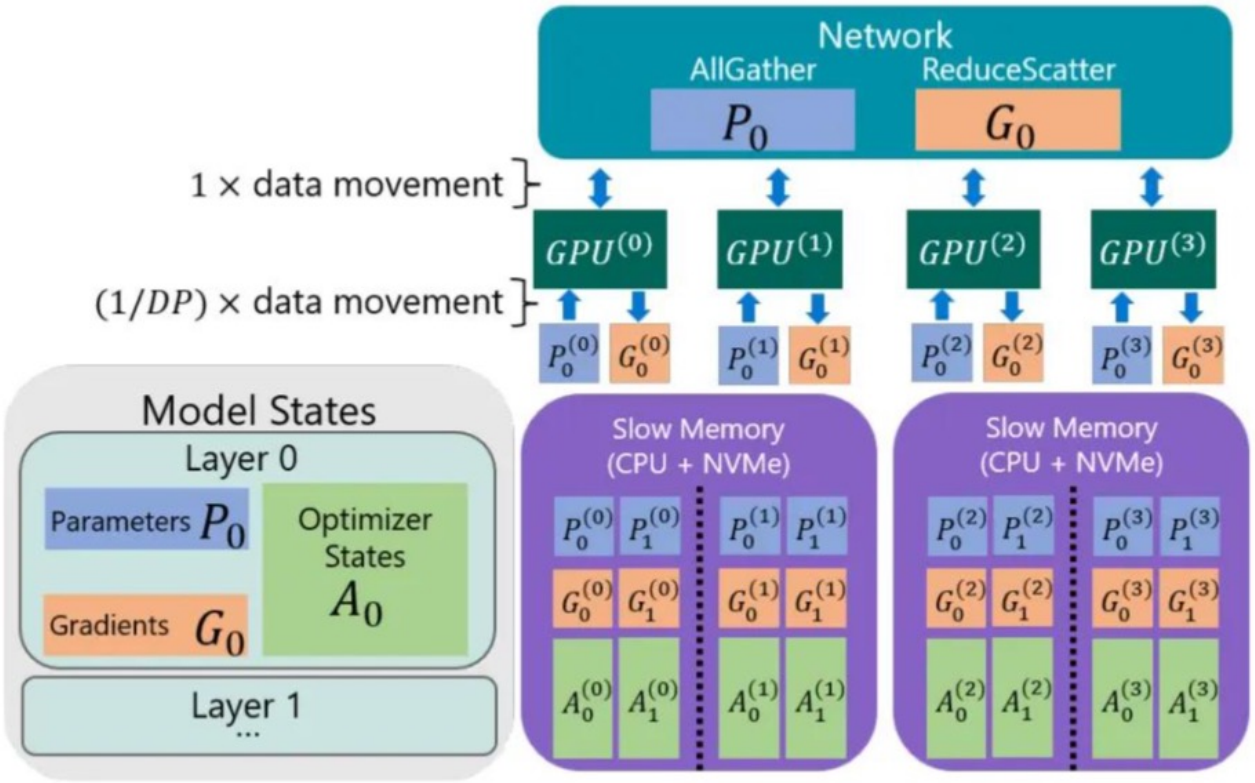


Eliminating bubbles

《MemFerry: A Fast and Memory Efficient Offload Training Framework with Hybrid GPU Computation》

LLM training: DP with Memory Optimization

- ZeRO Infinity (for your reference)

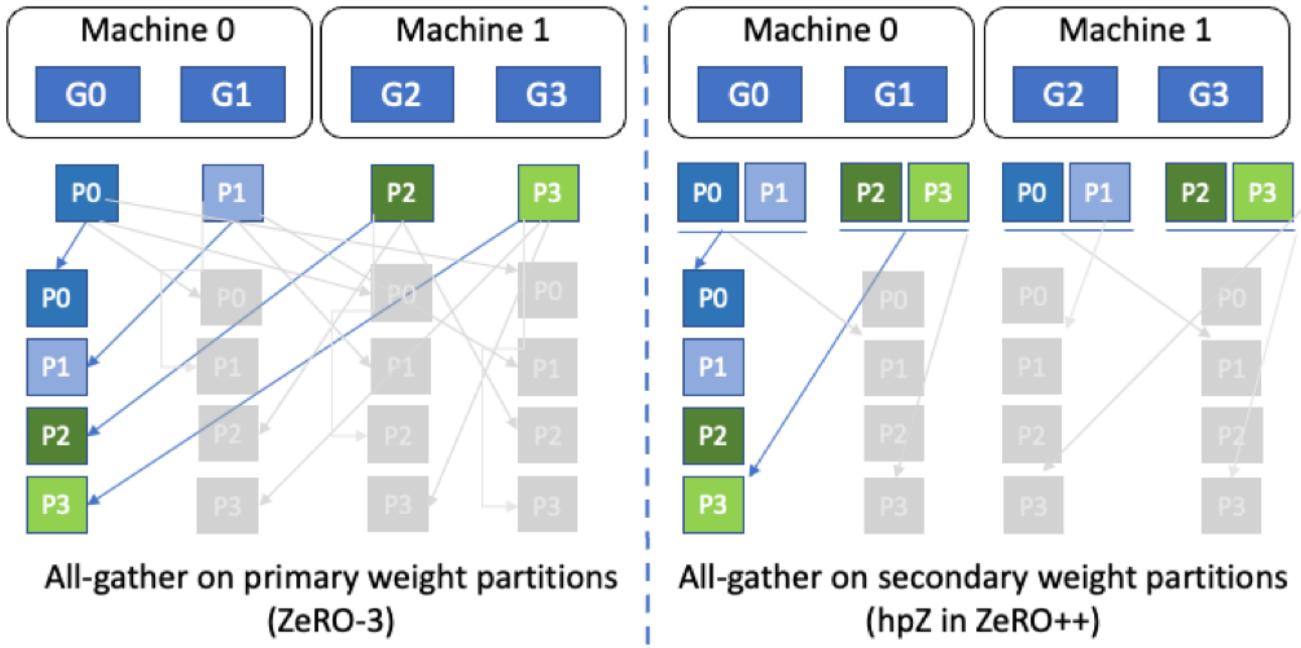


- Bandwidth
  - GPU > CPU > NVMe
- Read Data
  - Read parameter using All-Gather
- Data Path
  - CPU → GPU
  - NVMe → CPU → GPU

《ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning》

LLM training: DP with Memory Optimization

- ZeRO++: Low-bandwidth Scenario (for your reference)



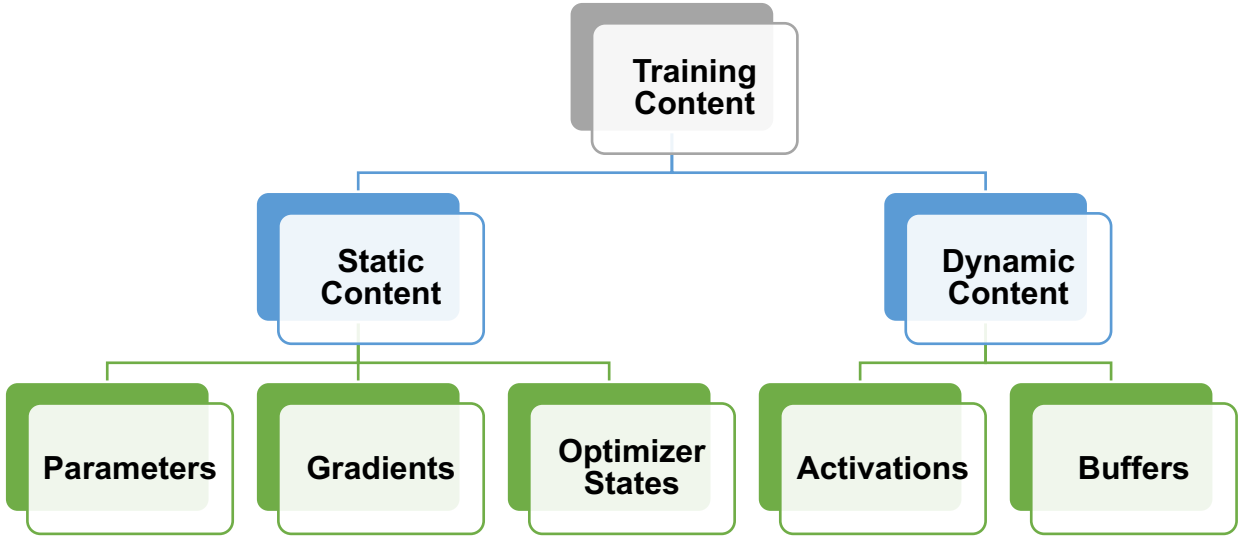
Hierarchical partitioning (not diving into details)

- Quantized Weight Communication
  - FP16 weight -> INT8 weight before All-Gather
  - block-quantization based all-gather in FP
  - INT8 -> FP16 after All-Gather
- Hierarchical Partitioning
  - (trying to) eliminate the inter-node all-gather
- Quantized Gradients Communication
  - Standard Reduce-Scatter involves a lot of quantization and dequantization steps
  - leverages all-to-all collectives to implement quantized reduce- scatter

《ZeRO++: Extremely Efficient Collective Communication for Giant Model Training 》

LLM training: DP with Memory Optimization

- Retrospect: GPU HBM Content During Training



- An example of GPT-3 175B
  - Optimizer States
    - 32-bit Parameter (700 GB)
    - Adam Moment (700 GB)
    - Adam Variance (700 GB)
  - 16-bit Parameter (350 GB)
  - 16-bit Gradient (350 GB)
  - **Activations (depending on batch size)**
  - Buffer and Fragmentation

LLM training: DP with Memory Optimization

- Activation is non-negligible
  - Forward propagation
  - Backward propagation
  - Size of activation
    - Standard transformer: b - batch size, s - sequence length, h - hidden layer dimension , a - head number

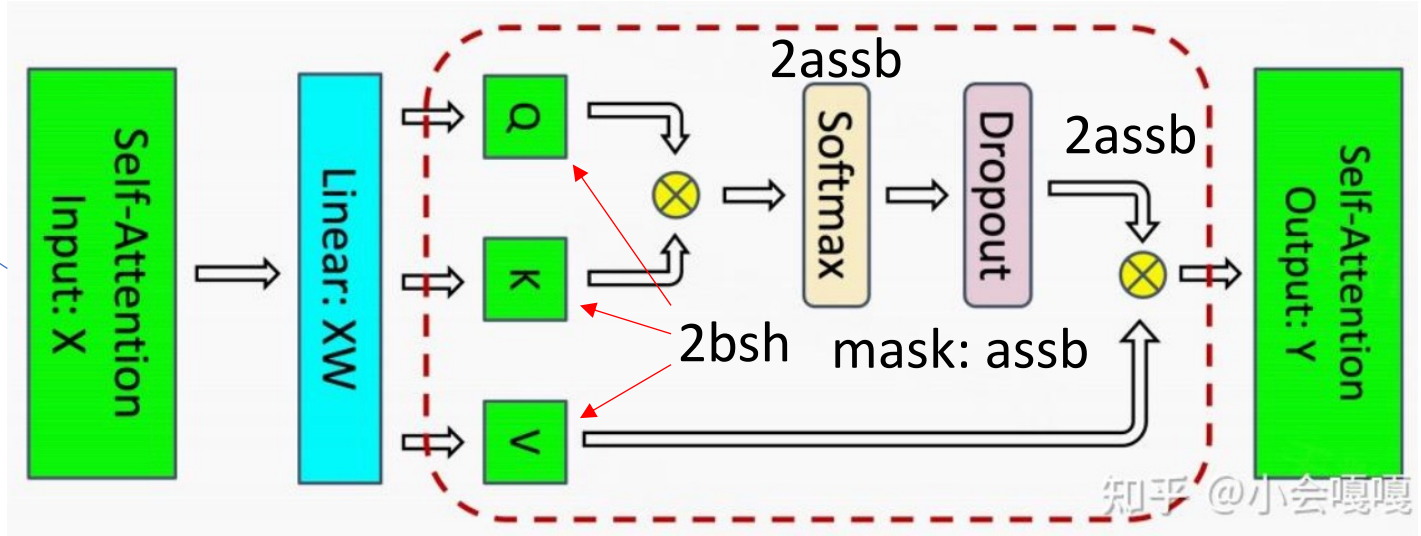
Output of a previous layer, intermediate variable

$$y = Wx + b$$

$$\frac{dL}{dW} = \frac{dL}{dy}x$$

Cannot be discarded after FP

bsh * 2 Bytes



In-total: 5bss + 8sbh

LLM training: DP with Memory Optimization

- Activation is non-negligible
  - Forward propagation
  - Backward propagation
  - Size of activation
    - Standard transformer: b - batch size, s - sequence length, h - hidden layer dimension, a - head number

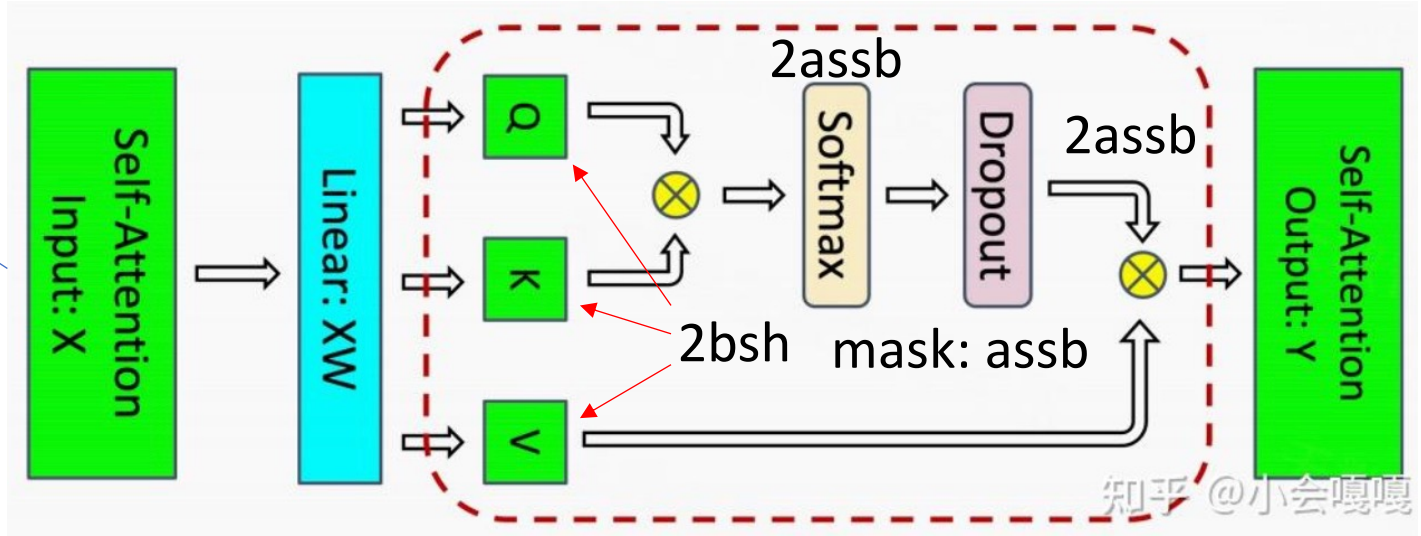
Output of a previous layer, intermediate variable

$$y = Wx + b$$

$$\frac{dL}{dW} = \frac{dL}{dy}x$$

Cannot be discarded after FP

bsh * 2 Bytes

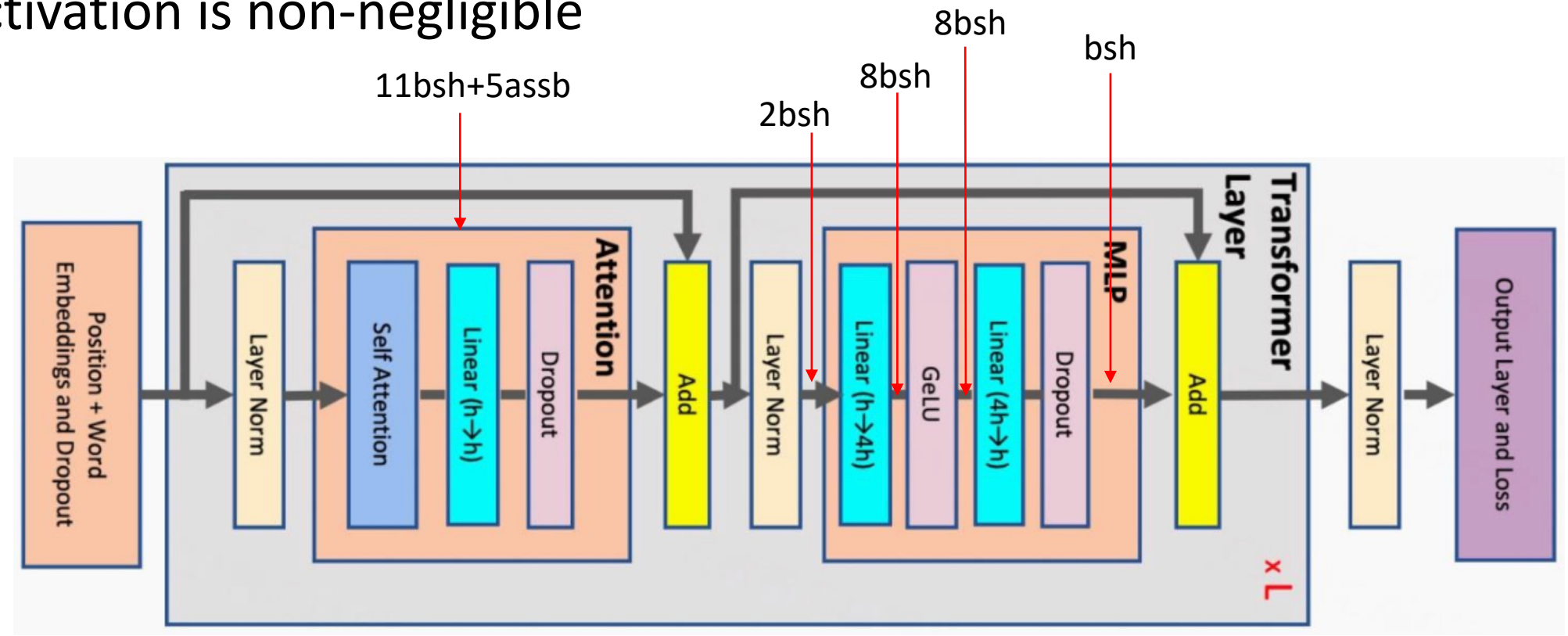


3sbh: Linear layer + dropout layer

In-total: 5abss + 11sbh

LLM training: DP with Memory Optimization

- Activation is non-negligible



In-total: 5abss + 34sbh

From Zhihu blogger and《Reducing Activation Recomputation in LLMs》

LLM training: DP with Memory Optimization

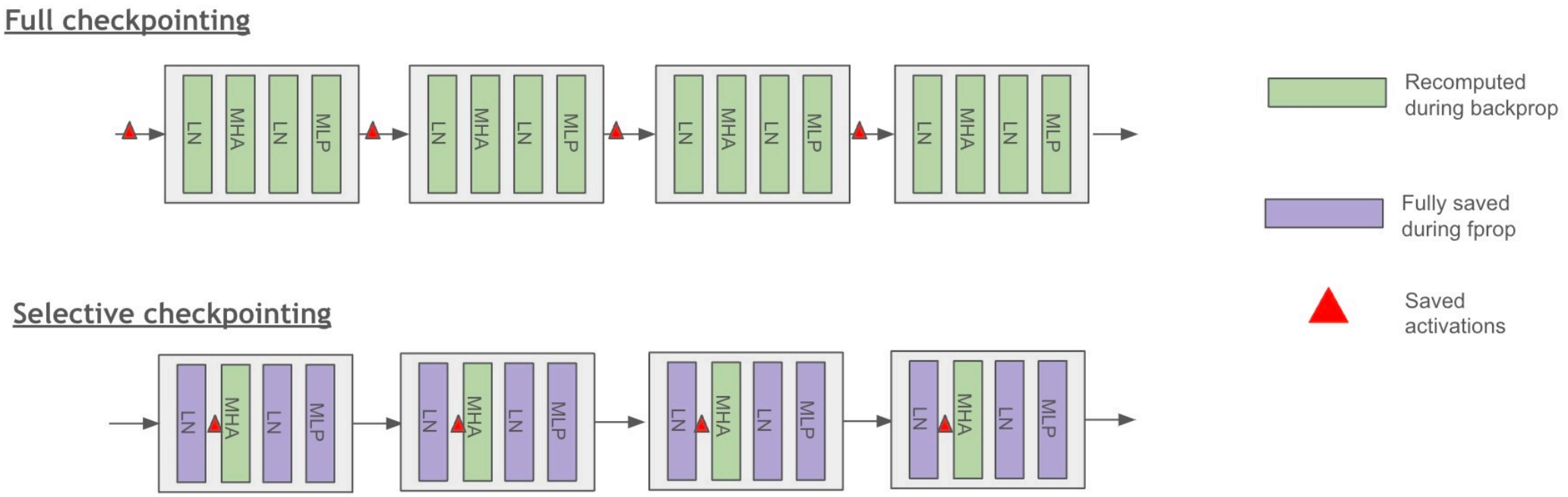
- Activation is non-negligible
  - b: 3.2M tokens/sequence length, s: 2048 tokens, h: 12288, l: 96 layers
  - Activation size ?
  - Full activation re-computation: only keeping the initial input and recompute everything
    - Minimal memory occupation
    - Prolonged training time (doing forward propagation once again) by 30%~40%
  - Goal of strategic recomputation (not the scope of this class)
    - Significantly reducing memory occupation while slightly increasing training time
    - Softmax and Softmax dropout are more suitable to be re-computed

https://lambda.ai/blog/demystifying-gpt-3

《Reducing Activation Recomputation in LLMs》

LLM training: DP with Memory Optimization

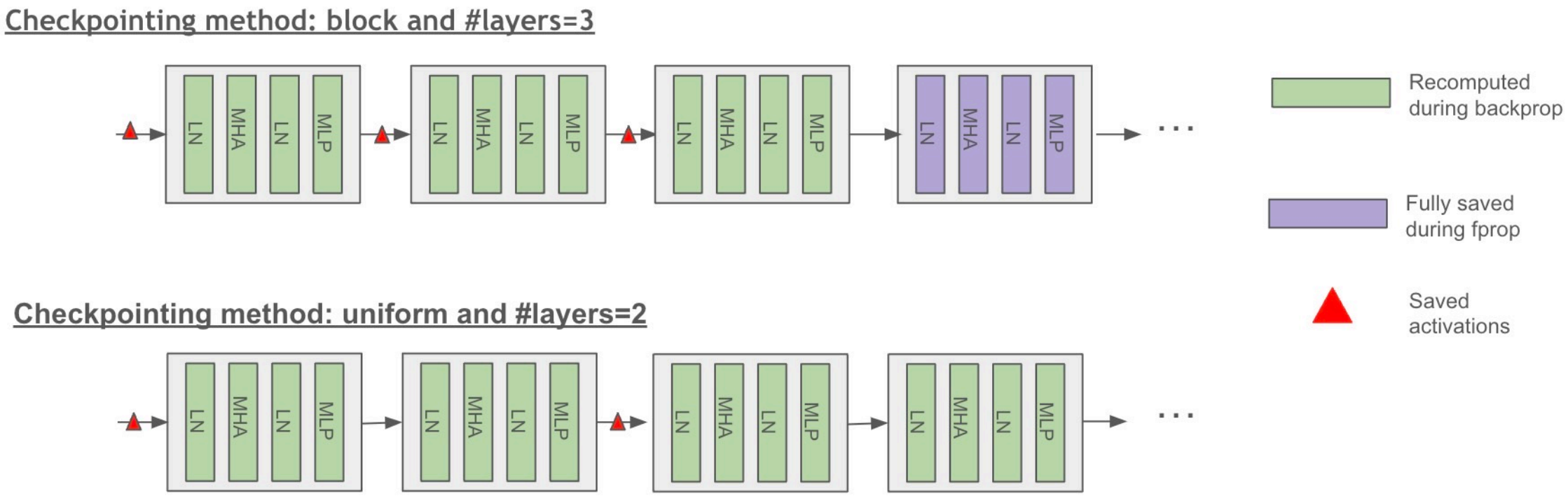
- Activation recomputation (simple)



setting *activations_checkpoint_granularity = selective or full*

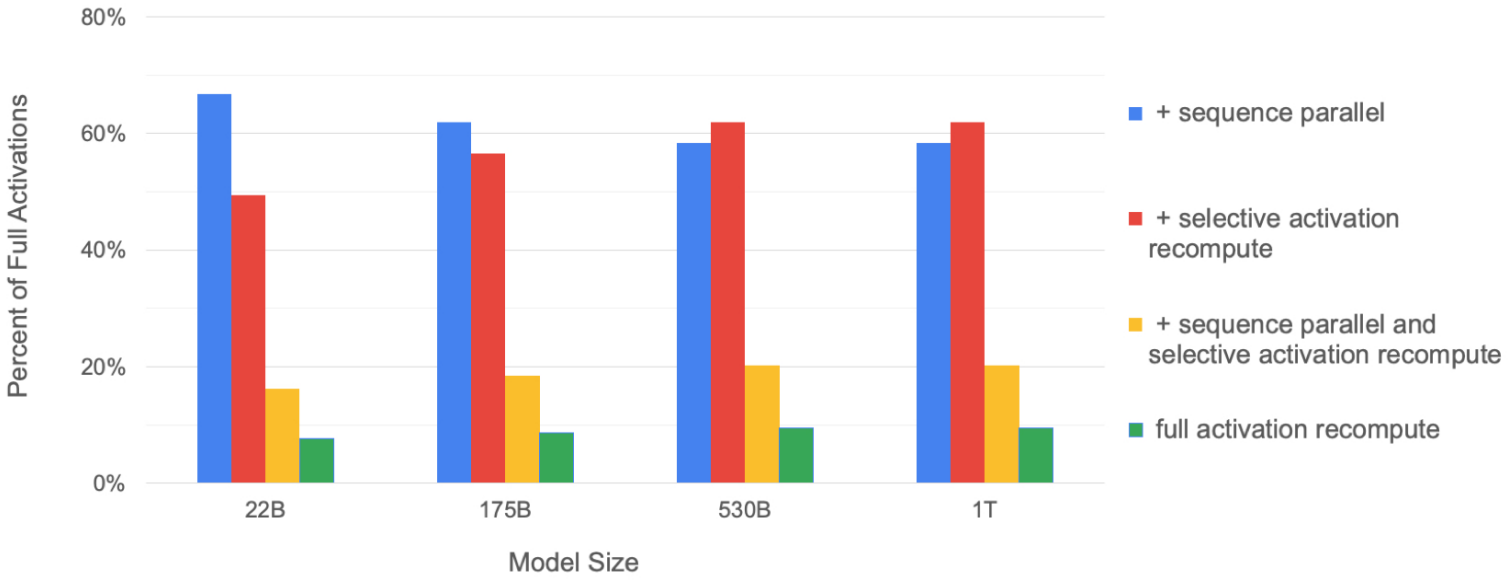
LLM training: DP with Memory Optimization

- Activation recomputation (simple)

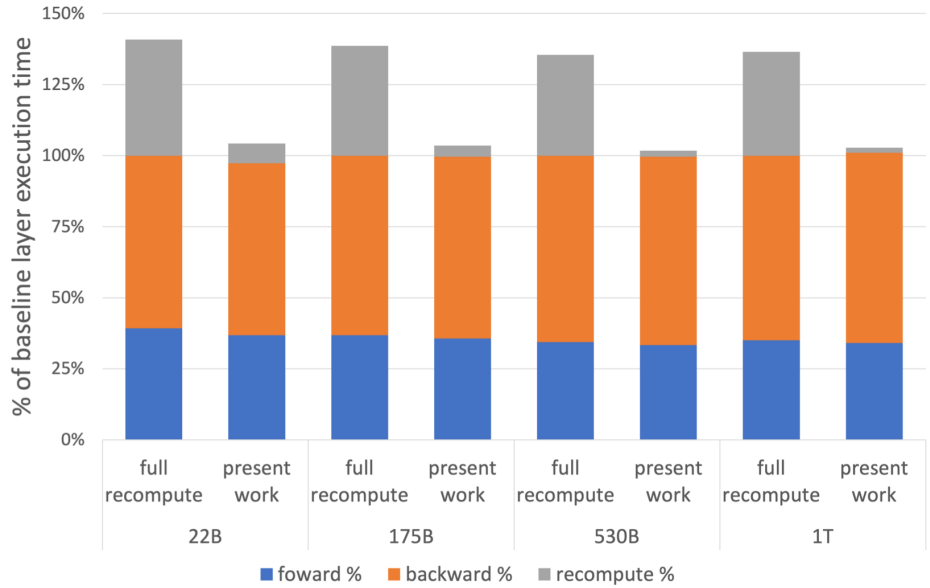


LLM training: DP with Memory Optimization

- Advantage of Re-computation



Recomputation saves memory footprint



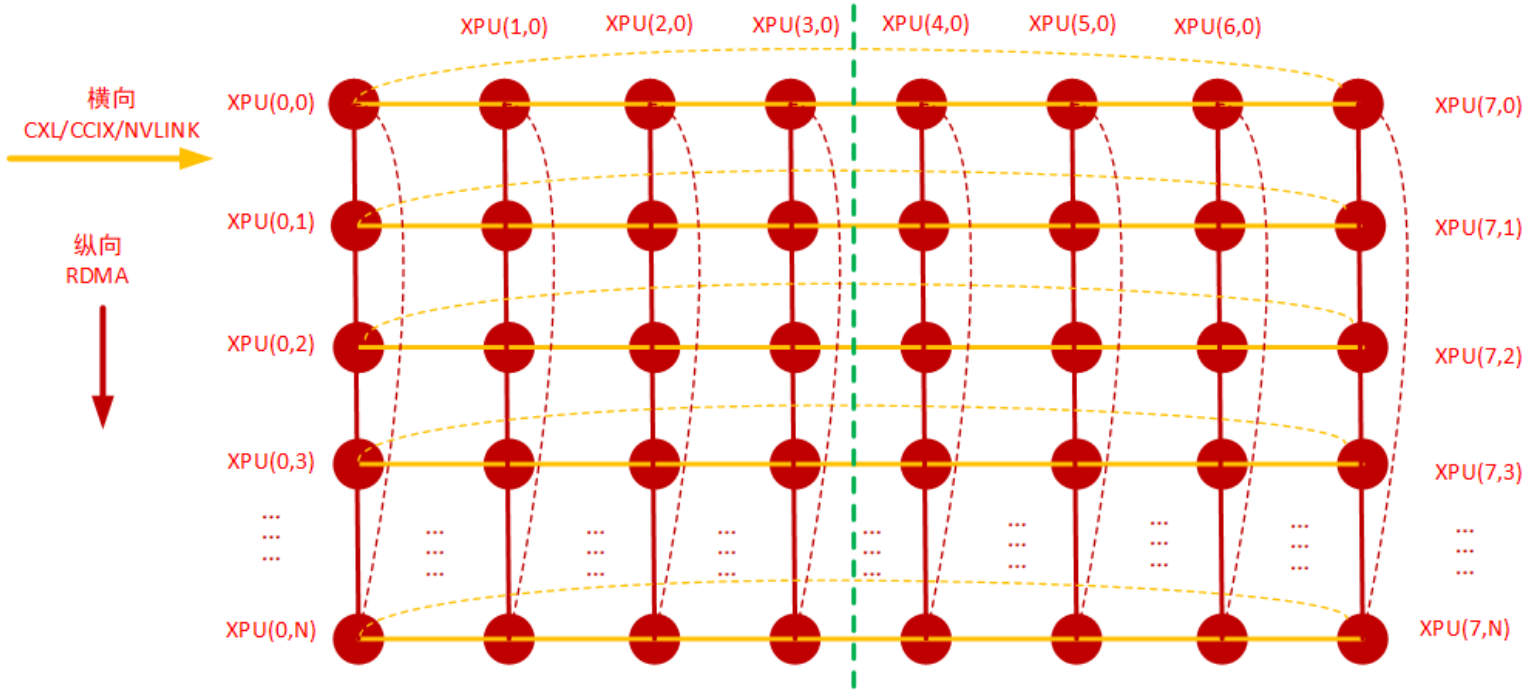
Recomputation brings more compute loads

《Reducing Activation Recomputation in Large Transformer Models》

Thanks!

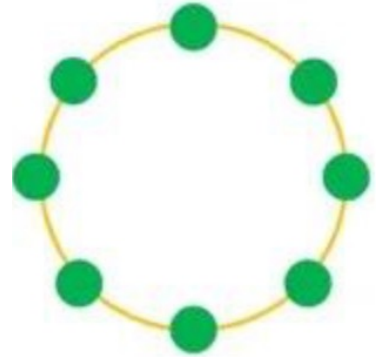
All-Reduce for LLM training

- 2D Torus All-Reduce (for reference)

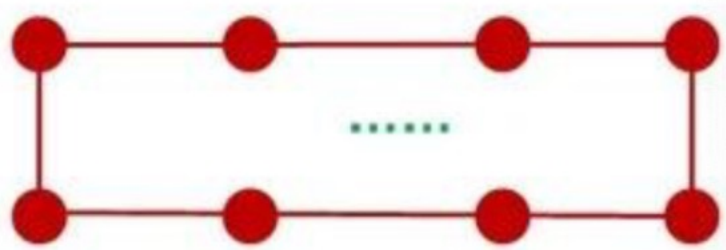


Horizontal: intra-machine GPU interconnect via NVLink/CXL
Vertical: inter-machine GPU interconnect via RDMA NICs

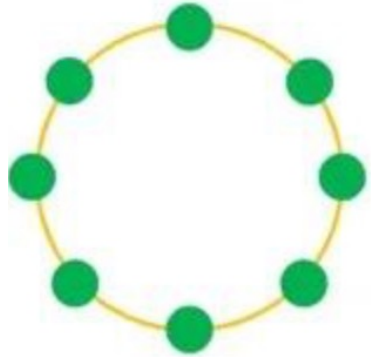
Intra-machine NVLink/CXL; Inter-machine RDMA (≥ 2 NICs)
Designed by SONIC



Step 1: Intra-ring Reduce-Scatter



Step 2: Inter-ring All-Reduce

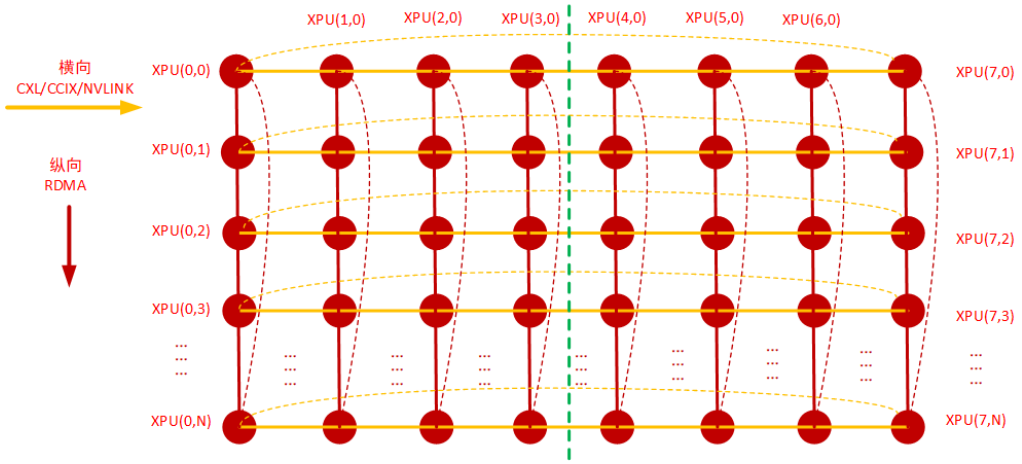


Step 3: Intra-ring All-Gather

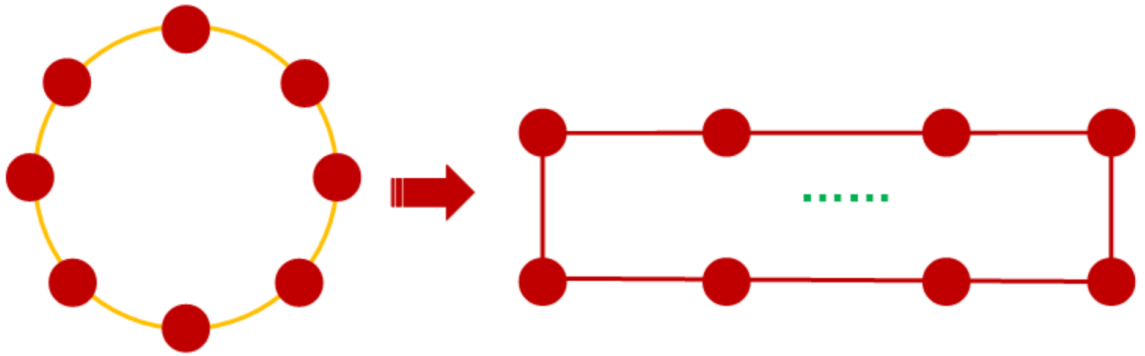
《分布式训练常用的网络结构及集合通信拓扑算法》知乎

All-Reduce for LLM training

- 2D Mesh All-Reduce (for reference)



2D Mesh: removing links connecting the first and the last GPUs of each row and column



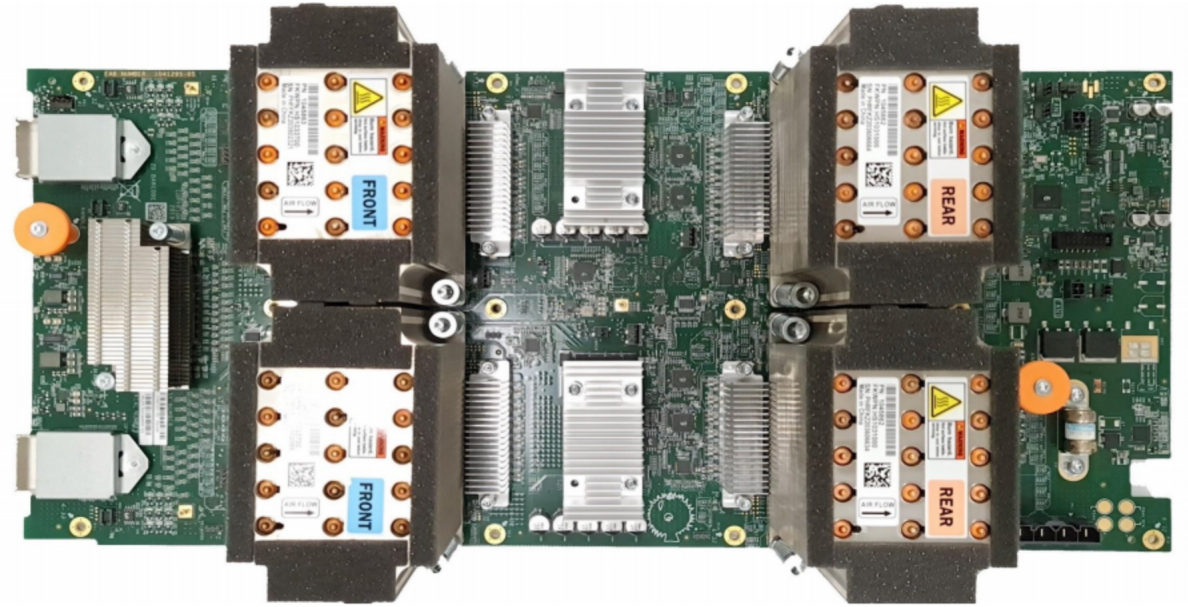
Step 1: Intra-Ring All-Reduce

Step 2: Inter-Ring All-Reduce

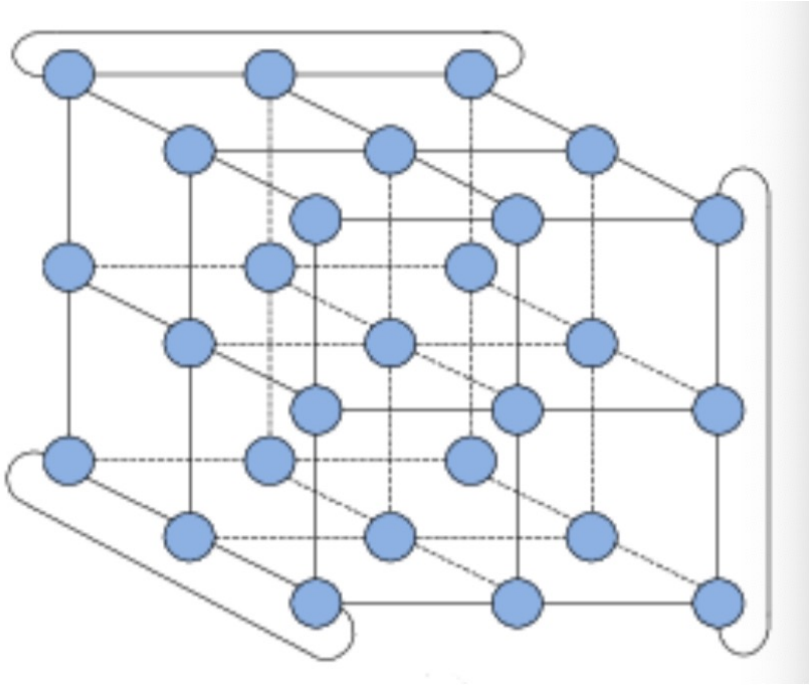
《分布式训练常用的网络结构及集合通信拓扑算法》知乎

All-Reduce for LLM training

- 3D Torus All-Reduce (for reference)



TPUv4i board with 4 chips that are connected by ICI.



3D Torus (3-ary 3-cube)
Initially designed by IBM

5D, 6D Torus in HPC areas

《TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings》